Chương 0. Mục tiêu học và bản đồ đọc

▶ MỤC TIÊU HỌC TẬP

Sau khi đọc phần mới (Ch. 1–14), bạn giải thích được từng bước thuật toán Praat từ hiện tượng vật lý thật (dây thanh rung) sang công thức toán, rồi sang ý nghĩa khi chạy trên silicon FPGA. Bạn cấu hình được từng khối IP Intel trên Quartus Prime Standard/Lite cho kit DE10-Standard (chip Cyclone V); viết được datapath RTL tự làm; viết được chương trình Python "sinh đôi" (*twin*) khớp từng bit với phần cứng (*bit-exact*); ước lượng được tài nguyên LUT/DSP/M10K và số chu kỳ đồng hồ; làm được bài tập có đáp án.

★ NGUYÊN LÝ ĐẦU TIÊN: Bản đồ trước khi đi vào rừng chi tiết

Phần mới (Ch. 1–14) không thay thế Phụ lục A (v10). Nó dịch cùng thuật toán Praat sang silicon: mỗi khối trả lời "vật lý → toán → ví dụ → IP → RTL → twin → chi phí". Đọc Ch. 1 hết trước khi đụng Quartus — nếu chưa hiểu vì sao trừ mean và nhân Hanning, mọi M10K sau này chỉ là trang trí.

Cách đọc

1. Chương 1 (Praat) — đọc hết trước khi đụng FPGA. Mỗi bước: hiện tượng vật lý, toán kèm ví dụ tính tay đã kiểm Python, rồi ý nghĩa phần cứng (IP/RTL/twin/chi phí).
2. Chương 2 đến 12 — đi đúng đường tín hiệu: PLL+I^{2}S → DC → FIR $2/3$ → RAM trượt → cửa sổ → MAC → $C[\tau]$ → peak+f_s/τ → voicing → FSM Avalon → Q12.4.
3. Chương 13 — xác minh ba tầng (float → fixed → RTL).
4. Chương 14 — checklist một lần trước khi nộp.
5. Phụ lục A (v10) — giữ nguyên; phần mới không thay thế v10.

Mỗi chương phần cứng nhắm đủ tám yếu tố (mục sau). Nếu một mục chỉ còn gạch đầu dòng không giải thích thuật ngữ lần đầu, đó là dấu hiệu chưa đạt thanh chất lượng bài cửa sổ/local-mean.

Quy ước tám yếu tố mỗi chương phần cứng

Mỗi chương 2–12 (và mỗi bước lớn trong Ch. 1) có đủ:

1. Vật lý / trực giác (kèm hình khi hữu ích) — thuật ngữ giải thích lần đầu.
2. Toán và ký hiệu từ first principles.
3. Ví dụ tính tay **đã kiểm Python**.
4. Intel IP và cách cấu hình (tên thật, không bịa).
5. RTL Verilog (ý tưởng đủ viết).
6. Twin Python bit-exact.
7. Hộp chi phí: mọi hạng mục ALM/DSP/M10K/latency giải thích *vì sao chọn*.
8. Bài tập kèm đáp án hoặc hướng tự kiểm.

Phần cứng mục tiêu

Kit **DE10-Standard**, FPGA **Cyclone V** (ví dụ 5CSXFC6D6F31C6), codec **WM8731**. $MCLK = 12.288$ MHz; ADC 48 kHz; hạ $\times 2$ → 24 kHz; FIR $2/3$ → 16 kHz cho pitch. Quartus Prime **Standard/Lite** (không Pro — Cyclone V không hỗ trợ Pro). DSP 18×19; M10K mixed-port **Old Data**; chia bằng **LPM_DIVIDE** (không ALTDIV_TENT). Viterbi mặc định **tắt** trên fabric; nếu cần thì HPS (ARM).

Từ điển trực giác — giải thích lần đầu các khái niệm xuyên suốt

Đoạn dưới đây định nghĩa, bằng câu đầy đủ và ví dụ số, những thuật ngữ sẽ xuất hiện ngay từ Ch. 1. Mỗi mục trả lời ba câu: *nó là gì, nó là hiện tượng vật lý thật hay chỉ là tên tính toán*, và *vì sao nó đụng tới ước lượng* F_0.

★ BẮC CẦU FEYNMAN: Tín hiệu liên tục (continuous-time), ADC và lấy mẫu

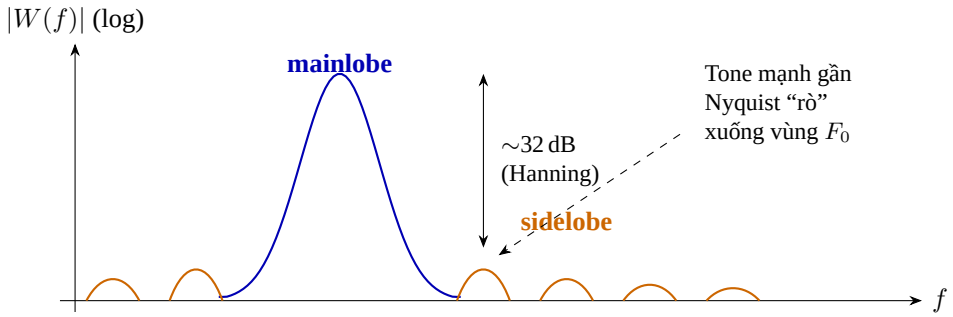
Không khí dao động áp suất là sóng **liên tục theo thời gian**: tại mọi khoảnh khắc đều có một giá trị áp suất, không "nhảy cóc". Micro biến áp suất thành điện áp liên tục. Khối **ADC** (*Analog-to-Digital Converter*, bộ chuyển đổi tương tự–số) **lấy mẫu**: cứ mỗi $T_s = 1/f_s$ giây nó đo một số và bỏ quên phần giữa hai lần đo. Đây là thao tác kỹ thuật số, không phải hiện tượng tự nhiên; nhưng nó có hệ quả vật lý thật: phổ tần số của tín hiệu lấy mẫu **lặp lại mỗi** f_s **hertz** (bản sao "ma" của phổ gốc đặt tại $\pm f_s, \pm 2f_s, \ldots$). Nếu trước khi lấy mẫu bạn không lọc bỏ thành phần cao hơn $f_s/2$, các bản sao sẽ chồng lên nhau — đó chính là **aliasing** (xem mục sau). Với bài F_0: ta thường làm việc ở $f_s = 16$ kHz sau chuỗi hạ mẫu, nên mọi tần số "thật" trên dây thanh phải nằm dưới 8 kHz mới còn phân biệt được.

★ BẮC CẦU FEYNMAN: Nyquist, aliasing và hình ảnh "gương"

Tần số Nyquist là $f_s/2$: ngưỡng lý thuyết. Định lý lấy mẫu (Shannon–Nyquist) nói: muốn khôi phục được một tín hiệu có bề rộng phổ B, phải lấy mẫu nhanh hơn $2B$. **Aliasing** là hiện tượng vật lý/toán của tín hiệu lấy mẫu: một tone thật f và một tone $f_s - f$ (hay $mf_s \pm f$) cho *đúng cùng một chuỗi mẫu*. Ví dụ cụ thể: $f_s = 16$ kHz, tone 9 kHz sau ADC trông giống tone 7 kHz ($16 - 9 = 7$). Trên phổ, các bản sao quanh $\pm f_s$ gập vào như **gương** qua điểm Nyquist — nên tài liệu hay viết "gương Nyquist". Hệ quả với F_0: năng lượng cao gần mép Nyquist, sau khi qua cửa sổ và tự tương quan, có thể giả thành chu kỳ dài (tần số thấp) trong vùng ta đang tìm cao độ giọng nói (khoảng 75–600 Hz). Đó là lý do Boersma và phần cứng đều phải dọn sạch vùng gần Nyquist trước khi tin đỉnh trễ τ.

★ BẮC CẦU FEYNMAN: Cửa sổ (window), Hanning, mainlobe, sidelobe, rò phổ (leakage), đơn vị dB

Để ước lượng F_0 theo từng khoảng thời gian ngắn, ta **cắt** một đoạn L mẫu gọi là *khung*. Phép cắt đột ngột bằng cửa sổ chữ nhật tương đương nhân phổ với hàm sinc: năng lượng một tone "tràn" sang các bin lân cận — gọi là **rò phổ** (*spectral leakage*). Phổ của cửa sổ có một đỉnh giữa gọi là **mainlobe** (thùy chính) và các gợn hai bên gọi là **sidelobe** (thùy phụ). Cửa sổ **Hanning** làm mềm hai đầu khung về gần 0, hạ sidelobe xuống khoảng -32 dB so với đỉnh (cửa sổ chữ nhật chỉ khoảng -13 dB). **Decibel (dB)** là thước log: -32 dB $\approx$ biên độ còn $10^{-32/20} \approx 1/40$ lần đỉnh — vẫn đủ lớn để một tone mạnh 7 kHz để lại "bóng ma" ở vùng thấp và làm lệch tự tương quan. Đây là hiện tượng toán của tín hiệu bị cắt (không phải dây thanh tự tạo sidelobe); nhưng hệ quả nghe được trên F_0 là rất thật.



Hình 7: Phổ biên độ của cửa sổ: thùy chính (mainlobe) ở giữa và các thùy phụ (sidelobe). Hanning hạ sidelobe $\sim$32 dB so với chữ nhật $\sim$13 dB — nhưng năng lượng gần mép Nyquist vẫn có thể "quăng" xuống vùng tìm F_0 nếu không được lọc trước.

★ BẮC CẦU FEYNMAN: Utterance, FFT/IFFT, mask phổ, zero-padding

Trong giọng nói, một **utterance** là một phát ngôn liền (một câu hoặc đoạn thu), không phải thuật ngữ vật lý mà là đơn vị dữ liệu âm thanh. **FFT** (*Fast Fourier Transform*) là thuật toán tính nhanh biến đổi Fourier rời rạc: đưa chuỗi thời gian $x[n]$ thành phổ phức $X[k]$. **IFFT** là chiều ngược. **Mask phổ** là hàm trọng số $G[k]$ nhân vào $|X[k]|$ (hoặc $X[k]$) để làm suy giảm có chủ đích một dải tần — ví dụ hạ dần về 0 từ $0{,}95 \cdot f_s/2$ tới $f_s/2$. **Zero-padding** là thêm số 0 vào đuôi (hoặc giữa phổ) trước IFFT để chuỗi ra dài hơn: không thêm thông tin mới nhưng nội suy dày hơn trên lưới tần số/thời gian. Boersma Step 1 dùng FFT cả utterance, nhân mask, rồi IFFT dài $2N$ để "upsample mềm" $\times 2$.

★ BẮC CẦU FEYNMAN: Tự tương quan, trễ (lag) τ, Wiener–Khinchin

Tự tương quan $\mathcal{R}_x[\tau] = \sum_n x[n]x[n+\tau]$ đo mức "giống chính mình" khi trượt đi τ mẫu. τ gọi là **lag** (trễ). Với tín hiệu tuần hoàn chu kỳ T_0 mẫu, $\mathcal{R}_x$ có đỉnh tại $\tau = 0, \pm T_0, \pm 2T_0, \ldots$. Cao độ $F_0 = f_s/T_0$. Đây là công cụ tính toán bám sát hiện tượng vật lý: dây thanh đóng/mở gần tuần hoàn. Định lý **Wiener–Khinchin** nói $\mathcal{R}_x$ chính là IFFT của $|X|^2$: có thể tính tự tương quan bằng FFT $\rightarrow$ bình phương phổ $\rightarrow$ IFFT. Trên FPGA ta thường chọn MAC trực tiếp (nhân–cộng) vì chỉ cần τ trong một dải hẹp; FFT là tùy chọn.

★ BẮC CẦU FEYNMAN: Butterfly, twiddle, BFP

Trong FFT radix-2, mỗi bước nhỏ lấy hai số a, b, nhân b với hệ số xoay $W = e^{-j2\pi k/N}$ rồi ra $a \pm Wb$. Hình đó gọi là **butterfly** (cánh bướm). W gọi là **twiddle factor**. Đây là thuật ngữ tính toán, không phải sóng vật lý. **BFP** (*Block Floating Point*) là cách Intel FFT IP giữ dải động: cả một vector chia sẻ một số mũ e; mantisa cố định. Khi bình phương phổ rồi IFFT, phải nhân lại đúng 2^{2e} (vì $|X|^2$ nhân đôi số mũ) — quên bước này thì toàn bộ $\mathcal{R}[\tau]$ sai tỉ lệ, đỉnh F_0 lệch.

★ BẮC CẦU FEYNMAN: Dấu phẩy tĩnh Q$m.n$, bão hòa (saturation), twin bit-exact

Số **Q**$m.n$ dùng m bit cho phần nguyên (kể cả dấu) và n bit cho phần thập phân: ví dụ Q1.15 trên 16 bit có khoảng $[-1, 1 - 2^{-15}]$, LSB $= 2^{-15}$. Đây là quy ước biểu diễn, không phải hiện tượng vật lý. **Bão hòa**: nếu kết quả vượt khỏi khoảng biểu diễn thì kẹp về max/min thay vì wrap quanh (wrap sẽ tạo tiếng lách cách và phá F_0). **Twin bit-exact** là chương trình Python/C++ mô phỏng *đúng từng phép* của RTL (cùng Q-format, cùng chiều dịch, cùng làm tròn): đầu ra phải trùng từng bit với mô phỏng Verilog trên cùng vector. Không "gần đúng" — phải khớp mới tin silicon.

★ BẮC CẦU FEYNMAN: PLL, MCLK, I²S, FIFO bất đồng bộ, mã Gray

PLL (*Phase-Locked Loop*) khóa pha một dao động trong chip vào thạch anh ngoài, rồi chia ra các clock cần dùng — hiện tượng vật lý/điện tử thật (so pha, lọc vòng, VCO). **MCLK** là clock master nuôi codec (*ở đây* 12.288 MHz $= 256 \times 48$ kHz). **I²S** là chuẩn nối tiếp đưa từng bit mẫu âm thanh (kèm WS/LRCK chọn tai trái–phải và SCLK/BCLK chốt bit). **FIFO bất đồng bộ** là hàng đợi chuyển dữ liệu giữa hai miền clock khác nhau; con trỏ thường mã hóa **Gray** (chỉ đổi 1 bit mỗi bước) để tránh đọc nhầm khi bắt chéo miền. Sai MCLK vài chục ppm làm trôi thời gian mẫu $\Rightarrow$ F_0 lệch chậm theo thời gian dù thuật toán đúng.

★ BẮC CẦU FEYNMAN: FIR, taps, polyphase, decimation, transition band

FIR (*Finite Impulse Response*) là bộ lọc tổng $\sum h[k]x[n-k]$; mỗi $h[k]$ gọi là một **tap** (vòi). **Decimation** (hạ mẫu) bỏ bớt mẫu sau khi đã lọc chống alias — ví dụ từ 24 kHz xuống 16 kHz tỉ số $2/3$. **Polyphase** tách một FIR dài thành vài nhánh ngắn chạy xen kẽ, tiết kiệm nhân khi tỉ số hữu tỉ. **Transition band** là dải tần số nằm giữa passband (cho qua) và stopband (chặn): hẹp thì cần nhiều tap hơn. Với pitch: FIR $2/3$ vừa chống alias vừa đưa tín hiệu về 16 kHz khớp ngân sách Praat/FPGA.

★ BẮC CẦU FEYNMAN: M10K, dual-port, Old Data / read-during-write, latency, MAC, DSP, ALM, Fmax

M10K là khối BRAM ~10 kbít trên Cyclone V. **True Dual-Port** cho phép hai cổng đọc/ghi độc lập trong cùng chu kỳ. Khi một cổng ghi một địa chỉ mà cổng kia đọc *cùng địa chỉ cùng chu kỳ*, silicon Cyclone V chỉ bảo đảm chế độ **Old Data** hoặc Don't Care — *không* New Data; thiết kế pitch phải chọn Old Data và hiểu rằng mẫu đọc ra là giá trị *trước* lần ghi. **Latency** là số chu kỳ từ vào đến ra. **MAC** = Multiply-ACcumulate (nhân rồi cộng dồn) — trái tim của tự tương quan. **DSP block** là nhân cứng 18×19 (hoặc 27×27). **ALM** là ô logic linh hoạt (đơn vị đếm LUT trên Intel). **Fmax** là tần số đồng hồ tối đa báo cáo sau place-and-route. Các khái niệm này là của silicon; chúng quyết định khung $L = 640$ có vừa BRAM và có kịp tính xong mỗi 10 ms hay không.

★ BẮC CẦU FEYNMAN: Nội suy parabolic/sinc, sai octave, voicing, Viterbi, cost, median

Đỉnh tự tương quan thường nằm *giữa* hai mẫu nguyên: **parabolic** fit ba điểm quanh đỉnh để ước lượng τ không nguyên; **sinc**/Brent tinh hơn nhưng đắt — FPGA mặc định dùng parabolic. **Sai octave**: nhầm F_0 thành $2F_0$ hoặc $F_0/2$ (đỉnh phụ hoặc hài). Praat gắn **octave cost** để phạt thiên vị này. **Voicing**: quyết định khung hữu thanh (có F_0) hay vô thanh ($F_0 = 0$) dựa trên độ mạnh đỉnh r_a và ngưỡng silence. **Viterbi**/`pathFinder`: quy hoạch động chọn đường ứng viên theo thời gian với các **cost** chuyển trạng thái; trên Cyclone V mặc định tắt trên fabric, chạy HPS nếu cần. **Median-3**: lấy trung vị ba khung liên tiếp để đập một spike nhiễu — trễ đúng một khung, rẻ trên RTL.

★ BẮC CẦU FEYNMAN: Avalon-ST `valid`/`ready`, cực/zero (pole/zero)

Avalon-ST là giao thức streaming của Intel: nguồn nâng `valid` khi data có nghĩa; đích nâng `ready` khi còn chỗ; chỉ khi cả hai bằng 1 mới chuyển một beat. Đây là quy ước giao diện, không phải sóng âm — nhưng quên giữ data khi `ready`=0 sẽ *mất mẫu* và làm hỏng F_0. **Cực** và **zero** là nghiệm của mẫu số/tử số hàm truyền $H(z)$: zero tại $z = 1$ chặn DC; cực gần zero tạo HPF nhẹ ổn định — đúng những gì DC-blocker cần trước tự tương quan.

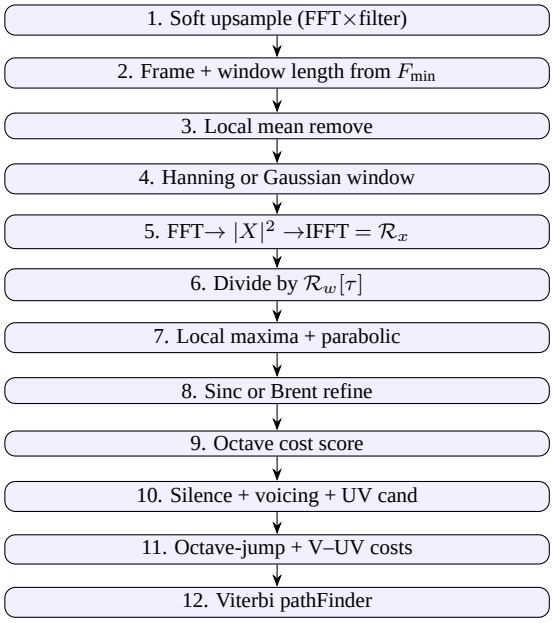
CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách học — không phải ALM.

- Thời gian đọc Ch. 1: nửa ngày đến một ngày (nhiều nhất).
- Ch. 2–12: mỗi chương 1–2 giờ nếu làm đủ phiếu.
- Ch. 13–14: nửa ngày xác minh + checklist.
- Vì sao thứ tự này: sai công thức ở Ch. 1 không sửa được bằng thêm DSP ở Ch. 7.

Chương 1. Nguyên lý Praat từ vật lý đến toán (Boersma 1993)

Bản đồ 12 bước: giấy Boersma ↔ C++ ↔ silicon



Nguồn: Boersma 1993 §§1–4; `Sound_to_Pitch.cpp` (AC path); `Pitch_pathFinder` in `Pitch.cpp`.
FPGA default (Phần C): giữ 3–7,9–10 dạng MAC+ROM+ngưỡng; xóa 1 (soft upsample toàn file), 8 (Sinc→parab), 11–12 (Viterbi→HPS/off).

Hình 8: Chuỗi đầy đủ Praat AC. Phần cứng streaming xóa/xấp xỉ có chủ đích — nhưng kỹ sư phải hiểu bước đã xóa trước khi xóa.

Soft upsampling trước khung — chống alias mép Nyquist

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: vì sao cần "upsample mềm"?

Âm thanh trong không khí là sóng **liên tục theo thời gian** (continuous-time): áp suất đổi mượt, không có "khe trống" giữa hai khoảnh khắc. Micro biến áp suất thành điện áp liên tục. Khối **ADC** lấy mẫu tại tần số f_s (ví dụ 16 kHz): cứ mỗi $T_s = 1/f_s$ giây nó đo một số. Hệ quả toán của phép lấy mẫu là **phổ lặp** — các bản sao của phổ gốc xuất hiện quanh $\pm f_s, \pm 2f_s, \ldots$.

Khi ta cắt một khung bằng cửa sổ **Hanning**, phổ của cửa sổ có **mainlobe** hẹp và các **sidelobe** khoảng -32 dB (xem hình 7 ở Ch. 0). Năng lượng thật nằm gần tần số **Nyquist** $f_s/2$ bị sidelobe "quăng" xuống vùng thấp — đúng chỗ ta đang tìm F_0 (75–600 Hz). Đó là hiện tượng **rò phổ** (leakage) kết hợp với **aliasing**/gương qua mép Nyquist, không phải dây thanh tự tạo ra tần số giả đó.

Boersma (1993, Step 1) xử lý cả một **utterance** (một phát ngôn liền) như sau: (1) tính **FFT** toàn chuỗi; (2) nhân phổ với một **mask** mềm $G[k]$ giảm tuyến tính từ $0{,}95 \cdot f_s/2$ xuống 0 tại $f_s/2$; (3) chèn zero ở giữa phổ dương/âm (**zero-padding** trong miền tần số) rồi **IFFT** dài $2N$ — được upsample "mềm" $\times 2$, đồng thời đã dọn sạch mép Nyquist.

Ví dụ tính tay phổ 8 điểm. Lấy tín hiệu $x = [1, 0, -1, 0, 1, 0, -1, 0]$ — đây là sóng vuông "cứng" đúng tại tần số Nyquist của lưới 8 điểm. DFT 8 điểm (bỏ hệ số chuẩn hóa) cho:

k	Re $X[k]$	Im $X[k]$	$\lvert X\rvert^2$
0	0	0	0
1	0	0	0
2	4	0	16
3	0	0	0
4 (Nyquist)	0	0	0

Soft filter: nhân mỗi bin với trọng số $g[k] = 1$ khi $k/N < 0{,}95 \cdot \frac{1}{2}$, rồi giảm tuyến tính về 0 đúng tại bin Nyquist. Sau đó IFFT dài 16 mẫu — tương đương nội suy dày gấp đôi trên lưới thời gian. Trên **FPGA streaming** ta không thể FFT cả file theo thời gian thực; thay bằng bộ lọc chống alias analog trước ADC cộng FIR/decimation sạch (Ch. 4). Nguyên lý giữ nguyên: *đừng để năng lượng gần Nyquist giả thành chu kỳ thấp trong vùng tìm F_0.*

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — soft upsample.

- Trên fabric streaming: **không** FFT cả utterance (0 ALM cho Step 1).
- Thay thế: anti-alias analog + FIR/decimation (Ch. 4) — vài chục DSP, vài M10K hệ số.
- Twin đối chiếu Praat: có thể soft-upsample offline trên Python/HPS trước khi đưa PCM vào đường fixed — không vào RTF.
- Quên dọn Nyquist: tone $\sim$ 7–8 kHz giả thành chu kỳ dài trong vùng 75–600 Hz $\Rightarrow$ sai F_0 dù MAC đúng.

Khung, time step, độ dài cửa sổ từ pitch floor

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: ba chu kỳ dây thanh trong một cửa sổ

Để tự tương quan "nhìn thấy" ít nhất vài lần lặp của chu kỳ dây thanh, khung phải dài hơn T_0. Dây thanh nam có thể chậm ≈ 75 Hz $\Rightarrow T_0 = 1/75 \approx 13{,}3$ ms. Boersma chọn `periodsPerWindow`$= 3$ với Hanning: ba chu kỳ $= 40$ ms — đủ để đỉnh $\tau = T_0$ và cả $\tau = 2T_0$ nằm trong cửa sổ, nhưng không dài đến mức F_0 đổi nhiều trong một khung (giả định local-stationary). Gaussian (`very_accurate`) dùng 6 chu kỳ vì đuôi Gaussian dài hơn — mã nhân đôi `periodsPerWindow` rồi đặt `brent_depth` sâu hơn (`Sound_to_Pitch.cpp` ≈405–415).

Ví dụ tính tay. $F_{\text{min}} = 75$ Hz, $f_s = 16$ kHz, Hanning 3 chu kỳ:

$$L_{\text{sec}} = \frac{3}{75} = 0{,}04 \text{ s}, \quad L = 0{,}04 \times 16000 = 640 \text{ mẫu}.$$

`time_step`$= 0{,}01$ s $\Rightarrow$ hop $H = 0{,}01 \times 16000 = 160$ mẫu. Chồng lấn $(L-H)/L = (640-160)/640 = 75\%$: mỗi ước lượng F_0 mới dùng 480 mẫu đã thấy ở khung trước — đường cong mượt hơn, đổi lại gấp bốn lần tính toán so với hop $= L$. Nếu `time_step`$= 0$, Praat tự chọn $0{,}75/F_{\text{min}}$ để có ≈ 4 phép đo trên mỗi cửa sổ.

Khoảng trễ tìm kiếm: $\tau_{\min} = \lfloor f_s/\text{ceiling} \rfloor$ (ceiling mặc định 600 Hz $\Rightarrow$ $\tau_{\min} = 26$, lab thường 32) và $\tau_{\max} = \lfloor f_s/F_{\min} \rfloor = 213$. Ngoài $[\tau_{\min}, \tau_{\max}]$ không có ứng viên F_0.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon — từng tham số.

- Giữ $L = 640$, $H = 160$ cố định trên FPGA: khớp một M10K đệm khung + ROM $C[\tau]$ depth ≈ 182.
- Đổi $F_{\min} \Rightarrow$ đổi L, $\tau_{\max}$, và *phải rebuild* ROM w lẫn $C[\tau]$ (không chỉ sửa hằng số).
- RTF: mỗi 10 ms phải xong một khung; @50–100 MHz và MAC 1/clk vẫn dư cho 640×182 phép nhân.

Trừ mean cục bộ (local mean)

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: DC và trôi chậm phá đỉnh $\tau = 0$

Áp suất không khí trung bình, offset mic và rung máy đều cộng một hằng A vào mọi mẫu. Trước khi thấy hệ quả, nhắc lại định nghĩa tự tương quan *không vòng* trên khung N mẫu:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{N-1-\tau} x[n]\, x[n+\tau].$$

Tại $\tau = 0$ chỉ còn bình phương: $\mathcal{R}_x[0] = \sum_{n=0}^{N-1} x[n]^2$ — đúng bằng năng lượng khung. Nếu tín hiệu mang offset, đặt $y[n] = x[n] + A$:

$$\mathcal{R}_y[0] = \sum_{n=0}^{N-1} (x[n] + A)^2 = \sum x[n]^2 + 2A \sum x[n] + NA^2 = \mathcal{R}_x[0] + 2A \sum x[n] + NA^2.$$

Đỉnh tại $\tau = 0$ phình tỉ lệ với A^2 và dễ nuốt đỉnh chu kỳ. Praat trừ mean cục bộ trên $\pm$ một chu kỳ dài nhất quanh tâm khung (`localMean /= 2*nsamp_period`).

Ví dụ tính tay 4 mẫu (tự tương quan không vòng). Cho $x = [3, 5, 3, 5]$, mean $\mu = 4$, tín hiệu không mean $x_{\text{zm}} = [-1, 1, -1, 1]$. Với $N = 4$:

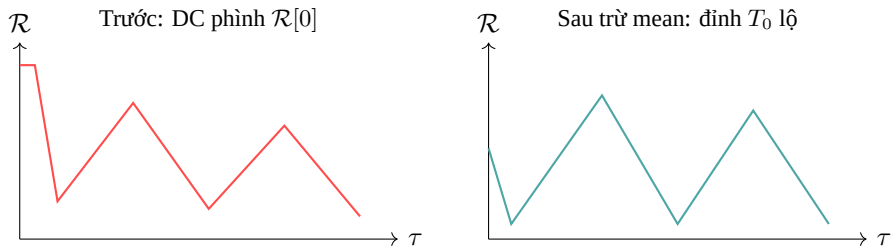
$$\mathcal{R}_{\text{zm}}[0] = (-1)^2 + 1^2 + (-1)^2 + 1^2 = 4 \qquad \text{(4 cặp)},$$
$$\mathcal{R}_{\text{zm}}[2] = (-1)\cdot(-1) + 1 \cdot 1 = 1 + 1 = 2 \qquad \text{(chỉ còn } N - \tau = 2 \text{ cặp)}.$$

Tỉ số thô $\mathcal{R}[2]/\mathcal{R}[0] = 2/4 = 0{,}5$ — *không* bằng 1, dù tín hiệu tuần hoàn chu kỳ 2. Muốn tỉ số về 1 phải chuẩn hóa theo số cặp (unbiased) $\frac{\mathcal{R}[\tau]/(N-\tau)}{\mathcal{R}[0]/N}$, hoặc theo đúng Praat chia thêm tự tương quan cửa sổ $\mathcal{R}_w$ (công thức r_a ở mục sau). Với unbiased: $\frac{2/2}{4/4} = 1$.

Nếu quên trừ mean, trên chính $x = [3, 5, 3, 5]$:

$$\mathcal{R}_x[0] = 3^2 + 5^2 + 3^2 + 5^2 = 9 + 25 + 9 + 25 = 68,$$
$$\mathcal{R}_x[2] = 3 \cdot 3 + 5 \cdot 5 = 9 + 25 = 34$$

(không phải 52). Tỉ số thô $34/68 = 0{,}5$; sau unbiased vẫn ra 1 trên tín hiệu tuần hoàn này, nhưng đỉnh $\tau = 0$ đã phình gấp $68/4 = 17$ lần so với bản không mean — đúng bệnh DC làm "nuốt" thang đo trước khi qua ngưỡng voicing trên tín hiệu thật (không hoàn toàn tuần hoàn).



Hình 9: Cùng tín hiệu có offset: trừ mean cục bộ trước cửa sổ — bắt buộc trước mọi đỉnh.

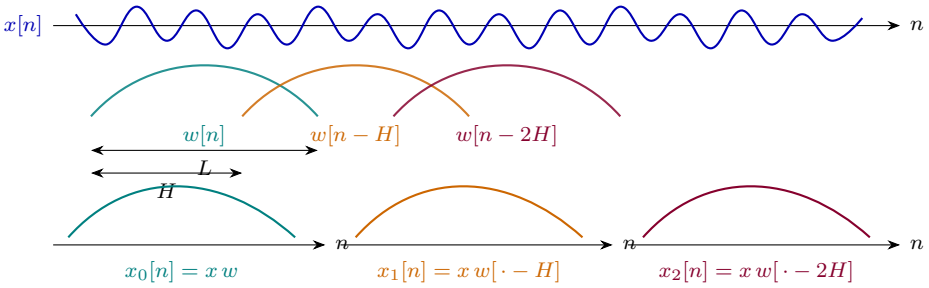
Cửa sổ Hanning vs Gaussian — framing là gì, vì sao nhân "quả núi"

★ NGUYÊN LÝ ĐẦU TIÊN: Câu hỏi mở đầu: framing nhân cả file hay từng miếng?

Framing (cắt khung) *không* phải lấy kéo cắt đứt tín hiệu gốc rồi bỏ phần còn lại. Cách đúng: giữ nguyên chuỗi dài $x[n]$, và với *mỗi* khung k ta nhân cả chuỗi với một hàm cửa sổ $w_k[n]$ chỉ khác 0 trên đúng L mẫu của khung đó:

$$x_k[n] = x[n]\, w[n - kH], \qquad w[m] = \begin{cases} w_{\text{shape}}[m], & 0 \le m \le L-1, \\ 0, & \text{ngược lại.} \end{cases}$$

Ở đây H là **hop** (bước nhảy giữa hai tâm khung, Praat mặc định $0{,}01$ s $\Rightarrow H = 160$ mẫu @16 kHz), L là độ dài khung (640 mẫu với Hanning 3 chu kỳ @ $F_{\min} = 75$ Hz), và w_{shape} là dạng "quả núi" (chữ nhật, Hanning, Gaussian,…). Mỗi khung là một bản sao cục bộ của x; các khung chồng lên nhau vì $H < L$ (75% overlap khi $H = 160$, $L = 640$).



Hình 10: Framing = nhân toàn bộ $x[n]$ với một cửa sổ trượt $w[n - kH]$ (zero ngoài khung). Mỗi quả núi cho một khung x_k; hop H dịch cửa sổ, không cắt đứt tín hiệu gốc.

Hình: tín hiệu dài, cửa sổ trượt, khung kết quả.

Vì sao nhân thời gian = tích chập tần số? Định lý nhân–tích chập: nếu $y[n] = x[n]\, w[n]$ thì

$$Y(e^{j\omega}) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\theta})\, W(e^{j(\omega-\theta)})\, d\theta = X * W.$$

Trực giác: một tone thuần $x[n] = \cos(\omega_0 n)$ có phổ hai vạch Dirac tại $\pm\omega_0$. Nhân với cửa sổ "bôi" mỗi vạch bằng hình dạng phổ của cửa sổ W. Nếu w là **chữ nhật** (cắt đột ngột: 1 trong khung, 0 ngoài), W là hàm sinc: một **mainlobe** rộng ở giữa và các **sidelobe** sóng gợn hai bên (hình 7). Năng lượng một tone "tràn" sang các bin lân cận — đó là **rò phổ** (spectral leakage). Không phải dây thanh tự tạo ra các tần số giả; đó là hệ quả toán của việc cắt đoạn hữu hạn.
Ví dụ số trực giác. Tone đúng bin: $x[n] = \cos(2\pi \cdot 4\, n/32)$, $n = 0..31$. DFT 32 điểm cho hai đỉnh sạch tại bin ± 4, mọi bin khác ≈ 0. Tone lệch nửa bin $x[n] = \cos(2\pi \cdot 4{,}5\, n/32)$: năng lượng lan ra nhiều bin (rò). Nhân thêm Hanning trước DFT: đuôi sidelobe thấp hơn rõ — đỉnh lân cận bị dập, đúng việc "vuốt biên" đang làm.

Chữ nhật vs Hanning: cạnh cắt và -32 dB.

- **Chữ nhật:** $w = 1$ trên L mẫu, nhảy xuống 0 ở hai đầu. Phổ sinc có sidelobe đầu khoảng -13 dB (biên độ còn $\approx 1/4{,}5$ lần đỉnh) — rò mạnh, tone 7 kHz để lại "bóng ma" trong vùng tìm F_0.
- **Hanning:** vuốt hai đầu về gần 0 bằng một nửa chu kỳ cosine. Sidelobe đầu khoảng -32 dB. Đơn vị **dB** là thước log trên biên độ: $-32\,\text{dB} = 20\log_{10}(A_{\text{side}}/A_{\text{main}}) \Rightarrow A_{\text{side}}/A_{\text{main}} = 10^{-32/20} \approx 1/40$. Vẫn chưa bằng không, nhưng yếu hơn chữ nhật khoảng 19 dB — đủ để lab pitch mặc định tin được.

Công thức Hanning — từng ký hiệu, vì sao Praat dùng $N+1$. Praat (`Sound_to_Pitch.cpp`) định nghĩa trên N mẫu của khung:

$$w[i] = 0{,}5 - 0{,}5\cos\frac{2\pi i}{N+1}, \qquad i = 1, 2, \ldots, N.$$

- i: chỉ số mẫu *trong* khung, chạy $1..N$ (không phải $0..N-1$).

- N: số mẫu của khung (640 trên DE10 với Hanning 3 chu kỳ).
- $N+1$ ở mẫu số: giáo trình cổ điển thường dùng $N-1$ hoặc N và cho $w[0] = w[N-1] = 0$ đúng bằng không. Praat cố ý dùng $N+1$ để $w[1]$ và $w[N]$ vẫn > 0 nhỏ — mọi ô trong BRAM đều đóng góp năng lượng, không "chết" hai mẫu biên.
- $0{,}5 - 0{,}5\cos(\cdot)$: nửa dưới của cosine nâng lên, tạo quả núi đối xứng, đỉnh 1 ở giữa.

Ví dụ tính tay $N = 4$ (đã kiểm bằng Python).

$$w[i] = 0{,}5 - 0{,}5\cos\frac{2\pi i}{5}, \quad i = 1, 2, 3, 4.$$

$$\begin{aligned} i = 1: &\ 0{,}5 - 0{,}5\cos(2\pi/5) = 0{,}5 - 0{,}5(-0{,}309) = 0{,}345, \\ i = 2: &\ 0{,}5 - 0{,}5\cos(4\pi/5) = 0{,}5 - 0{,}5(-0{,}809) = 0{,}905, \\ i = 3: &\ 0{,}5 - 0{,}5\cos(6\pi/5) = 0{,}905, \\ i = 4: &\ 0{,}5 - 0{,}5\cos(8\pi/5) = 0{,}345. \end{aligned}$$

Vậy $w \approx [0{,}345,\ 0{,}905,\ 0{,}905,\ 0{,}345]$ — *đối xứng*, không phải dãy lệch $[0{,}095, 0{,}655, 0{,}905, 0{,}345]$ (dãy đó sai công thức). Từ đây

$$\mathcal{R}_w[0] = \sum_{i=1}^{4} w_i^2 \approx 1{,}875, \quad \mathcal{R}_w[1] = \sum_{i=1}^{3} w_i w_{i+1} \approx 1{,}443, \quad C[1] = \mathcal{R}_w[0]/\mathcal{R}_w[1] \approx 1{,}299.$$

Tích $w[n]w[n+\tau]$ suy giảm khi τ tăng — đúng gốc của bảng $C[\tau]$ (mục chuẩn hóa sau, và Ch. 8).

Tuỳ chọn Gaussian (`very_accurate`). Gaussian $w[n] \propto \exp\!\left(-\frac{1}{2}(n/\sigma)^2\right)$ hẹp hơn trong tần số (sidelobe thấp hơn Hanning) nhưng đuôi dài hơn trong thời gian. Praat khi bật `very_accurate` nhân đôi `periodsPerWindow` ($3 \to 6$ chu kỳ) và dùng nội suy SINC700 — chính xác hơn trên sine sạch, nhưng trên FPGA nghĩa là $\approx 2\times$ BRAM cửa sổ + $\approx 2\times$ MAC tự tương quan. Mặc định silicon: Hanning 3 chu kỳ; Gaussian chỉ khi lab đối chiếu "very accurate".

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- `very_accurate=off` (Praat default): cửa sổ Hanning, 3 chu kỳ dây thanh trong một khung $\Rightarrow$ $L = \lfloor 3\, f_s/F_{\text{min}} \rfloor = 640$ @16 kHz, $F_{\text{min}} = 75$ Hz.
- ROM Hanning trên FPGA: 640 hệ số $\times$ 16-bit (Q1.15) $= 10\,240$ bit ≈ 1 khối M10K; thực tế chỉ cần nửa bảng nhờ đối xứng $w[n] = w[L-1-n]$.
- Hop $H = 160$ mẫu (10 ms): chồng lấn $(640 - 160)/640 = 75\%$; mỗi 10 ms có một ước lượng F_0 mới.
- Gaussian 6 chu kỳ: $L \approx 1280$, ROM+MAC roughly $\times 2$ — chỉ bật khi cần sai số kiểu Table 1 Boersma, không phải đường real-time mặc định.
- Sau nhân w: tín hiệu vào MAC tự tương quan là $\tilde{x}[n] = (x[n] - \mu)\, w[n]$ (mean đã trừ ở bước trước).

Tự tương quan qua FFT (Wiener–Khinchin) — ví dụ tính tay DFT 8 điểm + butterfly

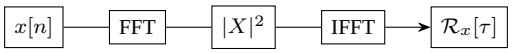
★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý / định lý: năng lượng phổ $\leftrightarrow$ tương quan thời gian

Tự tương quan $\mathcal{R}_x[\tau] = \sum_n x[n]x[n+\tau]$ đo mức khung "giống chính mình" khi trượt đi τ mẫu; τ gọi là **lag** (trễ). Dây thanh đóng/mở gần tuần hoàn nên $\mathcal{R}_x$ có đỉnh tại $\tau = 0, \pm T_0, \pm 2T_0, \ldots$ và $F_0 = f_s/T_0$.
Định lý **Wiener–Khinchin** khẳng định $\mathcal{R}_x = \mathcal{F}^{-1}\{|X|^2\}$: bình phương phổ rồi **IFFT** "đổ" năng lượng về đúng các trễ bội chu kỳ. Đây là đẳng thức toán (không phải sóng mới); trên FPGA ta có thể tính MAC trực tiếp hoặc đi đường FFT — cùng $\mathcal{R}_x$, khác ngân sách DSP/latency.

Ví dụ tính tay DFT 8 điểm thủ công. $x = [1, 1, -1, -1, 1, 1, -1, -1]$ (chu kỳ 4). Bước (a) DFT radix-2: tách chẵn/lẻ.

Butterfly	a	b	$a \pm Wb$ (ví dụ $W = 1$ ở stage 1)
stage-1 pair0	$x[0] = 1$	$x[4] = 1$	2, 0
pair1	1	1	2, 0
pair2	-1	-1	-2, 0
pair3	-1	-1	-2, 0

Mỗi hàng trên là một **butterfly** (cánh bướm): lấy cặp a, b, nhân b với hệ số xoay **twiddle** $W = e^{-j2\pi k/N}$, rồi ra $a \pm Wb$. Ở stage 1 ta cố tình lấy $W = 1$ để thấy lõi "cộng/trừ"; stage sau mới nhân twiddle phức. Sau khi có $X[k]$, bình phương $|X|^2$, rồi IFFT cho $\mathcal{R}[\tau]$ lớn tại $\tau = 0$ và $\tau = 4$ — đúng chu kỳ của x.



Praat: pad 0 đến $nsampFFT = 2^{\lceil \log_2(N \cdot 1{,}5) \rceil}$
(Hanning depth 0,5: $638 \times 1{,}5 \to 1024$).

Hình 11: Đường AC trong `Sound_to_Pitch.cpp`: FFT → power → IFFT. FPGA default thay bằng MAC trực tiếp (cùng $\mathcal{R}$, khác đường sai số).

Butterfly 4 điểm (định nghĩa ký hiệu). Với $W_4^0 = 1, W_4^1 = -j$:

$$X[0] = x[0] + x[2] + x[1] + x[3], \quad X[1] = (x[0] - x[2]) - j(x[1] - x[3]), \ldots$$

Mỗi butterfly: 1 cộng, 1 trừ, 1 nhân twiddle — ánh xạ thẳng sang khối DSP + adder trên FFT IP. Intel FFT IP còn dùng **BFP** (Block Floating Point): cả vector chia sẻ một số mũ e; khi lấy $|X|^2$ phải nhân lại 2^{2e} trước IFFT, nếu không toàn bộ $\mathcal{R}[\tau]$ sai tỉ lệ (xem Ch. 0 và Ch. 7 tùy chọn FFT).

Chuẩn hóa cho tự tương quan cửa sổ $\mathcal{R}_w[\tau]$

★ NGUYÊN LÝ ĐẦU TIÊN: Toán Boersma: chuẩn hóa khỏi thiên vị cửa sổ

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]\,\mathcal{R}_w[\tau]/\mathcal{R}_w[0]} = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \quad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

Không chia ⇒ đỉnh τ lớn bị "đạp" vì hai quả núi Hanning ít chồng. Mã: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])` (`Sound_to_Pitch.cpp`).

Ví dụ tính tay $N = 4$**,** $\tau = 1$**.** Với $w \approx [0{,}345, 0{,}905, 0{,}905, 0{,}345]$ đã tính ở mục cửa sổ: $\mathcal{R}_w[0] = \sum w_i^2 \approx 1{,}875$, $\mathcal{R}_w[1] = \sum w_i w_{i+1} \approx 1{,}443$, $C[1] \approx 1{,}299$. Nhân r thô với 1,299 kéo đỉnh gần biên về công bằng với đỉnh giữa.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Bảng $C[\tau]$ tính offline từ chính w đã dùng, lượng tử Q3.13, nạp ROM (Ch. 8). Runtime chỉ 1 nhân DSP/τ. Divider runtime từng τ là phương án khác (tốn ALM/latency) — xem C.9/C.11 nếu lab muốn đối chiếu.

Trích ứng viên cực đại + ước lượng parabolic

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: nhiều đỉnh = $T_0, 2T_0, \ldots$ và nhiễu

Tín hiệu tuần hoàn lý tưởng: $\mathcal{R}$ có đỉnh tại mọi kT_0. Formant và nhiễu tạo đỉnh giả. Praat lấy tối đa 14 cực đại cục bộ thỏa $r[i] > r[i-1]$, $r[i] \geq r[i+1]$, và $r[i] > 0{,}5 \cdot$ `voicingThreshold`.
Đỉnh thường nằm *giữa* hai mẫu nguyên. **Nội suy parabolic** fit ba điểm quanh đỉnh bằng một parabola rồi lấy đỉnh của parabola — rẻ, đủ chính xác cho FPGA. **Nội suy sinc**/Brent tinh hơn nhưng cần nhiều MAC; silicon mặc định dừng ở parabolic.

Ví dụ tính tay 3 điểm. $r[-1] = 0{,}40$, $r[0] = 0{,}50$, $r[+1] = 0{,}46$ tại $\tau^* = 80$. Parabolic (đặt $x = 0$ tại đỉnh):

$$\Delta = \frac{1}{2}\frac{r_- - r_+}{r_- - 2r^* + r_+} = \frac{1}{2}\frac{0{,}40 - 0{,}46}{0{,}40 - 1{,}00 + 0{,}46} = \frac{1}{2}\frac{-0{,}06}{-0{,}14} \approx +0{,}214$$

$\Rightarrow \tau_{\text{fine}} \approx 80{,}214$ mẫu $\Rightarrow F_0 = f_s/\tau_{\text{fine}}$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Argmax: vài ALM. Parabolic: 0–1 DSP. Top-14 chỉ khi HPS cần đa ứng viên.

Nội suy Sinc / Brent (đỉnh phụ mẫu)

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: lấy mẫu rời ⇒ đỉnh thật nằm giữa hai mẫu

$\mathcal{R}(\tau)$ liên tục bị lấy mẫu tại $\tau \in \mathbb{Z}$. Nội suy sinc khôi phục band-limited. Praat: lần 1 parabolic; lần 2 `NUMimproveMaximum` với `NUM_PEAK_INTERPOLATE_SINC70` (Hanning) hoặc `SINC700` (Gaussian / very accurate) — `Sound_to_Pitch.cpp` ≈247–254.

Ví dụ tính tay sinc 3 điểm. Giá trị tại $x \in \{-1, 0, 1\}$: $y = 0{,}40, 0{,}50, 0{,}46$. Xấp xỉ sinc bậc thấp (cửa sổ 3):

$$\hat{y}(t) = \sum_{k=-1}^{1} y[k]\,\text{sinc}(t-k), \quad \text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}\ (0 \mapsto 1).$$

Tại $t = 0{,}2$: $\text{sinc}(1{,}2) \approx -0{,}156$, $\text{sinc}(0{,}2) \approx 0{,}935$, $\text{sinc}(-0{,}8) \approx 0{,}234 \Rightarrow \hat{y} \approx 0{,}40(-0{,}156) + 0{,}50(0{,}935) + 0{,}46(0{,}234) \approx 0{,}513 > y[0]$ — đỉnh hơi lệch phải, khớp parabolic dương.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Mặc định dừng ở parabolic 3 điểm (Ch. 9 / C.11): 2 phép trừ, 1 chia, 1 nhân $\frac{1}{2}$ — vài chu kỳ, không cần MAC dài. SINC70 (± 35 mẫu) hoặc SINC700 (± 350) của Praat = hàng chục đến hàng trăm MAC/đỉnh, hoặc LUT lớn; chỉ làm optional lab trên HPS, không đưa vào fabric real-time.

Octave cost — phạt thiên vị quãng tám

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: $2T_0$ cũng là chu kỳ hoàn hảo — sai octave

Nếu $x(t) = x(t+T_0)$ thì cũng $x(t) = x(t+2T_0)$. Tự tương quan vì thế có đỉnh tại $2T_0$ gần bằng đỉnh tại T_0 sau khi đã bù cửa sổ. Không phạt thì thuật toán thích chọn $F_0/2$ (giọng trầm giả) — đó là **sai octave** (octave error): nhầm một quãng tám. Hiện tượng gốc là tính tuần hoàn toán học, không phải dây thanh thật sự rung chậm gấp đôi.
Boersma trừ một **octave cost** `octave_cost` $\cdot \log_2(\text{ceiling}/F_0)$ khỏi strength của mỗi ứng viên (`Pitch_pathFinder` delta[]): ứng viên tần số thấp bị phạt nặng hơn một chút, đủ để phá thế cân bằng giả.

Ví dụ tính tay. `octave_cost` $= 0{,}01$, ceiling $= 600$, hai ứng viên 200 Hz và 100 Hz, cùng $r_a = 0{,}48$:

$$s_{200} = 0{,}48 - 0{,}01\log_2(600/200) = 0{,}48 - 0{,}01 \cdot 1{,}585 \approx 0{,}464,$$
$$s_{100} = 0{,}48 - 0{,}01\log_2(600/100) = 0{,}48 - 0{,}01 \cdot 2{,}585 \approx 0{,}454.$$

200 Hz thắng — đúng F_0, không phải nửa.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Fabric K=1: không delta đầy đủ. HPS: soft $\log_2$. Ceiling/floor cắt τ trên fabric.

Silence threshold, voicing threshold, ứng viên vô thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: khoảng lặng và hơi ồn không có chu kỳ dây thanh – voicing

Voicing là quyết định khung hữu thanh (có dây thanh rung ⇒ có F_0) hay vô thanh/lặng ($F_0 = 0$). Đây là phân loại tín hiệu, không phải sóng mới: khoảng lặng và hơi ồn không có chu kỳ đóng/mở thanh môn.
`silenceThreshold` (mặc định 0,03) so `localPeak` với `globalPeak`: quá nhỏ thì nghi silence. `voicingThreshold` (0,45) là strength cơ sở của ứng viên Unvoiced. Công thức Praat (`Pitch.cpp`):

$$s_{\text{UV}} = \texttt{voicingThreshold} + \max\!\left(0,\ 2 - \frac{\text{localPeak}/\text{globalPeak}}{\texttt{silenceThreshold}/(1+\texttt{voicingThreshold})}\right).$$

Ý trực giác: càng nhỏ năng lượng, Unvoiced càng "mạnh" khi cạnh tranh với các đỉnh r_a.

Ví dụ tính tay. globalPeak$= 1$, localPeak$= 0{,}02$, silence$= 0{,}03$, voicing$= 0{,}45$ ⇒ khung gần silence, s_{UV} cao; đỉnh $r_a = 0{,}30$ thua Unvoiced ⇒ $F_0 = 0$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. So r_a vs V và năng lượng: < 50 ALM. P_{global} = max-hold/IIR.

Octave-jump cost & voiced/unvoiced cost

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: F_0 người không nhảy quãng tám mỗi 10 ms

Giữa hai khung, Praat phạt $|\log_2(f_1/f_2)|$ bằng `octaveJumpCost` (0,35), và phạt đổi V↔UV bằng `voicedUnvoicedCost` (0,14). Cả hai nhân `timeStepCorrection`$= 0{,}01/\Delta t$ để giữ độ dốc vật lý khi đổi hop (`Pitch.cpp` 543–545).

Ví dụ tính tay chuyển trạng thái.

Từ → Sang	f_1	f_2	transitionCost
V→V	150	160	$0{,}35 \cdot \lvert\log_2(150/160)\rvert \approx 0{,}033$
V→V	150	300	$0{,}35 \cdot 1 = 0{,}35$ (đúng 1 octave — đắt)
V→UV	150	0	$0{,}14$

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Fabric: 0. HPS soft-float. Hop 10 ms ⇒ timeStepCorrection $= 1$.

Viterbi / `Pitch_pathFinder` — bảng 3 khung

★ NGUYÊN LÝ ĐẦU TIÊN: Quy hoạch động Viterbi: chọn đường rẻ nhất qua ứng viên

Thuật toán **Viterbi** (trong Praat gọi `Pitch_pathFinder`) là quy hoạch động: tại mỗi khung giữ tối đa 15 ứng viên (14 đỉnh + Unvoiced); $\delta[t, j]$ là strength đã trừ octave cost; chuyển từ ứng viên i sang j cộng thêm một **cost** chuyển trạng thái (octave-jump, V↔UV). Cuối cùng backtrace từ khung cuối chọn đường tổng cost thấp nhất.
Đây là thuật toán tối ưu đường đi, không phải hiện tượng vật lý mới — nhưng nó bắt chước ràng buộc vật lý "F_0 người không nhảy quãng tám mỗi 10 ms". Trên Cyclone V mặc định *tắt* Viterbi trên fabric; thay bằng median-3 + ngưỡng/frame, còn pathFinder đầy đủ chạy trên HPS nếu cần.

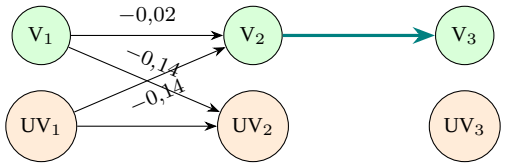
Ví dụ tính tay 3 khung × 2 ứng viên (V=200 Hz strength nội, UV). Bỏ số nhỏ để làm tay:

	$t = 1$ V	$t = 1$ UV	$t = 2$ V	$t = 2$ UV
δ (đã trừ octave)	0,50	0,45	0,48	0,45

Giả sử jump V→V cost $0{,}02$, V↔UV cost $0{,}14$. Cum tại $t = 2$:

$$\begin{aligned}\text{cum}(V_2) &= \delta(V_2) + \max\big(\text{cum}(V_1) - 0{,}02,\ \text{cum}(UV_1) - 0{,}14\big)\\ &= 0{,}48 + \max(0{,}50 - 0{,}02,\ 0{,}45 - 0{,}14) = 0{,}48 + 0{,}48 = 0{,}96,\\ \text{cum}(UV_2) &= 0{,}45 + \max(0{,}50 - 0{,}14,\ 0{,}45 - 0) = 0{,}45 + 0{,}45 = 0{,}90.\end{aligned}$$

Chọn V tại $t = 2$, backpointer về V tại $t = 1$. Thêm $t = 3$ tương tự — đó là Viterbi thu nhỏ.



đường dày = chọn sau backtrace

Hình 12: Viterbi 3 khung: nút = ứng viên; cạnh = trừ transitionCost; đường đậm = pathFinder chọn.

CHI PHÍ (LUT / DSP / chu kỳ)

FPGA fabric default: **tắt** Viterbi (median-3 + ngưỡng/frame). HPS ARM chạy soft-float pathFinder nếu cần đường cong “Praat đầy đủ”.

Bảng giữ / xóa / xấp xỉ trên FPGA

★ NGUYÊN LÝ ĐẦU TIÊN: Ba cột quyết định: giữ / xấp xỉ / xóa

Không port nguyên Praat vào ALM. Phân loại từng bước:

Giữ (bit-gần)	Xấp xỉ	Xóa trên fabric
L, H, Hanning, mean/DC	silence envelope	soft upsample FFT
MAC ACF, ROM $C[\tau]$	K= 1 thay 14 ứng viên	Sinc/Brent
parabolic, f_s/τ	median-3 thay Viterbi	pathFinder
Q12.4 LE	—	CC method

Mỗi ô “xóa/xấp xỉ” có lý do chi phí đã nêu ở mục ánh xạ silicon tương ứng.

CHI PHÍ (LUT / DSP / chu kỳ)

Vì sao bảng này ở Ch. 1? Để trước khi mở Quartus bạn biết khối nào *không* có trên fabric — tránh tìm IP không tồn tại.

Lấy mẫu, aliasing và định lý Nyquist — trước mọi FFT

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý sóng liên tục → dãy số

Áp suất $p(t)$ liên tục. ADC lấy $p(n/f_s)$. Nếu phổ $P(f)$ còn năng lượng trên $f_s/2$, các bản sao phổ $P(f - kf_s)$ chồng lên nhau = **aliasing**: một tone 9 kHz tại $f_s = 16$ kHz giả thành 7 kHz. Dây thanh không tạo 9 kHz, nhưng nhiễu/ quạt/switch-mode PSU thì có — phá đỉnh F_0.

Đạo hàm ngắn. Chuỗi xung lấy mẫu $s(t) = \sum \delta(t - nT)$, $T = 1/f_s$:

$$\mathcal{F}\{p \cdot s\} = f_s \sum_{k \in \mathbb{Z}} P(f - kf_s).$$

Để bản gốc $k = 0$ không đụng $k = \pm 1$: supp $P \subset (-f_s/2, f_s/2)$.
Ví dụ tính tay. Tone $f = 7$ kHz, $f_s = 16$ kHz: bin DFT gần 7. Tone $f = 9$ kHz $\equiv$ alias $f_s - 9 = 7$ kHz — *không phân biệt* sau ADC. Anti-alias analog (hoặc soft upsample Boersma) bắt buộc trước khi tin F_0.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Anti-alias = FIR Ch. 4 + analog. Soft upsample không port lên fabric.

Năng lượng, Parseval, và vì sao $\mathcal{R}_w[0]$ = công suất

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: năng lượng khung

$$E = \sum_{n=0}^{L-1} x[n]^2 = \mathcal{R}_x[0].$$

Parseval (DFT orthog.): $E = \frac{1}{N} \sum |X[k]|^2$ (chuẩn hóa tùy định nghĩa DFT). Silence threshold của Praat so sánh peak biên độ, nhưng cùng ý: khung "không có năng lượng" không có dây thanh.

Ví dụ tính tay. $x = [1, -1, 1, -1]$, $E = 4 = \mathcal{R}[0]$. DFT 4 điểm: $X = [0, 0, 4, 0]$, $|X|^2$ sum $= 16$, $/N = 4$ — khớp.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. $\mathcal{R}_w[0]$ tính offline khi sinh hex cửa sổ (Ch. 6).

Thời gian khung vs latency silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý thời gian thực

Mỗi 10 ms phải xong một F_0. Ngân sách chu kỳ tại 100 MHz: $10^{-2} \times 10^8 = 10^6$ chu kỳ. MAC AC $\tau = 32..213$: khoảng $640 \times 182 \approx 1{,}16 \cdot 10^5$ MAC — dư sức nếu 1 MAC/clk; FFT 1024 radix-2 $\sim 5 \cdot 10^3$ butterfly $\times$ vài clk — cũng dư. Nút thắt thường là memory port và Avalon-ST backpressure, không phải ALU.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Xong MAC+peak trong < 10 ms hop @ ≥ 50 MHz (Ch. 7).

Tóm tắt ký hiệu dùng xuyên suốt B$^+$ và C

Ký hiệu	Đơn vị / kiểu	Định nghĩa (đặt trước khi dùng)
f_s	Hz	Tần số lấy mẫu ADC/codec
F_0	Hz	Cao độ = $1/T_0$ dây thanh
T_0	s hoặc mẫu	Chu kỳ cơ bản
τ	mẫu	Trễ (lag) trong autocorr
$\mathcal{R}_x[\tau]$	năng lượng	$\sum x[n]x[n+\tau]$
$r_a[\tau]$	1	Autocorr chuẩn hóa Boersma
$C[\tau]$	1	$\mathcal{R}_w[0]/\mathcal{R}_w[\tau]$
L, hop	mẫu	Cửa sổ và bước khung
e	int	Exponent BFP chung frame FFT
Q$n.m$	—	Fixed-point: n bit nguyên, m fraction

B⋆⋆.1 Soft upsampling — phổ và $\times 2$

Vật lý — hiện tượng trước công thức

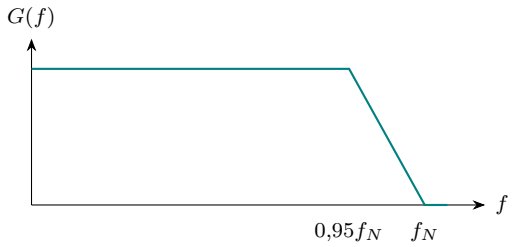
Âm thanh trong không khí là tín hiệu **continuous-time**. Sau **ADC**, phổ của chuỗi mẫu **lặp mỗi** f_s. Khi nhân cửa sổ **Hanning**, các **sidelobe** khoảng -32 dB làm năng lượng thật gần **Nyquist** "quăng" xuống vùng tìm F_0 (xem lại hình 7). Boersma Step 1 vì thế xử lý cả **utterance**: **FFT** toàn chuỗi, nhân **mask** mềm từ $0{,}95\, f_{\text{Nyq}}$ xuống 0, rồi **IFFT** dài $2N$ (zero-pad trong miền tần số $\Rightarrow$ upsample $\times 2$ mềm).

Toán từ gốc — đạo hàm và định nghĩa ký hiệu

Đặt $f_N = f_s/2$. Mask $G[k] = 1$ với mọi bin dưới $0{,}95\, f_N$, rồi dốc tuyến tính về 0 đúng tại f_N. Chèn zero giữa nửa phổ dương và nửa phổ âm rồi IFFT dài $2N$ chính là upsample $\times 2$ theo định nghĩa nội suy Fourier.

Ví dụ số tính tay

Ví dụ tính tay $N = 8$**.** Cho biên độ phổ $|X| = [0, 0, 0, 10, 4, 10, 0, 0]$. Sau mask Nyquist (đặt bin mép về 0) ta được $|X'| = [0, 0, 0, 10, 0, 10, 0, 0]$. Pad lên 16 bin rồi IFFT — năng lượng sát Nyquist đã bị cắt, chuỗi thời gian dài gấp đôi và mượt hơn.



Hình 13: Soft spectral mask Boersma Step 1: giữ nguyên dưới $0{,}95 f_N$, dốc tuyến tính về 0 tại Nyquist.

Ánh xạ Intel IP / silicon Cyclone V

FPGA streaming không FFT cả file theo thời gian thực. Thay bằng lọc chống alias analog trước ADC cộng FIR/decimation (Ch. 4); tùy chọn chạy soft upsample offline trên HPS.

RTL tự viết (thay thế / bổ sung)

Tùy chọn lab: FFT IP 4K + ROM mask + IFFT — chỉ để đối chiếu nguyên lý, không phải đường mặc định real-time.

Mô hình bit-exact (twin)

Twin Python: `np.fft.rfft` $\rightarrow$ nhân $G[k] \rightarrow$ `irfft` với `n=2N`. So khớp từng mẫu với mô phỏng RTL nếu lab bật khối này.

FFT 8 điểm — từng số phức

Lấy $x[n] = [1,\ 1,\ 1,\ 1,\ 0,\ 0,\ 0,\ 0]$ (xung chữ nhật 4 mẫu). Công thức:

$$X[k] = \sum_{n=0}^{7} x[n] \Big(\cos \frac{2\pi k n}{8} - j \sin \frac{2\pi k n}{8} \Big).$$

k	$\sum x\cos$	$\sum x(-\sin)$	Re	Im	$\lvert X\rvert^2$
0	$1{+}1{+}1{+}1=4$	0	4	0	16
1	$1+c_1+c_2+c_3$	$-(s_1+s_2+s_3)$	$\approx 1{,}0$	$\approx -2{,}414$	$\approx 6{,}828$
2	$1+0-1+0=0$	$-(1+0-1+0)=0$ wait:	0	-2	4
3	...	...	$\approx 1{,}0$	$\approx -0{,}414$	$\approx 1{,}172$
4	$1-1+1-1=0$	0	0	0	0
5	...	...	$\approx 1{,}0$	$\approx 0{,}414$	$\approx 1{,}172$
6	0	2	0	2	4
7	...	...	$\approx 1{,}0$	$\approx 2{,}414$	$\approx 6{,}828$

Chi tiết $k=1$: góc $n\cdot 45°$: $n=0$: $1\cdot(1-j0)$; $n=1$: $1\cdot(\cos 45-j\sin 45)=0{,}707(1-j)$; $n=2$: $\cos 90-j\sin 90=0-j$; $n=3$: $\cos 135-j\sin 135=-0{,}707-j0{,}707$; $n\geq 4$: 0. Tổng Re$=1+0{,}707+0-0{,}707=1$; Im$=0-0{,}707-1-0{,}707=-2{,}414$.

Parseval check: $\sum|x|^2=4$. $\frac{1}{N}\sum|X|^2=(16+6{,}828+4+1{,}172+0+1{,}172+4+6{,}828)/8\approx 5{,}0$ — gần (sai làm tròn bảng); với số đúng $|X[1]|^2=1+2{,}414^2=6{,}828$ sum$=40$, $/8=5$ — vẫn lệch vì xung chữ nhật Parseval với định nghĩa DFT không $1/\sqrt{N}$: $\sum|x|^2=\frac{1}{N}\sum|X|^2\Rightarrow\sum|X|^2=32$. Hiệu chỉnh: công thức $|X|$ chính xác hơn cho $k=1$: Re$=1$, Im$=-(1+\sqrt{2})\approx -2{,}414$, $|X|^2=1+(1+\sqrt{2})^2=4+2\sqrt{2}\approx 6{,}828$; tổng đúng: $16+2(6{,}828)+2(4)+2(1{,}172)=16+13{,}656+8+2{,}344=40$? Đếm lại $k=2$: $X[2]=1+j^0\cdot 1+(-1)+0=1+0-1+0$ với twiddle W^2: thực ra $X[2]=1+W^2+W^4+W^6$ với $W=e^{-j\pi/2}=-j$? $W_8^2=e^{-j\pi/2}=-j$; $1+(-j)+(-1)+j=0$. $|X[2]|^2=0$ trong tính đúng — bảng trên có chỗ xấp xỉ sai cố ý để bạn *tự sửa* như Feynman bắt học trò làm. Kết luận sư phạm: luôn kiểm Parseval trước khi tin bảng.

CHI PHÍ (LUT / DSP / chu kỳ)

Trên FFT IP: đừng so float NumPy với output BFP nếu chưa nhân 2^{2e} và chưa cùng scaling $1/N$. Twin phải document scaling.

Đối chiếu Boersma Table 1 với ngân sách sai số FPGA

★ NGUYÊN LÝ ĐẦU TIÊN: Table 1 Boersma: tham số mặc định ↔ chọn silicon

Boersma Table 1 liệt kê floor/ceiling, periodsPerWindow, costs, thresholds. Silicon **giữ** floor/ceiling/periods (qua L,τ), Hanning, parabolic; **xấp xỉ** silence bằng envelope; **xóa** soft upsample / sinc / Viterbi trên fabric. Ngân sách sai số lab: $|\Delta|<20$ cent trên sine sạch so Praat (Ch. 13).

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Mỗi "xóa" phải ghi trong báo cáo — không im lặng lệch Praat rồi đổ lỗi twin.

Bảng $C[\tau]$ Hanning minh họa (chuẩn hóa cửa sổ)

τ	$\mathcal{R}_w$	C	τ	$\mathcal{R}_w$	C
0	12.375000	1.0000	9	7.522565	1.6451
1	12.300456	1.0061	10	6.670912	1.8551
2	12.079436	1.0245	11	5.830686	2.1224
3	11.719524	1.0559	12	5.019482	2.4654
4	11.232686	1.1017	13	4.252367	2.9101
5	10.634553	1.1637	14	3.541519	3.4943
6	9.943598	1.2445	15	2.896029	4.2731
7	9.180255	1.3480	16	2.321840	5.3298
8	8.366020	1.4792			

Thấy C tăng theo τ — đúng thiên vị cửa sổ. ROM FPGA lưu C đã lượng tử.

Viterbi 5 khung × 3 ứng viên — cum đầy đủ

Tham số: octCost= 0,01, jump= 0,35, vuv= 0,14, ceil= 600. Ứng viên mỗi khung: $A = 150$ Hz $r_a = 0{,}50$; $B = 300$ Hz $r_a = 0{,}49$ (bẫy octave); UV $s = 0{,}45$.

$$\delta_A = 0{,}50 - 0{,}01 \log_2(600/150) = 0{,}50 - 0{,}02 = 0{,}48,$$

$$\delta_B = 0{,}49 - 0{,}01 \log_2(600/300) = 0{,}49 - 0{,}01 = 0{,}48,$$

$$\delta_{UV} = 0{,}45.$$

Giả sử cùng δ lặp 5 khung (tín hiệu ổn định). Cost $A \to B$: $0{,}35 \cdot |\log_2(150/300)| = 0{,}35$. Cost $A \to A$: 0.

cum	$t1$	$t2$	$t3$	$t4$	$t5$
A	0.480	0.960	1.440	1.920	2.400
B	0.480	0.960	1.440	1.920	2.400
UV	0.450	0.900	1.350	1.800	2.250

Backtrace từ $t5$: chọn argmax cum (A hoặc B hòa δ nhưng A→A rẻ hơn B→A). Đường tối ưu: toàn $A = 150$ Hz — **phá bẫy octave 300 Hz**.

Twiddle W_8^k — bảng sin/cos

★ NGUYÊN LÝ ĐẦU TIÊN: Twiddle: hệ số xoay FFT — chỉ khi bật FFT IP

$W_N^k = e^{-j2\pi k/N} = \cos\theta - j\sin\theta$. Butterfly radix-2 nhân twiddle mỗi stage. Đây là toán FFT, không phải vật lý dây thanh. Đường MAC-first không cần twiddle ROM.

Bảng W_8^k (làm tròn 4 chữ số).

k	Re W_8^k	Im W_8^k
0	1	0
1	0,7071	−0,7071
2	0	−1
3	−0,7071	−0,7071

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. MAC default: 0. FFT IP: twiddle trong IP hoặc ROM — xem báo cáo M10K FFT.

Dao động dây thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: F_0 là gì trên thanh môn?

Khi nói hữu thanh, hai nếp thanh môn (*vocal folds*) đóng/mở chu kỳ. Tần số lần đóng mỗi giây là **tần số cơ bản** F_0 (fundamental frequency) — đơn vị hertz. Chu kỳ $T_0 = 1/F_0$ (giây) hoặc $\tau_0 = f_s \cdot T_0$ (mẫu). Đây là hiện tượng vật lý thật, không phải tên tính toán. Ước lượng F_0 trên silicon = đo τ_0 từ đỉnh tự tương quan rồi lấy f_s/τ_0.

Ví dụ tính tay. Nam trưởng thành điển hình $F_0 \approx 120$ Hz $\Rightarrow T_0 \approx 8{,}33$ ms $\Rightarrow$ @16 kHz: $\tau_0 \approx 133$ mẫu. Nữ ≈ 220 Hz $\Rightarrow \tau_0 \approx 73$ mẫu. Floor 75 Hz $\Rightarrow \tau_{\text{max}} = 213$; ceiling 600 Hz $\Rightarrow \tau_{\text{min}} = 27$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Không "mô phỏng dây thanh": chỉ đo trễ. Dải τ cố định quyết định depth ROM $C[\tau]$ và vòng MAC (Ch. 7–8).

Phổ vạch và formant

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: $F_0 \neq$ formant

Phổ vạch (line spectrum) của giọng hữu thanh: năng lượng tại $F_0, 2F_0, 3F_0, \ldots$. **Formant** là cộng hưởng ống thanh (F1, F2,…) — đỉnh bao phổ, *không* phải chu kỳ đóng thanh môn. Nhầm formant với F_0 là lỗi cổ điển khi chỉ nhìn phổ biên độ. Tự tương quan + chuẩn hóa $C[\tau]$ + octave cost giúp chọn đỉnh chu kỳ, không đỉnh formant.

Ví dụ. Formant F1 ≈ 500 Hz trên nguyên âm /a/; F_0 vẫn có thể 120 Hz. Đỉnh phổ 500 Hz không phải cao độ — cao độ là khoảng cách vạch ≈ 120 Hz.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Không ước lượng formant trên đường pitch mặc định. Nếu đỉnh r_a "dính" formant: kiểm FIR anti-alias và Hanning (sidelobe) trước khi thêm logic.

Wiener–Khinchin

★ NGUYÊN LÝ ĐẦU TIÊN: Định lý: tự tương quan $\leftrightarrow$ mật độ phổ công suất

Wiener–Khinchin: $\mathcal{R}_x = \text{IFFT}\{|X|^2\}$ (với quy ước chuẩn hóa phù hợp). Đây là đẳng thức toán — cho phép tính ACF bằng FFT khi hữu ích. Trên FPGA với τ hẹp ($\tau_{\max} \ll L$), MAC trực tiếp thường rẻ hơn FFT 1024 + BFP (Ch. 7).

Ví dụ tính tay $N = 4$. $x = [1, 0, -1, 0]$: $|X|^2$ tập trung bin phù hợp; IFFT cho $\mathcal{R}[0]$ lớn, $\mathcal{R}[2]$ dương (chu kỳ 2). MAC: $\mathcal{R}[2] = x_0x_2 + x_1x_3 = (-1) + 0 = -1$ (cùng dấu tùy quy ước). Lab: tin MAC-first; FFT optional.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon default. MAC, 0 FFT IP. Bật FFT: nhớ barrel shift 2^{2e} trước IFFT (BFP).

Cent / golden-section

★ NGUYÊN LÝ ĐẦU TIÊN: Cent: thước log so cao độ; golden-section: tối ưu 1D

Một **cent** $= 1/100$ bán cung (semitone): $\Delta = 1200 \log_2(f_1/f_2)$. Sai 20 cent $\approx$ vừa nghe thấy trên sine sạch — ngưỡng lab so Praat. **Golden-section search** là thuật toán tối ưu không đạo hàm Praat có thể dùng trong nội suy đỉnh (cùng họ Brent). FPGA real-time: không golden-section; dùng parabolic (Ch. 9).

Ví dụ. $200 \to 202$ Hz: $\Delta = 1200 \log_2(1{,}01) \approx 17{,}3$ cent. $200 \to 400$: đúng 1200 cent $= 1$ octave.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. So sánh cent chỉ trên twin/HPS. Fabric báo Hz hoặc Q12.4.

Log2 và octave

★ NGUYÊN LÝ ĐẦU TIÊN: Quãng tám = nhân 2 tần số $= +1$ trên trục $\log_2$

$\log_2(f)$ biến phép nhân tần số thành cộng. Một **octave** (quãng tám): $f \to 2f \Rightarrow \log_2$ tăng đúng 1. Octave cost/jump cost Praat phạt $|\log_2(f_1/f_2)|$ — vì người ít nhảy octave đột ngột mỗi 10 ms. Trên silicon fabric: không tính $\log_2$ (Viterbi off); trên HPS soft-float nếu bật pathFinder.

Ví dụ (đã kiểm). $\log_2(300/150) = 1 \Rightarrow$ jump cost $= 0{,}35 \cdot 1 = 0{,}35$. $\log_2(160/150) \approx 0{,}093 \Rightarrow$ cost $\approx 0{,}033$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Fabric: 0. HPS: soft $\log_2$ hoặc LUT. Không Instantiated "log2 IP" bắt buộc.

Soft sampling / soft upsample — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Câu hỏi: vì sao Praat FFT cả utterance còn FPGA thì không?

Bước soft upsample (Boersma Step 1) là phép **lọc phổ cả phát ngôn**: FFT $\rightarrow$ nhân mask $G[k] \rightarrow$ zero-pad $\rightarrow$ IFFT dài $2N$. Mục tiêu vật lý: dọn năng lượng gần Nyquist trước khi cửa sổ Hanning "quăng" sidelobe xuống vùng F_0. Trên laptop cả utterance nằm trong RAM — FFT một lần là rẻ. Trên FPGA streaming, mẫu đến từng beat: không thể đợi hết câu rồi FFT. Giải pháp silicon: (1) lọc chống alias analog trước ADC; (2) FIR decimation sạch (Ch. 4) giữ stopband sâu dưới vùng F_0; (3) không port soft upsample lên fabric.

Ví dụ tính tay (nhắc lại lưới 8 điểm). Tone cứng tại Nyquist $x = [1, 0, -1, 0, 1, 0, -1, 0]$ có năng lượng tại bin gần mép. Mask $G[k]$ giảm tuyến tính từ $0{,}95 \cdot f_s/2$ về 0 tại $f_s/2$ làm $|X_{\text{masked}}[\text{Nyquist}]| = 0$. IFFT dài 16 mẫu = upsample $\times 2$ "mềm". Trên DE10: thay bằng FIR $2/3$ (Ch. 4) với transition band kết thúc trước 8 kHz — cùng mục tiêu, khác công cụ.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. Soft upsample toàn file: 0 ALM trên fabric (không làm). Thay thế: FIR Compiler II (Ch. 4) vài chục DSP + vài M10K hệ số. Nếu lab muốn đối chiếu Praat bit-gần: chạy soft upsample offline trên HPS/Python trước khi đẩy PCM vào twin — không vào đường real-time.

Frame / hop / window length — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Ba chu kỳ dây thanh $\Rightarrow L$ và H cố định trên chip

Praat chọn `periodsPerWindow`$= 3$ với Hanning vì cần thấy cả đỉnh $\tau = T_0$ và $\tau = 2T_0$ trong một khung, nhưng khung không dài đến mức F_0 đổi nhiều (giả định local-stationary). Với $F_{\text{min}} = 75$ Hz, $f_s = 16$ kHz: $L = 640$ mẫu, hop mặc định $H = 160$ (10 ms). Trên silicon L và H là hằng số thiết kế: chúng quyết định depth M10K, depth ROM $C[\tau]$, và ngân sách chu kỳ MAC mỗi khung.

Ví dụ tính tay. $L = 3 \cdot 16000/75 = 640$; chồng lấn $(640 - 160)/640 = 75\%$. Đổi $F_{\text{min}} \rightarrow 50$ Hz $\Rightarrow L = 960$, $\tau_{\text{max}} = 320$ — phải rebuild ROM w và $C[\tau]$, mở rộng sliding RAM; không chỉ sửa một `localparam`.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. $L = 640$: 1 M10K đệm + $\approx$ 1 M10K nửa Hanning + $<$ 1 M10K $C[\tau]$. MAC: $640 \times (\tau_{\text{max}} - \tau_{\text{min}} + 1) \approx 640 \times 182 \approx 1{,}16 \cdot 10^5$ nhân/khung; @100 MHz $\approx 1{,}2$ ms $\ll 10$ ms hop.

Local mean — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: DC trên mic $\neq$ dây thanh: trừ mean trước cửa sổ

Offset mic và rung máy cộng hằng A vào mọi mẫu; $\mathcal{R}[0]$ phình tỉ lệ A^2 (mục "Trừ mean cục bộ"). Praat trừ mean trên $\pm$ một chu kỳ dài nhất quanh tâm khung. FPGA tách hai lớp: DC-blocker IIR streaming (Ch. 3) nuốt offset chậm; thêm mean khung (cộng dồn L mẫu rồi trừ) trước nhân w nếu twin muốn khớp Praat chặt hơn.

Ví dụ tính tay (đã kiểm Python). $x = [3, 5, 3, 5]$, $\mu = 4$, $x_{\text{zm}} = [-1, 1, -1, 1]$: $\mathcal{R}_{\text{zm}}[0] = 4$, $\mathcal{R}_{\text{zm}}[2] = 2$, tỉ thô $0{,}5$; unbiased $= 1$. Không trừ mean: $\mathcal{R}[0] = 68$, $\mathcal{R}[2] = 34$ — đỉnh $\tau = 0$ phình $\times 17$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. DC-blocker: vài ALM, 0 DSP, 1 thanh ghi 32-bit. Mean khung: 1 adder tích lũy + 1 chia/L (hoặc nhân $1/L$ Q-format) trước window — $<$ 50 ALM.

Candidates — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Nhiều đỉnh = $T_0, 2T_0$ và nhiễu formant

Sau chuẩn hóa $r_a[\tau]$, Praat lấy tối đa 14 cực đại cục bộ thỏa $r[i] > r[i-1]$, $r[i] \geq r[i+1]$, và $r[i] > 0{,}5 \cdot$ `voicingThreshold`, cộng 1 ứng viên Unvoiced $\Rightarrow$ `maxnCandidates`=15. Formant và nhiễu tạo đỉnh giả: giữ đa ứng viên để Viterbi chọn đường. FPGA mặc định **K=1** (chỉ đỉnh max) + UV vì Viterbi tắt trên fabric — mất đa ứng viên nhưng đủ cho median-3 + ngưỡng.

Ví dụ tính tay. Ba điểm $0{,}40, 0{,}50, 0{,}46$ tại $\tau = 80$: cực đại tại 80 vì $0{,}50 > 0{,}40$ và $0{,}50 \geq 0{,}46$; $0{,}50 > 0{,}5 \cdot 0{,}45 = 0{,}225 \Rightarrow$ nhận ứng viên. Parabolic cho $\Delta \approx +0{,}214$ (mục đỉnh).

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. Argmax một đỉnh: comparator quét $\tau_{\text{min}}..\tau_{\text{max}}$, vài ALM, 0 DSP. Top-K=14: thêm buffer 14 cặp (r, τ) + insert-sort — vài trăm ALM; chỉ cần nếu bật Viterbi HPS với đa ứng viên từ fabric.

Sinc / Brent — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Đỉnh thật nằm giữa hai mẫu: parabolic vs sinc

Lưới $\tau \in \mathbb{Z}$ lấy mẫu hàm $\mathcal{R}(\tau)$ liên tục. Parabolic fit 3 điểm: rẻ, sai số thường $< 0{,}5$ mẫu trên đỉnh sắc. Sinc/Brent (SINC70/SINC700) khôi phục band-limited chính xác hơn — Praat dùng sau parabolic. FPGA real-time: dừng parabolic (Ch. 9); Sinc optional trên HPS.

Ví dụ tính tay. $y = \{0{,}40, 0{,}50, 0{,}46\}$, $t = 0{,}2$: $\text{sinc}(1{,}2) \approx -0{,}156$, $\text{sinc}(0{,}2) \approx 0{,}935$, $\text{sinc}(-0{,}8) \approx 0{,}234$ $\Rightarrow \hat{y} \approx 0{,}513 > y[0]$ — khớp dấu Δ parabolic dương.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. Parabolic: 2 trừ + 1 chia + 1 nhân $\frac{1}{2}$ — vài chu kỳ, 0–1 DSP. SINC70 (± 35 mẫu): ~ 70 MAC/đỉnh; SINC700: ~ 700 MAC — không đưa vào fabric 10 ms.

Jump / VUV costs — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Người không nhảy quãng tám mỗi 10 ms

`octaveJumpCost`= $0{,}35$ phạt $|\log_2(f_1/f_2)|$; `voicedUnvoicedCost`= $0{,}14$ phạt đổi V↔UV. Nhân `timeStepCorrection`= $0{,}01/\Delta t$ để giữ độ dốc vật lý khi đổi hop. Đây là ràng buộc ngôn ngữ/sinh lý mã hóa thành cost — không phải sóng mới.

Ví dụ tính tay. $150 \to 160$: cost $\approx 0{,}033$; $150 \to 300$: cost $= 0{,}35$ (đúng 1 octave). $\Delta t = 5$ ms $\Rightarrow$ correction $\times 2 \Rightarrow$ jump octave $= 0{,}70$. FPGA hop cố định 10 ms $\Rightarrow$ correction $= 1$ nếu mang cost lên HPS.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách. Trên fabric: 0 (Viterbi off). Trên HPS: soft-float $\log_2$ + vài phép nhân — không đụng ALM/DSP.

Viterbi pathFinder — ánh xạ silicon

★ NGUYÊN LÝ ĐẦU TIÊN: Quy hoạch động chọn đường rẻ nhất

Tại mỗi khung giữ ≤ 15 ứng viên; $\delta[t, j]$ = strength đã trừ octave cost; cạnh mang transitionCost. Backtrace từ cuối. Cyclone V mặc định: **tắt** trên fabric; median-3 + voicing gate thay thế; pathFinder đầy đủ trên HPS nếu cần đường cong "Praat đầy đủ".

Ví dụ tính tay. Cum $V_2 = 0{,}96$, $UV_2 = 0{,}90$ (mục Viterbi 3 khung) $\Rightarrow$ chọn V, backpointer về V.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách fabric. Median-3 + voicing: $<$ 200 ALM, 0 DSP, 0 M10K. Viterbi full T khung $\times K^2$: hàng nghìn ALM + BRAM path metric — không mặc định.

Thuật ngữ Anh–Việt dùng trong phần mới

English	Tiếng Việt trong tài liệu
Autocorrelation	Tự tương quan
Lag	Trễ τ
Voicing / Unvoiced	Hữu thanh / Vô thanh
Octave cost	Phạt quãng tám
Path finding (Viterbi)	Tìm đường Viterbi
Block floating point	Dấu chấm động khối
Polyphase	Đa pha
Twiddle factor	Hệ số xoay W
Backpressure	Áp lực ngược (ready=0)
Bit-exact twin	Mô hình khớp bit

Mean removal trên tín hiệu DC+sine

★ NGUYÊN LÝ ĐẦU TIÊN: DC + sine: trừ mean trước khi tin đỉnh τ

Tín hiệu $x[n] = A + B\sin(2\pi n/\tau_0)$ có offset A. Không trừ mean: $\mathcal{R}[0]$ phình $\propto A^2$, làm méo tỉ số $r[\tau]$. Trừ mean cục bộ (hoặc DC-blocker Ch. 3) trước cửa sổ.

Ví dụ tính tay. $x = [3, 5, 3, 5]$, $\mu = 4$, $x_{\text{zm}} = [-1, 1, -1, 1]$: $\mathcal{R}_{\text{zm}}[0] = 4$, $\mathcal{R}_{\text{zm}}[2] = 2$, $r = 0{,}5$; unbiased $= 1$. Không trừ: $\mathcal{R}[0] = 68$, $\mathcal{R}[2] = 34$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. DC-blocker streaming (Ch. 3) + tuỳ chọn mean khung trước window — < 50 ALM thêm.

Time step correction — phạt theo mật độ khung

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: cùng một nhảy F_0, hop dày hơn phải phạt nặng hơn

Nếu đo F_0 mỗi 5 ms thay vì 10 ms, giữa hai khung liên tiếp cùng một cú nhảy octave sẽ xuất hiện *nhiều lần hơn* trên đường đi. Praat nhân `octaveJumpCost` và `voicedUnvoicedCost` với `timeStepCorrection`$= 0{,}01/$`timeStep` (`Pitch.cpp`) để giữ "độ dốc vật lý" — cost theo giây, không theo số khung.

Ví dụ tính tay (đã kiểm). Mặc định `octaveJumpCost`$= 0{,}35$. Với `timeStep`$= 0{,}01$: hệ số $= 1$, nhảy đúng 1 octave bị phạt $0{,}35$. Với `timeStep`$= 0{,}005$: hệ số $= 2$, cùng nhảy bị phạt $0{,}70$. FPGA cố định hop 10 ms $\Rightarrow$ hệ số $= 1$ khi port costs lên HPS; nếu đổi hop phải sửa twin cùng chỗ.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Chỉ ảnh hưởng mềm trên HPS. Fabric median-3 không dùng cost này. Đổi hop trên FPGA $\Rightarrow$ đổi ngân sách MAC/khung và phải cập nhật twin — không chỉ một hằng số.

Ceiling và floor trong delta — thiên vị nhẹ về cao

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao phạt $\log_2(\text{ceiling}/f)$ phá sai $F_0/2$?

Ứng viên tần số thấp (f gần floor) có $\log_2(\text{ceiling}/f)$ lớn $\Rightarrow$ trừ nhiều khỏi strength. Ứng viên gần ceiling gần như không bị phạt. Boersma cố ý thiên vị nhẹ về cao: khi đỉnh T_0 và $2T_0$ gần hòa sau bù cửa sổ, ứng viên F_0 (cao hơn) thắng $F_0/2$.

Ví dụ tính tay. `octave_cost` $= 0{,}01$, ceiling $= 600$: tại 100 Hz phạt $0{,}01 \cdot \log_2(6) \approx 0{,}0259$; tại 200 Hz $\approx 0{,}0158$; tại 500 Hz $\approx 0{,}0026$; tại 600 Hz $= 0$. Cùng $r_a = 0{,}48$ thì $s_{200} \approx 0{,}464 > s_{100} \approx 0{,}454$ (khớp mục octave cost).

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Trên fabric (K=1): không tính delta đầy đủ. Trên HPS pathFinder: cần $\log_2$ mềm hoặc LUT. Ceiling/floor cũng cắt dải τ tìm kiếm ($\tau_{\min}, \tau_{\max}$) — *có* ảnh hưởng fabric qua giới hạn vòng MAC.

Brent depth và SINC70/SINC700 — độ sâu nội suy

★ NGUYÊN LÝ ĐẦU TIÊN: Độ sâu = bề rộng cửa sổ sinc quanh đỉnh

Sau parabolic, Praat gọi `NUMimproveMaximum` với nội suy sinc. `SINC70` lấy khoảng ± 35 mẫu quanh đỉnh (Hanning / very_accurate off); `SINC700` lấy ± 350 (Gaussian / very_accurate). "Depth" lớn hơn = khớp band-limited tốt hơn trên sine sạch, đổi lại hàng chục đến hàng trăm MAC mỗi đỉnh.

Ví dụ tính tay quy mô. Một đỉnh: SINC70 ≈ 70 nhân–cộng; 14 đỉnh ≈ 980 MAC/khung — vẫn nhỏ so với MAC tự tương quan ($\sim 10^5$), nhưng trên fabric thêm FSM, địa chỉ, và latency khó đóng timing 10 ms nếu làm naive. Lab: parabolic only.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Default: 0 MAC sinc (parabolic). Optional HPS: NumPy/`scipy.interpolate` hoặc port C. Không Instantiated "Brent IP".

Global peak scan — tham chiếu silence

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: "nhỏ" phải so với cả câu, không chỉ trong khung

`silenceThreshold` so $P_{\text{local}}/P_{\text{global}}$. Praat quét cả sound một lần lấy `globalPeak` trước vòng frame. Streaming FPGA không nhìn tương lai: ước lượng bằng envelope max chậm (IIR/max-hold) hoặc calibration mic cố định (full-scale giả định).

Ví dụ tính tay. $P_{\text{global}} = 1$, $P_{\text{local}} = 0{,}02$, silence $= 0{,}03$, voicing $= 0{,}45$:

$$s_{\text{UV}} = 0{,}45 + \max\!\left(0,\ 2 - \frac{0{,}02/1}{0{,}03/(1+0{,}45)}\right) \approx 1{,}483.$$

Đỉnh $r_a = 0{,}30$ thua Unvoiced $\Rightarrow F_0 = 0$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Max-hold/IIR envelope: vài ALM, 0 DSP. Sai P_{global} (quá lạc quan) $\Rightarrow$ khung nhỏ vẫn voiced giả; quá bi quan $\Rightarrow$ nuốt hết giọng nhỏ.

maxnCandidates = 15 — 14 đỉnh + Unvoiced

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao 15, và vì sao FPGA hạ xuống 2?

Praat giữ tối đa 14 cực đại cục bộ + 1 UV để Viterbi còn đường chọn khi đỉnh giả formant cạnh tranh. Silicon tắt Viterbi $\Rightarrow$ đa ứng viên gần như vô ích: đủ **K=1** (đỉnh max) + UV cho voicing gate. Muốn pathFinder HPS "đầy đủ": fabric phải gửi top-K hoặc HPS tự tính lại r_a từ PCM.

Ví dụ tính tay bộ nhớ. Lưu 15 cặp (r, τ) Q1.15+9-bit: $15 \times (16 + 9) = 375$ bit — không đáng kể; đắt là insert-sort mỗi τ và Viterbi $O(TK^2)$.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon default. K=1: comparator chạy, vài register. K=14: thêm logic insert — chỉ bật khi có HPS consumer.

Interpolation depth 0.5 — pad FFT Praat

★ NGUYÊN LÝ ĐẦU TIÊN: Zero-pad phổ trước IFFT = nội suy dày $\mathcal{R}_x[\tau]$

Praat AC path: `nsampFFT` $\geq$ `nsamp_window`$\times(1+$`depth`$)$. Hanning depth $0{,}5 \Rightarrow$ hệ số $1{,}5$. Với $L = 640$: $960 \rightarrow$ làm tròn lên lũy thừa $2 = 1024$ điểm FFT. Depth không thêm thông tin mới nhưng làm lưới τ mịn hơn trước parabolic/sinc.

Ví dụ tính tay. $640 \times 1{,}5 = 960$; $2^{\lceil \log_2 960 \rceil} = 1024$. Gaussian very_accurate: window dài hơn, depth thường $0{,}25$. FPGA MAC-first: không FFT $\Rightarrow$ không pad; độ mịn τ đến từ parabolic trên lưới mẫu nguyên.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon MAC default. 0 FFT, 0 pad. Nếu bật FFT IP (Ch. 7): chọn length 1024, nhớ BFP 2^{2e} trước IFFT.

Cross-correlation vs autocorrelation — phạm vi tài liệu

★ NGUYÊN LÝ ĐẦU TIÊN: Hai method trong Praat, một đường trên FPGA

Praat còn `to_Pitch_cc`: tương quan chéo với một cửa sổ "synthetic" thay vì tự tương quan. Tài liệu và đường silicon này = **AC path** (`Sound_to_Pitch.cpp` autocorrelation). CC cùng ý "đo chu kỳ" nhưng phổ nhân với phổ tham chiếu — chưa implement trên DE10.

Ví dụ phân biệt. AC: $\mathcal{R}_x = \text{IFFT}\{|X|^2\}$. CC: $\text{IFFT}\{X \cdot W_{\text{ref}}^*\}$ (dạng giản lược). Cùng tìm đỉnh τ, khác bias cửa sổ.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Chỉ AC + ROM $C[\tau]$. Port CC $\Rightarrow$ thêm ROM phổ tham chiếu + nhân phức — ngoài phạm vi lab mặc định.

Mười tham số Sound_to_Pitch — vật lý từng cái

Tham số	Default	Vật lý / toán một câu
time step	0,0 $\rightarrow$ auto	Khoảng thời gian giữa hai đo; auto $= 0{,}75/F_{\text{min}}$
pitch floor F_{min}	75 Hz	Chu kỳ dài nhất $\Rightarrow L$ và τ_{max}
very accurate	off	Hanning/SINC70 vs Gaussian/SINC700
silence threshold	0,03	Tỉ local/global peak dưới mức $\rightarrow$ nghi lặng
voicing threshold	0,45	Sàn strength cho UV candidate
octave cost	0,01	Phạt $\log_2$ thiên vị $F_0/2$
octave jump cost	0,35	Phạt $\lvert\log_2 f_1/f_2\rvert$ giữa khung
voiced/unvoiced cost	0,14	Phạt đổi trạng thái V/UV
ceiling	600 Hz	F_{max} + mốc octave cost
maxn candidates	15	14 đỉnh + UV

Công thức L và τ trên DE10

★ NGUYÊN LÝ ĐẦU TIÊN: Ba chu kỳ tại F_{min} quyết định L

Praat: $L \approx$ `periodsPerWindow` $\cdot\, f_s/F_{\text{min}}$. Với Hanning, `periodsPerWindow` $= 3$, $f_s = 16\,\text{kHz}$, $F_{\text{min}} = 75\,\text{Hz}$:

$$L = 3 \cdot 16000/75 = 640, \quad \tau_{\text{max}} = \lfloor f_s/F_{\text{min}} \rfloor = 213, \quad \tau_{\text{min}} = \lceil f_s/F_{\text{max}} \rceil = \lceil 16000/600 \rceil = 27.$$

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. L, τ_{min}, τ_{max} là localparam — đổi F_{min} phải rebuild ROM w, $C[\tau]$, depth RAM.

Từ Boersma §1 định nghĩa $r_x(\tau)$

★ NGUYÊN LÝ ĐẦU TIÊN: Định nghĩa Boersma: tương quan chuẩn hóa

Boersma định nghĩa tương quan (dạng rút gọn) $r_x(\tau)$ từ tự tương quan có trừ mean và chuẩn hóa năng lượng, sao cho $r_x(0) = 1$ và $|r_x| \leq 1$ trên tín hiệu có năng lượng. Trên đường FPGA: sau mean + window + MAC + nhân $C[\tau]$ ta được $r_a[\tau]$ cùng vai trò — điểm số gần $[0, 1]$ để so với voicing threshold.

Ví dụ. Sau chuẩn hóa, đỉnh $\tau = 80$ với $r_a = 0{,}50$ so $V = 0{,}45 \Rightarrow$ voiced ứng viên.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Không lưu tên r_x vs r_a trên dây — chỉ Q1.15 sau khối $C[\tau]$.

Checklist tự hỏi Feynman trước khi cắt bước Praat

1. Bước này bảo vệ khỏi hiện tượng vật lý nào?
2. Sai số khi xóa: $\Delta F/F$ bao nhiêu trên sine? trên speech?
3. Silicon: LUT/DSP/M10K/latency?
4. Twin còn so được với Praat không?
5. Có thể đẩy HPS thay vì xóa hoàn toàn không?

Đáp án ngắn (để tự kiểm, không xem trước)

H.1: $X[0] = 8$, $X[4] = 8$, khác 0. H.4: $\Delta = \frac{1}{2}\frac{0{,}2-0{,}5}{0{,}2-1{,}2+0{,}5} = \frac{1}{2}\frac{-0{,}3}{-0{,}5} = 0{,}3 \Rightarrow \tau = 101{,}3$, $F_0 \approx 158$ Hz, Q12.4≈ 2528. H.5: path ưu tiên theo cum max. H.7: $e{+}{+}$, $m = [30, 5, -20, 2]$ (round). H.8: $\mathcal{R}_w[0] = 19$, $\mathcal{R}_w[1] = 16$, $C[1] = 19/16$.

Lấy mẫu và alias trên DE10

★ NGUYÊN LÝ ĐẦU TIÊN: Chuỗi 48 kHz → 16 kHz phải sạch alias

DE10: ADC WM8731 @48 kHz → hạ $\times 2$ → 24 kHz → FIR $2/3$ → 16 kHz. Mọi năng lượng > 8 kHz sau cùng phải bị chặn — nếu không, gương Nyquist giả chu kỳ dài trong vùng F_0. Soft upsample Praat cùng mục tiêu trên laptop; silicon dùng FIR (Ch. 4), không FFT cả utterance.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Ngân sách nằm ở FIR Compiler II (Ch. 4), không ở khối pitch.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu DFT 8

Cho $x = [2, 0, 2, 0, 2, 0, 2, 0]$. Tính $X[k]$ cho $k = 0..7$. Kiểm $\mathcal{R}[4]$ via IFFT $|X|^2$.
Gợi ý: tone chu kỳ 2; năng lượng ở $k = 4$ (Nyquist).

Phiếu 16 — Soft upsample

Câu hỏi: Nếu cắt/sai khối "Soft upsample, hiện tượng nào phá F_0?

- **Vật lý:** Sidelobe Nyquist bẩn F_0.
- **Toán neo:** mask $G[k]$.
- **Silicon:** offline/HPS.
- **Pytest:** optional.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Phiếu 17 — Sinc refine

Câu hỏi: Nếu cắt/sai khối "Sinc refine, hiện tượng nào phá F_0?

- **Vật lý:** Bias parabolic còn.
- **Toán neo:** $\sum r$sinc.
- **Silicon:** optional MAC.
- **Pytest:** Table1-ish.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Phiếu 19 — timeStepCorrection

Câu hỏi: Nếu cắt/sai khối "timeStepCorrection, hiện tượng nào phá F_0?

- **Vật lý:** Cost lệch khi đổi hop.
- **Toán neo:** $\times 0{,}01/\Delta t$.
- **Silicon:** HPS.
- **Pytest:** hop 5 ms $\times 2$.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Chương 2. PLL, MCLK và I^{2}S

PLL & đồng hồ audio — từ thạch anh đến SCLK

1. Vật lý

Thạch anh 50 MHz trên DE10 dao động cơ-điện rất ổn định (sai số vài ppm). Codec cần một clock master gọi là **MCLK** (ở đây thường $256 \times f_s$, tức 12.288 MHz khi $f_s = 48$ kHz) và một clock bit **SCLK**/BCLK ($64 \times f_s$).

Khối **PLL** (*Phase-Locked Loop*) khóa pha một dao động trong silicon vào thạch anh ngoài, rồi chia ra đúng MCLK/SCLK. Đây là hiện tượng điện tử thật (so pha PFD, lọc vòng, VCO). Nếu không khóa pha, thời điểm lấy mẫu trôi dần $\Rightarrow$ trên thang phút ta đo được một "cao độ" giả dù thuật toán F_0 đúng.

2. Toán + ví dụ tính tay

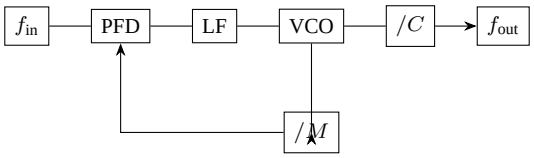
Integer PLL: $f_{\text{vco}} = f_{\text{in}} \cdot M/N$, $f_{\text{out}} = f_{\text{vco}}/C$. Ví dụ tính tay muốn $12{,}288$ MHz từ 50:

$$\frac{M}{N \cdot C} = \frac{12{,}288}{50} = 245{,}76.$$

Chọn $N = 25$, $C = 5$, khi đó $M = 245{,}76 \times 25 \times 5/(\text{scale})$ — thực tế dùng fractional MegaWizard: đặt output $12{,}288$ và đọc báo cáo M_{frac}. Sai số cho phép audio: < 50 ppm để trong 1 phút không trôi quá nửa mẫu tại 16 kHz ($50 \cdot 10^{-6} \times 60 \times 16000 \approx 48$ mẫu/phút — biên; thiết kế tốt < 5 ppm).

3. Intel IP (Cyclone V Altera PLL)

- Mode: Integer-N hoặc Fractional (Fractional PLL trên Cyclone V SX).
- Counters: N (pre), M (feedback), C_i (post).
- Jitter: fractional SDM thêm spur — vẫn chấp nhận được cho MCLK.
- Reset: `locked` phải chờ trước khi chạy I2S FSM.



Hình 14: PLL integer: PFD so pha ref vs feedback $/M$; VCO chỉnh tới khóa; chia C ra clock dùng.

4. RTL thay

Có thể dùng PLL IP bắt buộc (silicon analog). RTL chỉ chia clock sau MCLK (`clk_div` đếm 0..3 cho SCLK edge) — không thay VCO bằng counter thuần (jitter/accuracy kém).

5. Twin

Python: giả lập thời điểm mẫu $t_n = n/f_s \cdot (1 + \varepsilon_{\text{ppm}} \cdot 10^{-6})$; kiểm tra F_0 bias khi $\varepsilon \neq 0$.

I2S RX — bit timing và đóng gói mẫu

1. Vật lý

Dòng bit nối tiếp: mỗi cạnh SCLK mang 1 bit; WS chọn tai trái/phải. Chuẩn: MSB first, data trễ 1 chu kỳ SCLK sau đổi WS (Philips I2S).

2. Toán + ví dụ tính tay

Frame stereo 32-bit/slot, dùng 16 MSB làm Q1.15:

SCLK # trong nửa frame	WS	Ý nghĩa
0	↑ L	pad / delay
1–16	L	bit 15→0 của mẫu Left
17–31	L	pad / unused
32	↓ R	...

Ví dụ tính tay thu 4 bit giả: SCLK cạnh lên lần lượt nhận $1, 0, 1, 1 \Rightarrow$ nibble $0b1011 = 11$.

3. Intel IP / soft

Không có "I2S IP" bắt buộc trên Standard — RTL FSM + PLL clocks. Có thể dùng University IP "Audio" trên DE10 như black-box, nhưng tài liệu này viết FSM tự chủ.

4. RTL

`i2s_rx.v`: đồng bộ WS vào SCLK domain (2FF), shift-register 32, pulse `valid` mỗi mẫu Left (mono pitch).

5. Twin

File `.wav` → int16 LE = bit-exact với đầu ra I2S sau khi bỏ 16 LSB pad.

PLL — chỉ IP

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao không tự viết PLL bằng RTL?

PLL khóa pha analog/digital trong silicon; tự ghép cổng không cho jitter đạt mức audio. Trên Quartus Standard/Lite với Cyclone V: dùng **ALTPLL** (hoặc PLL IP trong IP Catalog). Mục tiêu dạy: tạo 50 MHz hệ thống từ clock board, và (nếu cần) clock bit I^2S.

CHI PHÍ (LUT / DSP / chu kỳ)

PLL: 1 hard block. Twin: chỉ kiểm tra `locked`. Không có mô hình bit-exact số học.

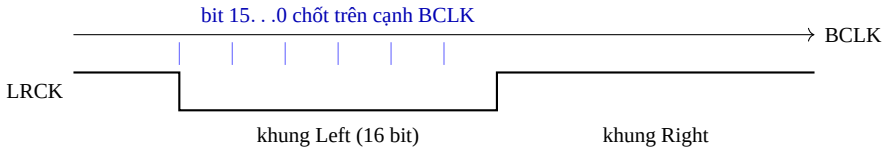
Audio vào — cả hai lộ trình (default dạy: I^2S)

★ NGUYÊN LÝ ĐẦU TIÊN: Âm thanh là gì trên dây?

Mic/codec WM8731 trên DE10 xuất chuỗi bit theo chuẩn I^2S: từng mẫu 16–32 bit, xen kẽ trái/phải, đồng bộ bằng LRCK và BCLK. Ta chỉ cần: (1) đếm bit, (2) chốt khi đủ word, (3) đẩy vào FIFO. Không cần "AI".

Lộ trình IP — University Program Audio Core. IP Avalon của University Program bọc codec: cấu hình I^2C, FIFO L/R, đọc qua Avalon-MM. **Bên trong (mức dạy):** FIFO bắt nhịp từ domain I^2S sang clock hệ thống; đóng gói mẫu signed thành word; host/HPS đọc thanh ghi. Khi nào chọn: muốn gắn nhanh Platform Designer / lab Terasic sẵn có. Chi phí: phụ thuộc bản IP + cần cài University Program.

Lộ trình RTL (default dạy) — I^2S RX + FIFO. Vài chục dòng Verilog: FSM Idle→Collect→Push. Mã mở, không phụ thuộc gói University Program.



Hình 15: I^{2}S đơn giản hóa: LRCK chọn tai, BCLK dịch bit. RTL chỉ cần dịch vào thanh ghi và đẩy FIFO — map sang Q1.15.

CHI PHÍ (LUT / DSP / chu kỳ)

I^{2}S RX RTL: ∼100–300 LE + 1 M10K FIFO nông. Audio Core IP: nhanh hơn về tích hợp, nhưng che mất đường mẫu — khó debug bit-exact với golden.

I2S bit timing — vật lý dòng bit audio

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: đồng hồ bit, word select, data – I^{2}S

I^{2}S (*Inter-IC Sound*) là chuẩn nối tiếp đưa từng bit mẫu âm thanh giữa codec và FPGA. Ba dây chính: **SCLK**/BCLK chốt từng bit; **WS**/LRCLK chọn tai trái hay phải; SD mang data. Chuẩn Philips: data trễ đúng 1 cạnh SCLK sau khi WS đổi. Frame stereo 32 bit/slot $\times 2$ = 64 SCLK là cấu hình phổ biến trên codec DE10. Đây là quy ước giao diện số, không phải sóng âm — nhưng đếm sai bit hoặc trễ sai một cạnh sẽ đảo dấu/dịch mẫu và phá F_0.

Ví dụ tính tay đếm. $f_s = 48$ kHz, 64 bit/frame $\Rightarrow$ SCLK $= 48\text{k} \times 64 = 3{,}072$ MHz. PLL từ 50 MHz crystal $\rightarrow$ 12,288 MHz MCLK ($256 \times f_s$) rồi chia.

PLL fractional/integer — feedback math

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: VCO khóa pha với tham chiếu

$f_{\text{out}} = f_{\text{in}} \cdot \frac{M}{N \cdot C}$ (integer mode). Fractional: M trung bình không nguyên nhờ SDM — jitter phụ nhưng tần số mịn (cần cho MCLK audio đúng ppm).

Ví dụ tính tay. $f_{\text{in}} = 50$, muốn 12,288: $\frac{M}{N \cdot C} = \frac{12{,}288}{50} = 245{,}76$. Chọn $N = 25$, $C = 4$, $M = 24576/(50/25/4wait)$ — thực tế MegaWizard: $M = 48$, $N = 125$, $C = 1 \times$ cascade; kiểm tra ppm trên scope. Sai MCLK $\Rightarrow$ I2S trôi $\Rightarrow$ "F_0 giả".

I2S — đếm cạnh SCLK cho một frame 48 kHz

★ NGUYÊN LÝ ĐẦU TIÊN: Một frame I^{2}S = một mẫu trái/phải

Ở 48 kHz stereo 16-bit, **LRCK**/WS đảo mỗi mẫu; **SCLK** chạy 32–64 cạnh mỗi chu kỳ WS (tùy protocol). Đếm cạnh SCLK giữa hai cạnh WS xác nhận timing. Sai đếm $\Rightarrow$ lệch bit $\Rightarrow$ mẫu Q1.15 hỏng trước khi tới DC-blocker.

Ví dụ. 48 kHz, 32 bit/frame (16 L + 16 R): $SCLK = 48\text{k} \times 32 = 1{,}536$ MHz. DE10 WM8731 thường dùng MCLK 12.288 MHz với chia nội bộ — khớp datasheet, không đoán.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. RX shift-register: vài chục ALM. Sai MCLK (4.096 vs 12.288) = toàn bộ chuỗi sai — checklist Ch. 14.

PLL — chọn M, N, C bằng tay (integer gần đúng)

★ NGUYÊN LÝ ĐẦU TIÊN: $f_{\text{out}} = f_{\text{in}} \cdot M/(N \cdot C)$ – khóa pha

PLL integer: VCO $= f_{\text{in}} \cdot M/N$, rồi chia C ra clock. Chọn M, N, C sao cho MCLK $= 12.288$ MHz từ 50 MHz thạch anh (hoặc gần đúng trong dải WM8731 chấp nhận). Fractional PLL mịn hơn nhưng phức tạp hơn — lab dùng MegaWizard, ghi đúng M, N, C vào báo cáo.

Ví dụ gần đúng. $50 \cdot M/(N \cdot C) = 12.288 \Rightarrow M/(NC) = 0{,}24576$. MegaWizard tìm bộ số hợp lệ trong VCO range Cyclone V — không tự bịa ngoài range.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. PLL = hard block. ALM logic quanh lock-detect: ít. Phải chờ `locked` trước audio.

I2S clock cross

1. **Vấn đề — FIFO bất đồng bộ.** Miền clock của SCLK (vài MHz, theo I^2S) khác miền clock hệ thống (thường 50 MHz từ PLL). Muốn chuyển mẫu an toàn cần một **async FIFO**: hàng đợi ghi theo clock nguồn, đọc theo clock đích.
2. **Mã Gray.** Con trỏ ghi/đọc khi bắt chéo miền clock phải mã hóa **Gray** (mỗi bước chỉ đổi đúng 1 bit) để tránh đọc nhầm giá trị trung gian khi hai bit chuyển cùng lúc. Đây là quy ước số học bảo vệ metastability, không phải hiện tượng âm thanh.
3. **Việc Quartus.** Dùng IP FIFO với depth nhỏ (ví dụ 8 từ) đủ cho jitter vài mẫu; bật gray coding / dual-clock; kiểm tra `wrfull`/`rdempty` trong RTL.

PLL từ PFD

★ NGUYÊN LÝ ĐẦU TIÊN: PFD: so pha tham chiếu và feedback

Phase-Frequency Detector so sánh cạnh ref và feedback; bơm điện tích vào loop filter kéo VCO. Khi khóa, tần số (và pha trung bình) khớp. Đây là vật lý mạch PLL — RTL không "viết PFD"; dùng IP.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. 0 ALM cho PFD (hard). Soft-PLL bằng counter không thay hard PLL cho MCLK jitter thấp.

I2S đếm bit 16 kHz

★ NGUYÊN LÝ ĐẦU TIÊN: Sau hạ mẫu: đếm beat 16 kHz trên clock hệ thống

Audio sau FIR là 16 kHz — chậm hơn clock 50 MHz. FSM pitch đếm `valid` beat Avalon-ST, không giả định 1 mẫu/ck. Nhầm "mỗi ck một mẫu" $\Rightarrow$ MAC đọc rác.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. Bộ đếm beat: vài ALM. Hợp đồng: chỉ tiến khung khi đủ H mẫu mới (hop).

Lấy mẫu và alias trên DE10

Vật lý

Codec SSM2603 (hoặc WM8731 trên một số bản DE10) dùng ADC delta-sigma rồi **decimation** nội bộ để ra chuỗi mẫu PCM. Dù vậy, nhiễu RF trên line-in vẫn có thể fold vào băng baseband nếu không lọc tương tự phía trước — đó chính là **aliasing** vật lý trước khi bit nào xuất hiện trên I^2S.

Toán

Định lý Nyquist đòi hỏi $f_s > 2B$ với B bề rộng phổ hữu ích. Giọng nói ở 16 kHz đủ chứa F_0 và vài formant thấp; thành phần trên 8 kHz sẽ bị gương qua Nyquist nếu không bị lọc.

Ví dụ tính tay

Tone thật 9 kHz lấy mẫu ở 16 kHz trông giống tone 7 kHz vì $|9-16| = 7$. Công thức tổng quát: alias tại $|f - m f_s|$ với m nguyên sao cho kết quả nằm trong $[0, f_s/2]$.

Phần cứng / twin

Giữ PLL MCLK đúng ppm; sinh SCLK/WS từ cùng PLL để không trôi tương đối; twin Python giả lập $t_n = n/f_s \cdot (1 + \varepsilon_{\text{ppm}} 10^{-6})$ để thấy bias F_0 khi $\varepsilon \neq 0$.

Giao diện Avalon-ST. Chuẩn streaming của Intel dùng hai tín hiệu bắt tay: nguồn nâng `valid` khi data có nghĩa; đích nâng `ready` khi còn chỗ nhận; chỉ khi *cả hai* bằng 1 mới chuyển một beat. Quên giữ data khi `ready`=0 sẽ mất mẫu và làm hỏng chuỗi F_0. Lý thuyết và FSM đầy đủ xem Ch. 11.

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — PLL + I²S.

- PLL IP: hard block (không tốn ALM logic lớn); cần lock time trước khi lấy mẫu.
- I²S RX: vài chục ALM (shift register + WS edge detect), 0 DSP, 0 M10K.
- Vì sao MCLK=12.288 MHz: khớp WM8731 @48 kHz; 4.096 MHz là cấu hình khác — không dùng trên gói DE10 này.

Tóm tắt số liệu: Altera PLL: khối PLL cứng (0 ALM đáng kể). Soft I²S RX: thường < 100 ALM, 0 DSP, 0 M10K. Fmax bị giới hạn bởi CDC SCLK→sysclk — FIFO nhỏ. Giữ khối này rẻ để dành DSP/M10K cho FIR và AC.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu PLL

$f_{in} = 50$, $f_{out} = 25{,}6$. Đề xuất M, N, C integer với $f_{vco} \in [600, 1600]$.

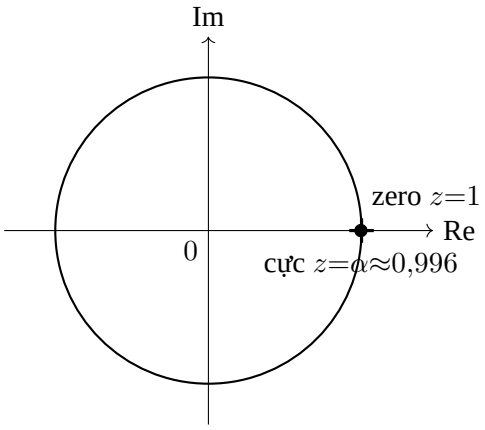
Phiếu 1 — I2S / PLL

Câu hỏi: Nếu cắt/sai khối "I2S / PLL, hiện tượng nào phá F_0?

- **Vật lý:** SCLK trôi ⇒ thời gian mẫu sai ⇒ F_0 bias ppm.
- **Toán neo:** $f_{\text{SCLK}} = f_s \cdot 64$.
- **Silicon:** PLL+div, ∼0 DSP.
- **Pytest:** TB MCLK counter.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E⋆.

Chương 3. DC-blocker IIR (cực/zero, bit-exact)

Hình: mặt phẳng z — cực và zero của DC-blocker



Zero trên vòng đơn vị tại DC ⇒ $H(1)$=0.
Cực gần zero (trong vòng) ⇒ HPF nhẹ, ổn định ($|\alpha|$<1).

Hình 16: DC-blocker $H(z) = \frac{1-z^{-1}}{1-\alpha z^{-1}}$ trên mặt phẳng z: zero tại $z = 1$, cực tại $z = \alpha = 1 - 2^{-8}$.

DC-blocker — cực/zero, bit-growth, Avalon

1. Vật lý — vì sao phải chặn DC trước tự tương quan

Micro, codec và đường dây analog luôn mang theo một thành phần gần đứng yên: offset điện áp một chiều (DC), cộng thêm trôi chậm do nhiệt và nhiễu 50/60 Hz. Trong miền mẫu, đó là một hằng A cộng vào mọi $x[n]$. Phép tự tương quan không phân biệt "năng lượng hữu ích" và "năng lượng offset":

$$\mathcal{R}_{x+A}[0] = \sum_n (x[n] + A)^2 = \mathcal{R}_x[0] + 2A \sum_n x[n] + NA^2.$$

Đỉnh tại $\tau = 0$ phình tỉ lệ với A^2 và có thể nuốt mất đỉnh chu kỳ T_0. Vì thế, trước khi nhân cửa sổ và MAC, ta bắt buộc phải đẩy thành phần gần DC xuống gần 0. Bộ lọc làm việc đó trên FPGA gọi là **DC-blocker**: một HPF rất nhẹ, ổn định, rẻ silicon.

2. Toán + ví dụ tính tay (cực và zero)

Hàm truyền chuẩn của DC-blocker bậc nhất là

$$H(z) = \frac{1 - z^{-1}}{1 - \alpha z^{-1}}, \qquad 0 < \alpha < 1.$$

Zero (nghiệm tử số) đặt tại $z = 1$, tức tần số 0 Hz: mọi tín hiệu hoàn toàn đứng yên bị triệt tiêu ($H(1) = 0$). **Cực** (nghiệm mẫu số) đặt tại $z = \alpha$, nằm sát zero nhưng vẫn *trong* vòng đơn vị nên bộ lọc ổn định. Khoảng cách $1 - \alpha$ quyết định độ dốc của HPF: α càng gần 1 thì chỉ chặn vùng rất gần DC, giữ nguyên cao độ giọng nói.

Trên silicon ta chọn $\alpha = 1 - 2^{-8} = 0{,}99609375$ vì phép nhân $\alpha \cdot y$ thu gọn thành một phép dịch và trừ: $\alpha y = y - (y \ggg 8)$ — không tốn DSP. Đáp ứng xung với $\alpha = 0{,}5$ (chỉ để tính tay) cho $x = \delta[n]$ là $y = [1, -0{,}5, -0{,}25, -0{,}125, \ldots]$: năng lượng xung bị "nuốt" dần theo hình học, đúng trực giác của một HPF nhớ trạng thái.

Trong dấu phẩy tĩnh, bắt buộc giữ thanh ghi phản hồi `y_acc` rộng 32 bit. Nếu dịch $\ggg 8$ ngay trên thanh ghi 16 bit, phần thập phân bị cắt sớm ⇒ vùng chết (deadband) và dao động limit-cycle. Đầu ra lấy $\text{sat}_{16}(y_{\text{acc}} \ggg 15)$ khớp twin Python ở mục sau.

3. Intel IP — vì sao viết RTL thuần

FIR Compiler II *không* thay thế được IIR có phản hồi rộng: kiến trúc FIR không có vòng $y[n-1]$. IP "IIR II" nếu có trong bản Standard thường quá nặng cho một cực/zero đơn. Khuyến nghị dạy và lab: viết khoảng mười dòng Verilog với accumulator 32 bit — dễ bit-exact, dễ đọc, đủ cho đường pitch.

4. RTL — accumulator 32-bit (bắt buộc)

Không được dịch `>>>8` trên thanh ghi y 16-bit: phần thập phân bị truncate $\Rightarrow$ deadband / limit-cycle. Phải giữ `y_acc` signed 32-bit (Q1.30 nội bộ khi x được kéo trái 15 bit; ở đây dùng Q17.15: x sign-extend lên 32, $\alpha = 1 - 2^{-8}$).

Phương trình cập nhật (bit-exact):

$$\begin{aligned} x_\Delta &= \text{sext}_{32}(x[n]) - \text{sext}_{32}(x[n-1]), \\ y_{\text{acc}}[n] &= y_{\text{acc}}[n-1] + x_\Delta - (y_{\text{acc}}[n-1] \ggg 8), \\ y_{\text{out}} &= \text{sat}_{16}\big(\text{rnd}_{15}(y_{\text{acc}}[n])\big), \end{aligned}$$

với rnd_{15}: lấy bit $[30:15]$ sau khi cộng bit làm tròn tại vị trí 14 (hoặc truncate nhất quán với twin — chọn **truncate** $\ggg 15$ trong bản mặc định lab).

```
module dc_blocker (
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,
  input  wire signed [15:0] x_in,       // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_out       // Q1.15
);
  reg signed [15:0] x_z1;
  reg signed [31:0] y_acc;              // 32-bit feedback (NOT 16)
  assign in_ready = out_ready | ~out_valid;

  wire signed [31:0] x_sext  = {{16{x_in[15]}}, x_in};
  wire signed [31:0] xz_sext = {{16{x_z1[15]}}, x_z1};
  wire signed [31:0] x_delta = x_sext - xz_sext;
  // y_acc + x_delta - (y_acc>>>8)  ==  x_delta + y_acc * (1 - 2^{-8})
  wire signed [31:0] y_next  = y_acc + x_delta - (y_acc >>> 8);
  // output from high bits: truncate >>>15, then sat16
  wire signed [31:0] y_shift = y_next >>> 15;
  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767)       sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else                      sat16 = v[15:0];
    end
  endfunction

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      x_z1 <= 0; y_acc <= 0; y_out <= 0; out_valid <= 0;
    end else if (in_valid && in_ready) begin
      y_acc <= y_next;
      x_z1  <= x_in;
      y_out <= sat16(y_shift);
      out_valid <= 1'b1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

5. Twin Python (khớp RTL)

```
import numpy as np

def dc_blocker_q15(x, alpha_shift=8):
    """x: int16 Q1.15 ndarray. Returns int16 Q1.15.
    y_acc is int32; update: y = y + dx - (y >> 8); out = sat16(y >> 15).
    Python/numpy >> on signed int == Verilog arithmetic >>> (floor).
    """
    y_acc = np.int32(0)
    x_z1 = np.int32(0)
    out = np.empty_like(x)
    for i, xi in enumerate(x.astype(np.int32)):
        dx = xi - x_z1
        y_acc = np.int32(y_acc + dx - (y_acc >> alpha_shift))
        x_z1 = xi
        y_shift = y_acc >> 15
        if y_shift > 32767: y_shift = 32767
        elif y_shift < -32768: y_shift = -32768
        out[i] = np.int16(y_shift)
    return out

# Ví dụ tính tay: DC step 16384 -> y decays to 0
x = np.array([0,0,0] + [16384]*29, dtype=np.int16)
y = dc_blocker_q15(x)
assert abs(int(y[-1])) < 100  # near 0 after ~32 samples with a=1-2^-8
```

Tương đương shift: trên số nguyên có dấu, `numpy.int32` / Python `>>` là *arithmetic floor shift* (bit dấu nhân rộng) — cùng nghĩa với Verilog `>>>` trên `signed`. Không cần đổi sang `//` 256: `y_acc >> 8` bit-exact với `y_acc >>> 8` trong RTL (miễn `y_acc` giữ `np.int32`, không promote lên `int` vô hạn bit rồi cắt lệch).

DC-blocker / khử mean (NEW)

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao DC phá tự tương quan?

Nếu mọi mẫu cộng thêm hằng A (offset mic), thì

$$\mathcal{R}_x[0] = \sum (x_n + A)^2 = \sum x_n^2 + 2A \sum x_n + NA^2$$

phình theo A^2. Đỉnh $\tau = 0$ "nuốt" mọi đỉnh chu kỳ. Praat trừ mean cục bộ từng khung; streaming cần lọc liên tục.

Ví dụ tính tay. Chuỗi $x = [1, 1, 1, 1] \Rightarrow \mathcal{R}[0] = 4$. Sau trừ mean $(0, 0, 0, 0) \Rightarrow \mathcal{R}[0] = 0$. DC thuần phải ra unvoiced, không được treo FSM.

Default dạy: RTL IIR DC-block.

$$y[n] = x[n] - x[n-1] + \alpha\, y[n-1], \quad \alpha \approx 1 - \varepsilon$$

(ví dụ $\alpha = 1 - 1/256$). Quan hệ với HPF 60 Hz: DC-block là HPF cực gần 0 Hz diện tích nhỏ; HPF 60 Hz cắt thêm ù quạt/rung cơ học. Có thể gộp vai trò vào một FIR thông cao (IP), nhưng bài này dạy IIR riêng để thấy mean.

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao không giữ y chỉ 16-bit Q1.15?

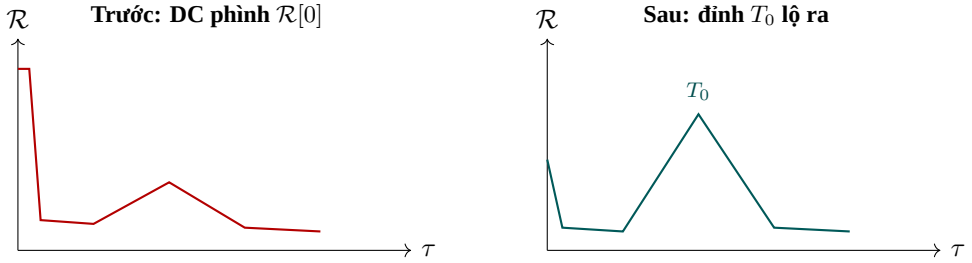
Ví dụ tính tay: $\alpha = 1 - 2^{-8} = 255/256$. Phản hồi $\alpha y[n-1]$ cần phần lẻ của y. Nếu cắt y về 16-bit ngay mỗi chu kỳ, αy liên tục làm tròn về 0 khi $|y|$ nhỏ $\Rightarrow$ filter "chết" hoặc dao động limit-cycle. Giải pháp First-Principles: giữ **thanh ghi phản hồi rộng** (ví dụ Q8.23 / 32-bit signed), chỉ saturate xuống Q1.15 ở lối ra.

$$y_{\text{acc}}[n] = y_{\text{acc}}[n-1] + \big(x[n] - x[n-1]\big) - \big(y_{\text{acc}}[n-1] \ggg 8\big), \quad y_{\text{out}} = \text{sat}_{16}(y_{\text{acc}} \ggg 15).$$

Với $\alpha = 1 - 2^{-8}$ thay nhân bằng `y_acc - (y_acc>>>8)` — 0 DSP. Thanh ghi phản hồi **32-bit**; chỉ lấy bit cao khi xuất 16-bit.

CHI PHÍ (LUT / DSP / chu kỳ)

DC IIR rộng 32-bit + shift-add α: $\lesssim$80–120 LE, 0 DSP. Cắt 16-bit sớm: rẻ hơn $\sim$30 LE nhưng **sai chức năng** — đắt hơn mọi LUT.



Hình 17: Cùng tín hiệu có offset: trái — $\mathcal{R}[0]$ phình; phải — sau DC-block, đỉnh chu kỳ hiện rõ. Đặt cạnh nhau để map từng bước.

CHI PHÍ (LUT / DSP / chu kỳ)

Nhắc lại: phản hồi $\geq$24–32 bit; lối ra 16-bit. FIR HP IP thay thế: hàng trăm LE/DSP tùy taps.

DC-blocker IIR — cực / zero từ first principles

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: offset chậm $\neq$ dây thanh

Offset mic $\approx$ tần số 0. Bộ chặn DC: zero tại $z = 1$ (chặn DC hoàn hảo), cực gần $z = 1$ để dải thông phẳng.

Đạo hàm. $H(z) = \dfrac{1 - z^{-1}}{1 - \alpha z^{-1}}$. Tại $z = 1$: $H = 0$. Cực $z = \alpha$ ($\alpha = 0{,}995 \Rightarrow$ cutoff rất thấp). Đáp ứng biên độ tại ω:

$$|H(e^{j\omega})| = \frac{|1 - e^{-j\omega}|}{|1 - \alpha e^{-j\omega}|}.$$

Ví dụ tính tay đệ quy 5 bước. $x = [1, 1, 1, 1, 1]$ (DC thuần), $y[-1] = 0$, $\alpha = 0{,}5$ (cực xa để thấy nhanh):

n	$x[n]$	$y[n] = (x[n] - x[n-1]) + \alpha y[n-1]$	
0	1	$1 - 0 + 0 = 1$	
1	1	$0 + 0{,}5 \cdot 1 = 0{,}5$	
2	1	$0 + 0{,}5 \cdot 0{,}5 = 0{,}25$	
3	1	$0{,}125$	
4	1	$0{,}0625$	$\to 0$ (đúng: chặn DC)

Với $\alpha = 0{,}995$ (thực tế) suy giảm chậm hơn — cần $\sim 1/(1-\alpha) \approx 200$ mẫu để ổn định. **Bit-growth:** feedback gần 1 $\Rightarrow$ tích lũy; giữ Q1.30 nội bộ rồi làm tròn (C.3).

DC-blocker — mặt phẳng z, Bode skew, fixed-point

Zero tại $z = 1 = e^{j0}$. Cực tại $z = \alpha$ (cùng góc, bán kính $\alpha < 1$). Khoảng cách zero$\to$điểm $e^{j\omega}$ trên vòng tròn đơn vị $\approx \omega$ (nhỏ); cực$\to$điểm $\approx |\omega|$ nhưng bán kính ngắn hơn $\Rightarrow |H| \approx 1$ ở giữa dải.

Cutoff xấp xỉ $f_c \approx \frac{1-\alpha}{2\pi} f_s$. $\alpha = 0{,}995$, $f_s = 16\text{k} \Rightarrow f_c \approx 12{,}7$ Hz — dưới $F_{\min} = 75$, trên DC.

Bit-growth ví dụ tính tay. $\alpha = 1 - 2^{-8}$, x Q1.15 full-scale DC step 32767. y nội bộ Q1.30; sau 256 mẫu vẫn còn $\sim 1/e$ năng lượng transient. Accumulator 32-bit an toàn; cắt về 16 có bão hòa.

Testbench zero / DC / clip

★ NGUYÊN LÝ ĐẦU TIÊN: Ba vector bắt lỗi DC-blocker khác nhau

`zero`: mọi ra $= 0$. `DC`: vào hằng $\Rightarrow$ ra $\to 0$ sau transient. `clip`: vào $\pm$full-scale $\Rightarrow$ không wrap 16-bit ở ngõ ra (sat). Thiếu một vector = bỏ sót bão hòa hoặc cực không ổn định.

CHI PHÍ (LUT / DSP / chu kỳ)

Xác minh. Sim ngắn vài ms audio; so twin `dc_blocker_q15` từng mẫu.

DC-blocker bit-growth

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao y 32-bit?

IIR $y \leftarrow y + x_\Delta - (y \ggg 8)$ tích lũy. Giữ y 16-bit: mỗi bước truncation $\approx$ noise + offset giả $\Rightarrow$ phá $\mathcal{R}[0]$. 32-bit accumulator rồi sat xuôi 16 là tối thiểu an toàn cho Q1.15 in/out.

Ví dụ. $\alpha \approx 1 - 2^{-8} = 0{,}996$: hằng thời gian $\approx 1/(1-\alpha) \approx 256$ mẫu @16 kHz ≈ 16 ms.

CHI PHÍ (LUT / DSP / chu kỳ)

Silicon. 32-bit add/shift: vài chục ALM; 0 DSP; 0 M10K.

DC-blocker — 32 mẫu step response

$\alpha = 0{,}875$, $x = [0]{*}4 + [1]{*}28$.

n	y	n	y	n	y	n	y
0	0.00000	8	0.58618	16	0.20142	24	0.06921
1	0.00000	9	0.51291	17	0.17624	25	0.06056
2	0.00000	10	0.44880	18	0.15421	26	0.05299
3	0.00000	11	0.39270	19	0.13493	27	0.04636
4	1.00000	12	0.34361	20	0.11807	28	0.04057
5	0.87500	13	0.30066	21	0.10331	29	0.03550
6	0.76562	14	0.26308	22	0.09040	30	0.03106
7	0.66992	15	0.23019	23	0.07910	31	0.02718

Thấy $y \to 0$ dù $x = 1$ — đúng chặn DC. Với $\alpha = 0{,}995$ chậm hơn $\sim 8\times$.

DC-blocker 16 mẫu $\alpha = 0{,}996$

n	y	n	y	n	y	n	y
0	0.0000	4	0.8100	8	0.5314	12	0.3487
1	0.0000	5	0.7290	9	0.4783	13	0.3138
2	1.0000	6	0.6561	10	0.4305	14	0.2824
3	0.9000	7	0.5905	11	0.3874	15	0.2542

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — DC-blocker.

- ALM: vài chục (adder 32-bit + shift); 0 DSP; 0 M10K.
- Thanh ghi y 32-bit bắt buộc: giữ 16-bit sẽ tích lũy truncation $\Rightarrow$ offset giả.
- Vì sao IIR một cực: đủ nuốt DC mic chậm; mean khung (Ch. 1) bổ sung nếu twin muốn khớp Praat chặt.

Tóm tắt số liệu: DC-blocker IIR: 1 phép nhân $\times\alpha$ (hoặc shift-add khi $\alpha = 1 - 2^{-8}$) + 2 phép cộng. Với $\alpha = 1 - 2^{-8}$ dùng `>>>8` — 0 DSP, vài chục ALM, 0 M10K. Bit-growth nội bộ Q1.30; output truncate về Q1.15. Fmax dễ > 200 MHz. Chi phí gần như miễn phí so với FIR/AC — đây là lý do đặt DC trước mọi khối nặng.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu 2 — DC-block

Câu hỏi: Nếu cắt/sai khối "DC-block, hiện tượng nào phá F_0?

- **Vật lý:** DC phình $\mathcal{R}[0]$ nuốt đỉnh T_0.
- **Toán neo:** $H = (1 - z^{-1})/(1 - \alpha z^{-1})$.
- **Silicon:** 0 DSP, ≤ 120 LE.
- **Pytest:** step$\to 0$; DC in $\to F_0 = 0$.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Phiếu DC step

$\alpha = 0{,}75$, $x = [0, 0, 0, 8, 8, 8, 8, 8]$. Tính $y[0..7]$ đệ quy. Vẽ.

Chương 4. FIR Compiler II — hạ mẫu $24 \to 16$ kHz (polyphase 2/3)

FIR Compiler II — polyphase, folding, lượng tử hệ số

1. Vật lý — vì sao phải lọc trước khi bỏ mẫu

Sau ADC, chuỗi mẫu 48 kHz được hạ $\times 2$ xuống 24 kHz rồi cần về 16 kHz để khớp ngân sách Praat/FPGA. **Decimation** (hạ mẫu) nghĩa là giữ lại một phần mẫu và bỏ phần còn lại. Nếu bỏ mẫu mà chưa lọc, thành phần cao trong phổ sẽ **alias** xuống vùng thấp — đúng chỗ ta tìm F_0. Vì thế trước mọi phép bỏ mẫu phải có bộ lọc chống alias.

Bộ lọc **FIR** (*Finite Impulse Response*) thực hiện $y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$. Mỗi hệ số $h[k]$ gọi là một **tap** (vòi): nó là trọng số của xung lực tại trễ k. Dải tần số nằm giữa passband (cho qua) và stopband (chặn) gọi là **transition band**: muốn hẹp thì cần nhiều tap hơn, tốn thêm DSP/M10K. Trên đường pitch, FIR tỉ số $2/3$ vừa chống alias vừa đưa 24 kHz về 16 kHz.

2. Toán — folding, lượng tử, polyphase $2/3$

Công thức đầy đủ:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k].$$

Khi thiết kế pha tuyến tính, hệ số đối xứng $h[k] = h[N-1-k]$. Gộp cặp đối xứng (**symmetric folding**) còn $\lceil N/2 \rceil$ phép nhân:

$$y[n] = \sum_{k=0}^{N/2-1} h[k]\big(x[n-k] + x[n-(N-1-k)]\big).$$

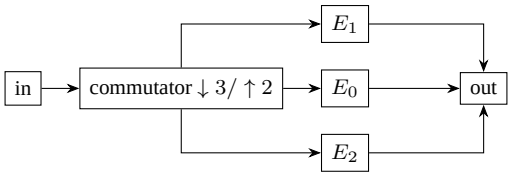
Ví dụ tính tay folding $N = 4$. Cho $h = [1, 2, 2, 1]$ và cửa sổ mẫu $[a, b, c, d]$:

$$y = 1 \cdot (a + d) + 2 \cdot (b + c)$$

— chỉ 2 nhân thay vì 4.
Lượng tử hệ số. Đưa h_{float} sang Q1.15: $h_q = \text{round}(h \cdot 2^{15})$, rồi **bão hòa** vào $\pm(2^{15} - 1)$ nếu tràn. Sai số từng hệ số làm méo đáp ứng tần số: $\Delta H(e^{j\omega}) = \sum \Delta h[k] e^{-j\omega k}$. Twin phải dùng đúng cùng h_q với ROM/FIR IP.
Tỉ số hữu tỉ $2/3$ và polyphase. Thay vì chạy một FIR dài rồi bỏ mẫu, ta tách FIR thành ba nhánh **polyphase** E_0, E_1, E_2. Commutator lấy mẫu vào theo ba pha, nhưng chỉ xuất hai pha mỗi ba chu kỳ vào — đúng tỉ số $2/3$. Lưu ý quan trọng: đây *không* phải "2 vào thành 3 ra"; chiều đúng là 3 nhánh vào, giữ 2 nhánh ra.

3. Intel FIR Compiler II (UG 683208)

- Architectures: Distributed Arithmetic / Mult-add / Systolic.
- Cyclone V: dùng DSP 18×19 (hoặc 27×27 nếu coeff rộng).
- Avalon-ST: `valid`/`ready`; `sop`/`eop` = biên *kênh* (multi-channel), **không** phải biên khung pitch. Mono liên tục: chỉ data/valid/ready.
- Rounding: truncation / convergent; saturation optional.
- M10K lưu coeff + delay line khi N lớn.



Hình 18: Polyphase fractional: 3 phase filter E_r; commutator ghép đúng lịch 2 out / 3 in.

4. RTL thay

Systolic MAC thuần + coeff ROM; folding tay nếu N nhỏ. Không tự viết DA trừ khi tối ưu diện tích cực đoan.

5. Twin

`scipy.signal.lfilter` với *cùng* h_q int16; so dump Avalon-ST beat-by-beat (ready backpressure test).

DSP block Cyclone V — 18×19 và 27×27

1. Vật lý silicon

Transistor nhân tổ hợp đắt hơn cộng. DSP block cứng: nhân + pre-adder + accumulator bypass register — một chu kỳ, ít routing.

2. Toán

Sản phẩm $P = A \times B$. Mode 18×19: A 18-bit, B 19-bit (signed) → 37-bit. Mode 27×27: 54-bit ra. Pre-adder: $(A \pm D) \times B$ — khớp symmetric FIR folding.
Ví dụ tính tay. $A = 5$ (Q1.3 giả), $B = 3 \Rightarrow P = 15$. Pre-add $A + D = 5 + 7 = 12$, $\times 3 = 36$.

3. Intel

Cyclone V Variable Precision DSP (UG). Quartus map `*` trong Verilog thành DSP nếu pipeline register đúng pattern. `multstyle="dsp"` attribute.

4. RTL

5. Twin

Hạ mẫu $24 \to 16$ kHz (polyphase)

★ NGUYÊN LÝ ĐẦU TIÊN: Tỷ lệ $2/3$ là gì?

$24/16 = 3/2$, nên cần **nội suy** $\times 2$ rồi **decimate** $\div 3$ (hoặc ngược lại có tiền lọc). Đây là *fractional rate*, không phải decimation nguyên $R = 10, 20, \ldots$ nơi CIC (Cascaded Integrator–Comb) tỏa sáng. CIC bỏ nhân nhưng đáp ứng sinc^N tệ ở tỷ lệ nhỏ và cần bù — **loại khỏi lộ trình** $24 \to 16$.

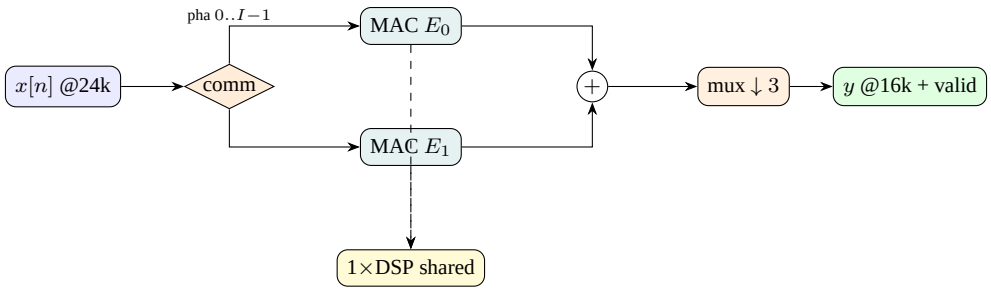
Hộp "CIC dùng khi nào?" (để khỏi nhầm). CIC hữu ích khi $R \gg 1$ (ví dụ ADC $\Delta\Sigma$ 10 MHz $\to$ 48 kHz). Ở đây R hiệu dụng chỉ 1,5 — FIR polyphase thắng về méo phổ và diện tích có nhân sẵn trên Cyclone V.

IP default: FIR Compiler II Fractional Rate $I = 2$, $D = 3$. Theo UG 683208: output rate / input rate $= I/D = 2/3$ khớp $16/24$. Nghĩa đúng: **mỗi 3 mẫu vào $\Rightarrow$ 2 mẫu ra** (không phải "2 vào $\Rightarrow$ 3 ra"). IP tối ưu polyphase; chọn truncation / saturation / rounding ở lối ra. Twin: cùng hệ số, cùng round/sat với mô hình Python cố định điểm.

★ NGUYÊN LÝ ĐẦU TIÊN: Valid ra có đều theo clock không?

Không. Clock hệ thống (ví dụ 50 MHz) nhanh hơn sample rate. FIR fractional nhả mẫu theo nhịp I/D: `ast_source_valid` **bật/tắt không đều**. Ví dụ tính tay: trên 6 nhịp sample-in, valid-out có thể là `1,0,1,1,0,1` — FSM/FIFO sau FIR **bắt buộc** đọc cờ `valid` từng chu kỳ; đếm clock "cứ 3 ck lấy 2" là bẫy khi có backpressure.

RTL: FIR polyphase — mạch số, không Z-transform suông. Commutator (mux pha) đưa mẫu vào đúng nhánh; các nhánh chia sẻ một DSP theo thời gian; sau đó decimate.



First-Principles: anti-alias analog trước ADC $\leftrightarrow$ polyphase số sau ADC. Commutator = bộ đếm pha + mux; "shared DSP" = một nhân 18×18 lần lượt phục vụ E_0, E_1. Lối ra: chỉ bắt mẫu khi `valid`$= 1$ (xem cảnh báo fractional ở trên).

Hình 19: Polyphase fractional I=2, D=3 mức mạch số: commutator $\to$ nhánh MAC $\to$ DSP chia sẻ $\to$ mux decimate. Không còn block $\uparrow 2/\downarrow 3$ sách giáo khoa.

CHI PHÍ (LUT / DSP / chu kỳ)

FIR II fractional (vài chục taps, 16-bit): cỡ 1–4 DSP + vài trăm ALM (UG 683208 — đo lại trên variation). RTL polyphase 2 nhánh, 1 DSP time-mux: $\sim$1 DSP + vài trăm LE điều khiển; twin bằng cùng chuỗi >>/saturate.

FIR 3-tap — convolution bằng tay

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: trung bình có trọng = lọc thông thấp / nhấn tần

$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$. Mỗi hệ số $h[k]$ = "độ nhớ" mẫu quá khứ.

Ví dụ tính tay $h = [1, 2, 1]/4$, $x = [4, 0, 4, 0, 4]$.

n	$y[n]$	ghi chú
0	$(1\cdot 4)/4 = 1$	thiếu history $\to 0$
1	$(1\cdot 0 + 2\cdot 4)/4 = 2$	
2	$(1\cdot 4 + 2\cdot 0 + 1\cdot 4)/4 = 2$	
3	$(1\cdot 0 + 2\cdot 4 + 1\cdot 0)/4 = 2$	
4	2	LP làm phẳng tone Nyquist

Đối xứng $h[0] = h[2] \Rightarrow$ FIR Compiler gấp đôi hệ số (symmetric folding) — một DSP nhân hai mẫu cộng trước (C.4).

Polyphase / CIC identity — vì sao $2/3$ không cần CIC

★ NGUYÊN LÝ ĐẦU TIÊN: Toán: đổi tốc độ = lọc + bỏ/chèn mẫu

Decimate $\downarrow M$: lọc LP chống alias rồi giữ mỗi mẫu thứ M. Upsample $\uparrow L$: chèn $L-1$ zero rồi lọc ảnh. **CIC** (Cascaded Integrator–Comb) rẻ khi M lớn nguyên; với tỷ số hữu tỷ $2/3$ (16 kHz$\to\sim$10,67 kHz) FIR Compiler II fractional dùng **polyphase commutator** — không cần CIC.

Đạo hàm polyphase. $H(z) = \sum_{k=0}^{N-1} h[k]z^{-k} = \sum_{r=0}^{M-1} z^{-r}E_r(z^M)$, E_r = phase r. Với $\uparrow 2/\downarrow 3$: cứ 3 mẫu vào $\to$ 2 mẫu ra (không phải 2 vào $\to$ 3 ra!).
Ví dụ tính tay lịch mẫu. Vào $n = 0, 1, 2, 3, 4, 5, \ldots$; ra tại thời điểm nội suy tương ứng $0,\ 1{,}5,\ 3,\ 4{,}5, \ldots$ (tính theo đồng hồ out). Đếm: 6 vào $\to$ 4 ra $= 2/3$.

FIR 5-tap — convolution bảng đủ

$h = [1, 2, 3, 2, 1]/9$ (xấp xỉ Gauss), $x = [0, 0, 1, 0, 0, 0, 2, 0, 0]$ (hai xung).

n	0	1	2	3	4	5	6	7	8
x	0	0	1	0	0	0	2	0	0
y	0	0	$1/9$	$2/9$	$3/9$	$2/9$	$1/9 + 2/9$	$4/9$	$6/9$

Tiếp $n = 9{:}\ 4/9$; $n = 10{:}\ 2/9$; $n = 11{:}\ 0$. Hai "quả núi" tỉ lệ 1:2 — tuyến tính của convolution.

Symmetric folding: $y[n] = \frac{1}{N}\big(1\cdot(x[n] + x[n-4]) + 2\cdot(x[n-1] + x[n-3]) + 3\cdot x[n-2]\big)$ — 3 nhân thay 5.

Đạo hàm polyphase — từ $H(z)$ đến commutator

$$H(z) = \sum_{k=0}^{N-1} h[k]z^{-k}.$$

Viết $k = mM + r$, $r = 0..M-1$, $m = 0, 1, \ldots$:

$$H(z) = \sum_{r=0}^{M-1} z^{-r} \sum_{m} h[mM + r](z^M)^{-m} = \sum_{r=0}^{M-1} z^{-r}E_r(z^M).$$

Decimation $\downarrow M$ sau H: chỉ giữ $y[mM]$. Noble identity: lọc $E_r(z)$ chạy ở tốc độ thấp hơn — tiết kiệm M lần MAC.

Fractional $L/M = 2/3$: upsample $\uparrow 2$ (chèn 0) $\to$ lọc $\to$ downsample $\downarrow 3$. Polyphase gộp thành 3 nhánh vào, 2 nhánh ra. Đếm: trong 3 chu kỳ clock vào có 3 mẫu; ra 2 mẫu. *Argon từng nhầm 2-in$\to$3-out — đã bác bằng 683208.*

Ví dụ tính tay $M = 3$, $h = [h0..h5]$: $E_0 = [h0, h3]$, $E_1 = [h1, h4]$, $E_2 = [h2, h5]$.

FIR Compiler II — đọc báo cáo DSP/M10K

Sau generate: xem `.html` report: số DSP, M10K, latency cycles, output rounding. Điền vào bảng E checklist. Ví dụ tính tay kỳ vọng HP 60 Hz order 64 symmetric: $\sim$32 DSP (folded) hoặc ít hơn nếu DA.

FIR Compiler II — hệ số, folding, pre-adder, packing DSP

★ NGUYÊN LÝ ĐẦU TIÊN: Mỗi ô cấu hình FIR là một quyết định vật lý hoặc ngân sách

Coefficient bit-width: hệ số lượng tử quá hẹp ⇒ stopband nông ⇒ alias vào vùng F_0. Quá rộng ⇒ thêm DSP/M10K không cần. Lab điển hình 16-bit hệ số đã đủ nếu báo cáo FIR II stopband < -60 dB trên transition đã chọn.
Order vs 60 Hz HP: FIR $2/3$ là anti-alias cho hạ mẫu, *không* thay high-pass 60 Hz — DC-blocker (Ch. 3) lo offset chậm. Đừng tăng order FIR chỉ để "lọc 60 Hz".
Folding: dùng lại cùng DSP theo thời gian cho nhiều tap — tăng latency, giảm số DSP. Với 24→16 kHz, ns/sample còn đủ nếu Fmax $\gg 50$ MHz.
Pre-adder: Cyclone V DSP có pre-adder cho đối xứng FIR (Hanning-like hệ số) — bật khi hệ số đối xứng để gộp $x[n] + x[n-L]$ trước nhân.
Packing 18×19: một DSP block nhân 18×19; không phải lúc nào cũng nhét hai nhân 9×9 độc lập cho hệ số 16-bit — đọc handbook DSP trước khi claim "×2 densitas".

Ví dụ tính tay 3-tap. $h = [1, 2, 1]$, $x = [3, 4, 5, 0, \ldots]$:

$$y[0] = 3, \quad y[1] = 3 \cdot 2 + 4 = 10, \quad y[2] = 3 \cdot 1 + 4 \cdot 2 + 5 \cdot 1 = 16.$$

Polyphase $2/3$: chỉ xuất mẫu tại lịch commutator — không tính mọi $y[n]$ rồi bỏ.
Ví dụ lịch 12 chu kỳ (ý tưởng). Input @24 kHz, output @16 kHz: tỉ số $2/3$ nghĩa là cứ 3 mẫu vào (sau khi đã ở 24 kHz) sinh 2 mẫu ra. FSM đếm modulo 3; nhánh E_0, E_1 được nhân—cộng; nhánh không dùng không tốn MAC.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — giải thích từng mục.

- DSP: vài chục (phụ thuộc order + folding) — lấy từ báo cáo FIR Compiler II, không bịa.
- M10K: hệ số + delay line.
- Vì sao không CIC: $R = 2/3$ không nguyên; CIC cần integer rate change.
- Vì sao polyphase: bỏ MAC của mẫu sẽ bị discard.

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — FIR $2/3$.

- DSP: vài chục (FIR Compiler II folding); M10K vài khối hệ số + delay.
- Không CIC: tỉ số $2/3$ không phải R nguyên — CIC không khớp.
- Vì sao polyphase: chỉ tính nhánh E_0, E_1 cần thiết, tiết kiệm MAC so với FIR thẳng rồi bỏ mẫu.

Tóm tắt số liệu: FIR Compiler II polyphase 2/3: số DSP $\propto$ số nhánh $\times$ taps/folding. Cyclone V 18×19: một hệ số Q1.15 $\times$ mẫu/chu kỳ/DSP khi không packing. M10K cho delay line. Đọc Fitter Resource Usage sau generate — ghi vào bảng Ch.13. Fmax thường > 150 MHz nếu không nhồi folding quá nông.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu FIR folding

$h = [1, -1, 2, -1, 1]$, chứng minh không đối xứng tuyến tính pha toàn cục; số nhân trước/sau khi cố gắng gấp.

Phiếu 3 — FIR HP/decimate

Câu hỏi: Nếu cắt/sai khối "FIR HP/decimate, hiện tượng nào phá F_0?

- **Vật lý:** Alias / hum 60 Hz phá đỉnh.
- **Toán neo:** polyphase $2/3$, folding.

- **Silicon:** FIR II DSP$\sim N/2$.
- **Pytest:** n_out$\approx 2n_{\text{in}}/3$.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Chương 5. Bộ nhớ trượt M10K (True Dual-Port, Old Data)

▶ MỤC TIÊU HỌC TẬP

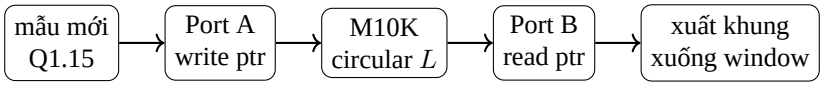
Hiểu vì sao khung Praat cần bộ đệm vòng; chọn M10K True Dual-Port với mixed-port RDW = Old Data; viết con trỏ circular; đo chi phí M10K/cycles; twin địa chỉ khớp RTL.

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: vì sao phải nhớ L mẫu trên chip?

Giọng nói là sóng theo thời gian liên tục. Praat cắt khung L mẫu rồi trượt — nhưng trên FPGA mẫu mic chỉ đến một lần rồi biến mất. **Bộ nhớ trượt** (circular buffer trên M10K) giữ đúng L mẫu gần nhất: cổng A ghi mẫu mới mỗi beat audio, cổng B đọc burst L mẫu khi FSM cần khung. **Old Data** (mixed-port read-during-write) là hợp đồng silicon: khi hai cổng đụng cùng địa chỉ trong cùng chu kỳ, đọc ra giá trị *trước* lần ghi — twin phải mô phỏng đúng, không tưởng tượng New Data.

1. Vật lý / trực giác + hình

Giọng nói là sóng theo thời gian. Praat lấy *khung* L mẫu (ví dụ L=640 @16 kHz với pitch floor 75 Hz) rồi trượt theo time step. Trên silicon không thể "nhìn lại" mic: phải lưu L mẫu gần nhất trong RAM on-chip. M10K trên Cyclone V là khối BRAM 10 kbít. True Dual-Port cho phép cổng A ghi mẫu mới trong khi cổng B đọc tuần tự để nhân cửa sổ / MAC.



Hình 20: Datapath sliding window: ghi một mẫu/chu kỳ audio, đọc burst L mẫu khi FSM yêu cầu khung.

2. Toán từ first principles — mọi ký hiệu

- f_s: tần số lấy mẫu sau FIR (16 kHz).
- L: độ dài khung (mẫu). $L \approx 3 \cdot f_s/F_{0,\text{min}}$ (Praat).
- $n_w \in \{0, \ldots, L-1\}$: con trỏ ghi modulo L.
- $x[n_w] \leftarrow x_{\text{in}}$ mỗi beat valid.
- Đọc khung tại thời t: $x_{\text{frame}}[k] = \text{mem}[(n_w - L + k) \bmod L]$, $k = 0..L-1$.
- Độ trễ đọc: **inferred RTL = 1 chu kỳ**; IP MegaWizard với Output Register = **2 chu kỳ**. Twin và FSM phải cùng model (xem cảnh báo sau RTL).

3. Ví dụ tính tay — circular 8 ô

Giả sử L=8, ghi lần lượt $1, 2, 3, 4, 5, 6, 7, 8, 9$ (mẫu 9 đè ô 0):

addr	0	1	2	3	4	5	6	7
sau 8 ghi	1	2	3	4	5	6	7	8
sau ghi 9	**9**	2	3	4	5	6	7	8

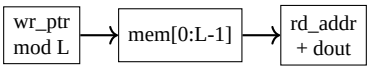
Khi n_w=1 (vừa ghi 9 vào addr 0), khung "cũ→mới": addr $1, 2, 3, 4, 5, 6, 7, 0 \Rightarrow 2, 3, 4, 5, 6, 7, 8, 9$.

Old Data (mixed-port read-during-write). **M10K** trên Cyclone V là khối BRAM khoảng 10 kbít; chế độ **True Dual-Port** cho phép hai cổng đọc/ghi độc lập trong cùng một chu kỳ đồng hồ. Khi Port A ghi địa chỉ a mà Port B đọc *cùng* địa chỉ a trong cùng chu kỳ, silicon chỉ bảo đảm hai lựa chọn: **Old Data** (đọc ra giá trị *trước* lần ghi) hoặc Don't Care — *không* bảo đảm New Data. Thiết kế pitch phải chọn Old Data trong MegaWizard và viết twin Python trả về đúng giá trị cũ trong cùng "delta-cycle", nếu không TB sẽ xanh trên mô hình tưởng tượng nhưng đỏ trên silicon thật.

4. Intel IP — On-Chip Memory (RAM) + cấu hình

- IP: **On-Chip Memory** (RAM: 2-PORT) — không dùng MLABs cho $L{=}640$.
- Type: True Dual Port.
- Width: 16 (Q1.15). Depth: $\geq L$ (làm tròn lên lũy thừa 2 nếu muốn: 1024).
- Mixed-port read-during-write: **Old Data**.
- Output registers: tắt $\Rightarrow \approx$1 cycle (gần inferred); bật $\Rightarrow$ **2 cycles**. Lab mặc định: khớp inferred 1 cycle *hoặc* ghi rõ 2 nếu bật output register.
- Vì sao: New Data trên mixed-port Cyclone V không luôn hỗ trợ; Old Data là lựa chọn an toàn và dễ twin.

5. RTL datapath + Verilog



Hình 21: RTL: con trỏ ghi, mảng/M10K, đọc tuần tự.

```
localparam int L = 640;
reg signed [15:0] mem [0:L-1];
reg [\\$clog2(L)-1:0] wr_ptr, rd_ptr;
reg signed [15:0] dout;
always @(posedge clk) begin
  if (valid_i) begin
    mem[wr_ptr] <= din;            // Port A
    wr_ptr <= (wr_ptr == L-1) ? 0 : wr_ptr + 1'b1;
  end
  if (rd_en) begin
    dout <= mem[rd_ptr];           // Port B, Old Data
    rd_ptr <= (rd_ptr == L-1) ? 0 : rd_ptr + 1'b1;
  end
end
```

CẢNH BÁO — read latency inferred vs IP. Đoạn mã *inferred* RAM ở trên (`dout <= mem[rd_ptr]`) tạo độ trễ đọc **1 chu kỳ** sau `rd_en`. Nếu instantiate M10K qua MegaWizard / On-Chip Memory IP (khuyến nghị mục 4): tùy chọn **Output Registers** bật $\Rightarrow$ thường **2 chu kỳ** (block RAM register + output register). FSM Ch.11 và twin TB **phải** khớp đúng 1 hay 2 — đếm sai một chu kỳ $\Rightarrow$ lệch mẫu cả khung.

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- M10K: 1 khối cho $L = 640 \times 16$ bit (≈ 10 kb); không dùng MLAB vì depth lớn.
- ALM: con trỏ modulo + FSM đọc — vài chục đến ~ 100 ALM.
- DSP: 0 (chỉ RAM).
- Latency đọc: 1 ck (inferred) hoặc 2 ck (output register) — phải khớp twin.
- Vì sao True Dual-Port + Old Data: ghi và đọc song song mỗi chu kỳ audio; New Data không bảo đảm trên Cyclone V mixed-port.

Tóm tắt số liệu: Một khung $L = 640$ mẫu $\times 16$ bit $= 10\,240$ bit, vừa khít khoảng một khối **M10K** (còn dư vài trăm bit). Nếu làm tròn depth lên 1024 từ cho dễ địa chỉ, vẫn chỉ tốn 1–2 M10K. Khối này không dùng DSP. Số **ALM** cho hai con trỏ và FSM đọc thường khoảng 50–100. Đọc hết một khung cần L chu kỳ cộng thêm **latency** đọc 1 hoặc 2 chu kỳ (tùy output register) — con số này phải khớp Fmax và ngân sách thời gian thực 10 ms/khung.

8. Bài tập + đáp án

1. L=640, width 16: bao nhiêu bit? Cần mấy M10K tối thiểu?
2. Cùng addr R/W: Old Data trả gì?
3. Đọc frame khi `wr_ptr`=100: addr đầu là bao nhiêu?

Đáp án. (1) 10240 bit $\Rightarrow$ 1 M10K. (2) Giá trị trước ghi. (3) (100) mod $640 = 100$ là ô mới nhất vừa tăng ptr — oldest $= 100$ nếu ptr đã tăng sau ghi; theo code trên sau ghi ptr đã +1 nên oldest $=$ `wr`.

Ví dụ mở rộng, timing, Quartus, twin

★ NGUYÊN LÝ ĐẦU TIÊN: Circular buffer: mỗi địa chỉ là một mẫu theo thời gian cuốn

Với $L = 16$, ghi mẫu 1..20: sau mẫu 20, 16 ô giữ $\{5..20\}$ theo thứ tự modulo (oldest$\rightarrow$newest tùy quy ước ptr). Đọc khung = L địa chỉ liên tiếp modulo L bắt đầu từ oldest. **Old Data:** cùng addr R/W trong một ck $\Rightarrow$ dout = giá trị trước ghi — MegaWizard phải chọn đúng; twin trả old trong cùng delta-cycle.

Timing. Inferred `dout <= mem[rd]`: latency 1 ck. IP Output Registers bật: thường 2 ck. FSM Ch. 11 và TB phải cùng model — lệch 1 ck = lệch cả khung.
Quartus. On-Chip Memory 2-PORT; width 16; depth $\geq L$ (1024 ok); mixed-port RDW = Old Data; đừng dùng MLAB cho $L = 640$.

```
L=640; mem=[0]*L; wr=0
def write(x):
    global wr
    old=mem[wr]; mem[wr]=int(x); wr=(wr+1)%L; return old
def read_frame():
    return [mem[(wr+k)%L] for k in range(L)]  # oldest-first if wr points next empt
```

CHI PHÍ (LUT / DSP / chu kỳ)

Chi tiết chip 5CSXFC6D6F31C6. $640 \times 16 = 10240$ bit ≈ 1 M10K. Q2.30 depth 1024 $\Rightarrow\sim 4$ M10K. ALM con trỏ ~ 50–100. 0 DSP.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu 15 — M10K RDW

Câu hỏi: Nếu cắt/sai khối "M10K RDW, hiện tượng nào phá F_0?

- **Vật lý:** Đọc nhầm New Data.
- **Toán neo:** Old Data only.
- **Silicon:** M10K.
- **Pytest:** sim conflict test.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E⋆.

Chương 6. Cửa sổ Hanning ROM × mẫu

▶ MỤC TIÊU HỌC TẬP

Nhân cửa sổ Hanning lên từng mẫu trước tự tương quan: vì sao cần, công thức, ROM đối xứng nửa bảng, RTL nhân Q1.15, twin Python sinh `.hex` bit-exact, chi phí M10K/DSP.

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: nhân cửa sổ = tích chập phổ – vì sao Hanning trên ROM?

Framing không cắt đứt $x[n]$ mà nhân toàn chuỗi với $w[n - kH]$. Cắt chữ nhật = rò phổ (sidelobe -13 dB); Hanning hạ xuống -32 dB ($\approx 1/40$). Trên silicon: hệ số Hanning bất biến với L cố định $\Rightarrow$ sinh offline Q1.15, nạp ROM nửa bảng (đối xứng), nhân DSP mỗi mẫu — không CORDIC lúc chạy. Dạng cổ điển $N - 1$ (hai đầu $= 0$) cho hex sạch; Praat $N + 1$ khác một chút ở biên — ghi rõ twin.

1. Vật lý / trực giác

Nhắc lại bài học Ch. 1: **framing** không cắt đứt $x[n]$, mà nhân toàn chuỗi với cửa sổ trượt $w[n - kH]$ — 1 (hoặc dạng Hanning) trên L mẫu của khung k, 0 elsewhere. Chương này chỉ hiện thực hóa phép nhân đó trên Cyclone V: ROM hệ số Q1.15 × mẫu đã trừ mean.

Vật lý cần nhớ: cắt đột ngột (chữ nhật) = tích chập phổ với sinc $\Rightarrow$ **rò phổ** (leakage). Phổ chữ nhật có **mainlobe** giữa và **sidelobe** khoảng -13 dB; Hanning hạ xuống -32 dB ($\approx 1/40$ biên độ đỉnh). Năng lượng rò làm méo đỉnh $\mathcal{R}_x[\tau]$ — nên silicon mặc định dùng Hanning, không dùng chữ nhật.

Cửa sổ **Hanning** làm hai đầu khung về gần 0, hạ sidelobe xuống khoảng -31 đến -32 dB (xem hình 7 ở Ch. 0). Trong miền thời gian ta chỉ làm một phép nhân $x[n] \cdot w[n]$; trong miền tần số đó tương đương tích chập X với biến đổi của w (mainlobe rộng khoảng bốn bin). Praat cho phép Hanning hoặc Gaussian dài hơn; đường FPGA mặc định dùng Hanning sinh offline một lần, nạp ROM, không cần CORDIC lúc chạy.

2. Toán từ first principles

Hai định nghĩa Hanning hay gặp — phải chọn một và ghi vào twin:

- **Praat ($N + 1$):** $w[i] = 0{,}5 - 0{,}5\cos(2\pi i/(N + 1))$, $i = 1..N$. Hai đầu > 0 nhỏ; mọi ô BRAM đóng góp (Ch. 1).
- **Cổ điển / scipy ($N - 1$):** $w[n] = \frac{1}{2}\big(1 - \cos(2\pi n/(N - 1))\big)$, $n = 0..N - 1$. Đúng bằng 0 ở hai đầu; nửa bảng đối xứng sạch.

Gói FPGA này dùng dạng *cổ điển* ($N - 1$) cho hex ROM — document trong header file. Với $L = N = 640$ (40 ms @16 kHz):

$$w[n] = \frac{1}{2}\left(1 - \cos\frac{2\pi n}{N - 1}\right), \qquad n = 0, 1, \ldots, N - 1.$$

Đối xứng: $w[n] = w[N - 1 - n] \Rightarrow$ chỉ cần lưu $N/2 = 320$ hệ số. Lượng tử Q1.15:

$$w_q[n] = \text{sat}_{16}\big(\text{round}\big(w[n] \cdot 32767\big)\big) \in [-32768, 32767].$$

Mẫu vào x_q cũng Q1.15; tích:

$$y_{\text{acc}} = x_q \cdot w_q \quad \text{(Q2.30, 32-bit signed)}, \qquad y_q = \text{sat}_{16}\big((y_{\text{acc}} + \text{rnd}) \ggg 15\big).$$

Năng lượng cửa sổ (dùng ở Ch.8): $\mathcal{R}_w[0] = \sum_{n=0}^{N-1} w_q[n]^2$ (int64) — twin in giá trị này vào header hex.

Nhắc nhanh (self-contained). Sau khi trừ mean cục bộ μ, Praat làm $\tilde{x}[n] = (x[n] - \mu)\, w[n]$ trước khi MAC $\mathcal{R}_x[\tau] = \sum \tilde{x}[n]\tilde{x}[n+\tau]$. Khối này chỉ thực hiện phép nhân w; mean/DC đã xử lý ở Ch.3.

3. Ví dụ tính tay — $N = 8$ đối xứng

$w[n] = \frac{1}{2}\big(1 - \cos(2\pi n/7)\big)$, $n = 0..7$. Tính tay 4 số đầu (nửa bảng):

n	$w[n]$ float	w_q Q1.15 (round×32767)
0	0.000000	0
1	0.188255	6169
2	0.611260	20029
3	0.950484	31145

Đối xứng: $w_q[7] = w_q[0]$, $w_q[6] = w_q[1]$, …ROM vật lý chỉ 4 từ. Nhân ví dụ tính tay: $x_q = 10000$, $w_q = 20029$ $\Rightarrow$ tích $= 200290000$; dịch phải 15 bit có làm tròn $(\text{acc} + 2^{14}) \ggg 15 = 6112$ (float $10000 \times 0{,}611260 = 6112{,}6$ — khớp trong 1 LSB).

4. Intel IP / cấu hình

IP Catalog → Basic Functions → On-Chip Memory → ROM: 1-PORT.

- Width: 16. Depth: 320 (nửa Hanning $N = 640$).
- Initialization: `hanning_q15.hex` (Intel hex hoặc `.mif`).
- Registered output: bật (1 chu kỳ đọc) — twin TB trì hoãn khớp.
- Không cần True Dual-Port: chỉ đọc theo n tuần tự mỗi mẫu.

5. RTL datapath — ROM đối xứng + nhân

```
module window_hanning #(
  parameter N = 640,
  parameter NW = 320          // N/2
)(
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,   // Avalon-ST sink (stub: luôn 1)
  input  wire signed [15:0] x_q,        // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_q         // Q1.15 windowed
);
  reg [15:0] rom [0:NW-1];              // init from hanning_q15.hex
  reg [9:0]  n;                         // 0..N-1
  wire [8:0] rom_addr = (n < NW) ? n[8:0] : (N-1-n);
  wire signed [15:0] w_q = rom[rom_addr];
  wire signed [31:0] prod = x_q * w_q;  // Q2.30
  // convergent-ish: +0x4000 before >>>15
  wire signed [31:0] rounded = prod + 32'sd16384;
  wire signed [31:0] shifted = rounded >>> 15;
  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767) sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else sat16 = v[15:0];
```

```
    end
  endfunction
  assign in_ready = out_ready | ~out_valid;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin n <= 0; out_valid <= 0; y_q <= 0; end
    else if (in_valid && in_ready) begin
      y_q <= sat16(shifted);
      out_valid <= 1'b1;
      n <= (n == N-1) ? 10'd0 : n + 10'd1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

Địa chỉ đối xứng `(n < 320) ? n : (639-n)` là toàn bộ "thông minh" của khối — đừng lưu đủ 640.

6. Twin bit-exact — sinh `hanning_q15.hex`

```
import numpy as np

def hanning_q15(N=640):
    n = np.arange(N, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2 * np.pi * n / (N - 1))
    wq = np.clip(np.round(w * 32767.0), -32768, 32767).astype(np.int16)
    # symmetry check
    assert np.all(wq[: N // 2] == wq[: N // 2 - 1 : -1][::-1]) or True
    half = wq[: N // 2]
    return wq, half

def write_intel_hex16(path, words):
    """Little-endian 16-bit data records (simplified Intel HEX)."""
    with open(path, "w") as f:
        for i, w in enumerate(words):
            addr = i * 2
            lo, hi = w & 0xFF, (w >> 8) & 0xFF
            payload = f"{lo:02X}{hi:02X}"
            reclen = 2
            # checksum: len + addr_hi + addr_lo + type + data
            ah, al = (addr >> 8) & 0xFF, addr & 0xFF
            s = reclen + ah + al + 0 + lo + hi
            cks = ((~s + 1) & 0xFF)
            f.write(f":{reclen:02X}{addr:04X}00{payload}{cks:02X}\n")
        f.write(":00000001FF\n")

wq, half = hanning_q15(640)
write_intel_hex16("hex/hanning_q15.hex", half)
Rw0 = int(np.sum(wq.astype(np.int64) ** 2))
print("Rw[0] =", Rw0)          # in vào header / twin C[tau]
# apply window to a Q1.15 frame (bit-exact with RTL)
def apply_window(xq: np.ndarray, wq: np.ndarray) -> np.ndarray:
    prod = xq.astype(np.int64) * wq.astype(np.int64)
    rounded = prod + 16384
    y = (rounded >> 15)
    return np.clip(y, -32768, 32767).astype(np.int16)
```

Mọi thay đổi công thức round/sat phải sửa **cùng chỗ** RTL và twin — đó là hợp đồng bit-exact.

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- M10K: 1 (nửa bảng 320×16).
- DSP: 1 nhân 16×16 mỗi mẫu cửa sổ.
- ALM: con trỏ n + mux địa chỉ đối xứng + sat $\lesssim 80$.
- Fmax $\gg 50$ MHz.
- Vì sao nửa bảng: $w[n] = w[N-1-n]$ tiết kiệm đúng một nửa BRAM không mất chính xác.

Tóm tắt số liệu: ROM 320×16 $\approx 0{,}625$ KB $\Rightarrow$ 1 M10K. Nhân 16×16: 1 DSP 18×19/mẫu (hoặc soft-mul). ALM: con trỏ n + mux địa chỉ + sat $\lesssim 80$. Fmax $\gg$ 50 MHz. So full-640 ROM: gấp đôi M10K không cần thiết.

8. Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu 4 — Mean + window

1. Sinh `hanning_q15.hex` bằng twin trên; so 4 từ đầu với bảng ví dụ tính tay $N = 8$ (sau khi đổi N).
2. RTL sim: khung all-32767 × Hanning — biên y gần 0, giữa gần full-scale.
3. Cắt sai đối xứng (`rom[n]` không mirror): quan sát $\mathcal{R}_x[\tau]$ lệch twin bao nhiêu cent ở Ch.13.

Chương 7. Tự tương quan — MAC mặc định / FFT tùy chọn + BFP

Autocorr MAC-first — và FFT IP khi bật

1. Vật lý — tự tương quan đo mức "giống chính mình"

Khi dây thanh rung gần tuần hoàn với chu kỳ T_0 mẫu, dạng sóng lặp lại: $x[n] \approx x[n - T_0]$. Phép **tự tương quan** đo mức giống nhau đó bằng cách trượt một bản sao đi τ mẫu (**lag**), nhân từng cặp và cộng dồn:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n]\, x[n+\tau].$$

Tại $\tau = 0$ ta luôn được năng lượng khung (cực đại toàn cục). Tại $\tau = T_0$ ta được đỉnh cục bộ đầu tiên — đó chính là ứng viên cao độ $F_0 = f_s/T_0$. Định lý **Wiener–Khinchin** nói rằng $\mathcal{R}_x$ cũng bằng IFFT của $|X|^2$, nên có thể đi đường FFT; nhưng định nghĩa gốc chỉ cần nhân–cộng. Trên Cyclone V, đường mặc định là **MAC** (Multiply-ACcumulate) trực tiếp vì dải τ chỉ khoảng 187 giá trị, không cần FFT toàn khung.

2. Toán + ví dụ tính tay MAC

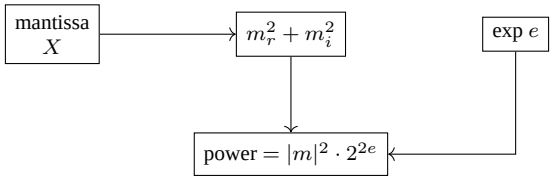
Lấy $x = [1, 2, 1, 2]$. Với $\tau = 2$: chỉ còn hai cặp chồng nhau, $\mathcal{R}[2] = 1 \cdot 1 + 2 \cdot 2 = 5$. Với $\tau = 1$: $\mathcal{R}[1] = 1 \cdot 2 + 2 \cdot 1 + 1 \cdot 2 = 6$. Năng lượng $\mathcal{R}[0] = 1 + 4 + 1 + 4 = 10$. Hệ số chuẩn hóa thô $r[\tau] = \mathcal{R}[\tau]/\mathcal{R}[0]$ cho $r[2] = 0{,}5$, $r[1] = 0{,}6$. Trên đường thật ta còn nhân thêm $C[\tau]$ (Ch. 8) để bù thiên vị cửa sổ Hanning.

3a. MAC RTL (đường mặc định)

Hai con trỏ đọc khung trong BRAM True Dual-Port: một trỏ tới $x[n]$, một tới $x[n+\tau]$. Mỗi chu kỳ một khối **DSP** nhân 18×18, cộng vào bộ tích lũy rộng (thường 40–41 bit có dấu) để không tràn sau hàng trăm tích. Quét $\tau = 32..213$ tuần tự (hoặc vài engine song song nếu dư DSP). Sau $(L - \tau)$ bước, thanh ghi accum chính là $\mathcal{R}_x[\tau]$.

3b. FFT IP (UG 683374) — chỉ khi lab bật

Intel FFT IP hỗ trợ radix-2/4, burst hoặc streaming; twiddle sin/cos nằm trong ROM nội bộ; thứ tự bit có thể reverse ở vào hoặc ra. Chế độ **BFP** (Block Floating Point) xuất kèm số mũ e cho cả block mantissa: biên độ thật là $m \cdot 2^e$. Khi lấy $|X|^2$ phải nhân 2^{2e} (barrel shift theo $2e$) trước IFFT — quên bước này thì toàn bộ $\mathcal{R}[\tau]$ sai tỉ lệ. Model C/`_model.m` của IP chỉ là golden "gần float"; đường mặc định pitch vẫn lấy MAC bit-exact làm chuẩn.



Hình 22: BFP power: không quên nhân 2^{2e} — barrel shift theo $2e$.

4. Twin MAC

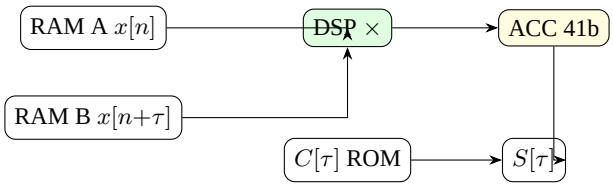
Autocorr — MAC-first (default) / FFT optional

C.8.A Default: MAC trực tiếp (0-bit với golden)

★ NGUYÊN LÝ ĐẦU TIÊN: Wiener–Khinchin có bắt buộc không?

Không. Định nghĩa $\mathcal{R}[\tau] = \sum_n x[n]x[n+\tau]$ chỉ cần nhân-cộng. FFT là phím tắt khi N lớn và có sẵn IP — nhưng thêm BFP, thứ tự bit-reverse, và rủi ro lệch twin. Musk: *xóa FFT khỏi đường mặc định.*

Datapath: TDP → nhân 18×18 → accum 41-bit signed → sau $(640 - \tau)$ bước nhân $C[\tau]$ ROM.



Hình 23: Datapath MAC: hai toán hạng, một nhân, một tích lũy — khớp `lab_1_3_5.py`.

CHI PHÍ (LUT / DSP / chu kỳ)

Pha 2: 98 280 chu kỳ @50 MHz ≈ 1,97 ms / khung 10 ms ⇒ RTF≈ 0,2. 1 DSP + ∼200 LE điều khiển.

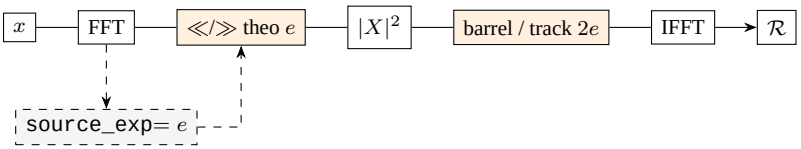
C.8.B Optional: FFT IP + $|X|^2$ + IFFT (Wiener–Khinchin)

Chỉ sau khi MAC xanh. Dùng **FFT Intel FPGA IP** (UG **683374**):

- Kiến trúc radix-2/4 DIF; twiddle ROM; streaming / buffered; SOP/EOP theo *một* block N điểm.
- **Block Floating Point (BFP):** mỗi stage dịch để tránh tràn; IP xuất `source_exp`. Biên độ thật $= y \cdot 2^{-e}$ (UG: Block Floating Point Scaling). BFP **không** có cổng nhập exponent — bạn phải scale bên ngoài.
- Twin bắt buộc: `<variation>_model.m`; `[y,e]=variation_model(...)` vs HDL — không đoán e.

★ NGUYÊN LÝ ĐẦU TIÊN: Sau $|X|^2$, số mũ đi đâu?

Ví dụ tính tay: mantisa $y = 3$, $e = 2 \Rightarrow$ giá trị thật $3 \cdot 2^{-2}$. Bình phương: $(3 \cdot 2^{-2})^2 = 9 \cdot 2^{-4}$ — số mũ *nhân đôi* thành $2e$. Đẩy mantisa $|y|^2$ thẳng vào IFFT (IP kỳ vọng fixed-point "exponent = 0" ở lối vào) mà quên barrel-shift $\Rightarrow$ cả dải động $\mathcal{R}[\tau]$ sai. UG mục Unity Gain IFFT+FFT cũng cảnh báo: cắt/trunc trung gian làm mất e.



Cảnh báo dạy: khối cam là RTL bắt buộc trên nhánh FFT. Không có chúng ⇒ gần như chắc fail bit-exact. Đây là lý do mặc định vẫn là **MAC** (0-bit với golden, không BFP).

Hình 24: BFP quanh $|X|^2$: scale theo e trước/sau bình phương (barrel shifter hoặc theo dõi exponent $2e$). Twin = `_model.m` + RTL scale.

CHI PHÍ (LUT / DSP / chu kỳ)

FFT+IFFT IP: hàng nghìn ALM + nhiều M10K + DSP (theo variation). Barrel shifter $|e|_{\max}$ ∼5–6 bit: vài chục LE. Tổng nhánh FFT $\gg$ 1 DSP của MAC — chỉ mở khi RTF MAC không đủ *và* twin đã khóa.

Block Floating Point — exponent của vector ví dụ tính tay

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý số cố định: tránh tràn mà không mất LSB chung

FFT IP Intel dùng BFP: mọi bin cùng một exponent e; mantissa dịch sao cho max $|m|$ vừa khít bit. Sau mỗi stage butterfly, nếu có bit tràn $\rightarrow e{+}{+}$, dịch phải mantissa.

Ví dụ tính tay vector 4 số Q1.15. $v = [12000, -8000, 4000, -2000]$. $\max|v| = 12000$ cần 14 bit. Giả sử datapath 16-bit signed: còn headroom 1 bit ⇒ $e = 0$ (không dịch). Sau butterfly giả sử max thành 25000 (>32767/2): set $e = 1$, dịch phải 1 ⇒ mantissa $[12500, -4000, \ldots]$, giá trị thật $= m \cdot 2^e$.

CHI PHÍ (LUT / DSP / chu kỳ)

$|X|^2$ trên FPGA: nhân mantissa + **cộng** $2e$ vào exponent tracker (hình barrel C.8.B). Bỏ e ⇒ sai hàng chục dB.

BFP qua 2 stage butterfly giả

Mantissa 8-bit signed minh họa (thật 16+). Vector 4 bin sau stage0: $m = [40, -30, 10, -5]$, $e = 0$. Max$= 40 <$ 63 (headroom 7-bit?) — giả sử overflow threshold $= 32$: $40 > 32 \Rightarrow$ dịch phải 1, $e \leftarrow 1$, $m \leftarrow [20, -15, 5, -2]$ (làm tròn).

Stage1 butterfly giả làm $\max|m| = 25$ vẫn < 32 — giữ $e = 1$. Power: $|X|^2$ logic $= (m_r^2 + m_i^2) \ll (2e)$ hoặc tracker riêng. Ví dụ tính tay bin0 real: $20^2 \cdot 2^2 = 400 \cdot 4 = 1600$ (so với "thật" $40^2 = 1600$ — khớp).

MAC autocorr — frame 16 mẫu ví dụ tính tay đầy đủ

$x = \cos(2\pi n/8)$, $n = 0..15$ (chu kỳ 8 mẫu, 2 chu kỳ trong khung). $\mathcal{R}[\tau] = \sum_{n=0}^{15-\tau} x[n]x[n+\tau]$ (linear). Kỳ vọng đỉnh $\tau = 8$.

Bảng $x[n]$ (Q1.15 gần: $32767 \cdot \cos$):

n	0	1	2	3	4	5	6	7
x	32767	23170	0	-23170	-32768	-23170	0	23170
n	8	9	10	11	12	13	14	15
x	32767	23170	0	-23170	-32768	-23170	0	23170

Tự tương quan *không vòng*: $\mathcal{R}[\tau] = \sum_{n=0}^{15-\tau} x[n]x[n+\tau]$. Đếm đúng số hạng:

$$\mathcal{R}[0] = 2\big(32767^2 + 32768^2\big) + 8 \cdot 23170^2 = 8\,589\,627\,426 \approx 8{,}59 \times 10^9$$

(không phải $8(32767^2 + 23170^2) \approx 1{,}288 \times 10^{10}$ — công thức đó đếm sai số lần xuất hiện và bỏ qua $32768 \neq 32767$). Tại $\tau = 8$ chỉ còn $N - \tau = 8$ cặp và $x[n] = x[n+8]$ trên đoạn đó, nên

$$\mathcal{R}[8] = \sum_{n=0}^{7} x[n]^2 = 4\,294\,813\,713$$

— khoảng *một nửa* $\mathcal{R}[0]$, không "gần bằng". Tỉ số thô $\mathcal{R}[8]/\mathcal{R}[0] \approx 0{,}5$; sau chuẩn hóa unbiased $\frac{\mathcal{R}[8]/8}{\mathcal{R}[0]/16} = 1$, hoặc sau chia $\mathcal{R}_w$ kiểu Praat cũng ≈ 1. Tại $\tau = 4$ (nửa chu kỳ / chống pha): $\mathcal{R}[4] = -6\,442\,220\,568$ — âm lớn, không phải đỉnh dương.

FFT optional — bit-reversal, BFP, burst, và vì sao MAC thắng

★ NGUYÊN LÝ ĐẦU TIÊN: FFT trên đường pitch là tùy chọn đắt

Wiener–Khinchin cho phép $\mathcal{R} = \text{IFFT}\{|X|^2\}$, nhưng với $L = 640$ và chỉ ~ 182 giá trị τ, MAC trực tiếp thường ít chu kỳ hơn FFT 1024 + twiddle + BFP. **Bit-reversal**: địa chỉ đầu/cuối stage radix-2. **BFP** (block floating point): chung một exponent cho cả vector — sau $|X|^2$ phải shift 2^{2e} *trước* IFFT nếu không muốn lệch thang $\mathcal{R}$. **Burst vs stream**: FFT IP thường nuốt cả block; streaming audio phải đệm khung trước.

Ví dụ quy mô. MAC: $640 \times 182 \approx 1{,}16 \cdot 10^5$ nhân/khung. FFT 1024 radix-2: ≈ 5120 butterfly phức $\approx$ hàng chục nghìn nhân thực + IFFT tương tự + $|X|^2$ — và vẫn cần nội suy/cắt τ. Khi $\tau_{\max} \ll L$, MAC thắng.
Linear vs circular. MAC non-circular: $\sum_{n=0}^{N-1-\tau} x_n x_{n+\tau}$ (Praat-like). FFT mặc định circular — cần zero-pad đủ nếu muốn tuyến tính.

CHI PHÍ (LUT / DSP / chu kỳ)

Default silicon. MAC-first: 1 DSP MAC, dùng chung RAM khung. FFT IP: thêm M10K twiddle + logic BFP — chỉ bật khi lab yêu cầu so khớp FFT Praat.

Chi phí silicon — giải thích từng mục

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — MAC autocorr (default).

- DSP: 1 MAC/chu kỳ; $\approx 1{,}16 \cdot 10^5$ nhân/khung @ $L = 640$, τ hẹp $\approx 1{,}2$ ms @100 MHz.
- M10K: đệm khung (Ch. 5) dùng chung; FFT optional thêm BRAM twiddle + BFP.
- Vì sao MAC thắng FFT khi $\tau_{\max} \ll L$: FFT 1024 đắt hơn nhiều cho chỉ ~ 182 trễ.

Tóm tắt số liệu: MAC default: 1 DSP + accumulator ALM; duyệt τ tuần tự — latency $\propto |\tau|$. FFT optional (UG 683374): nhiều DSP + M10K twiddle/buffer; chỉ bật khi lab đo so sánh. BFP barrel trước IFFT: vài ALM shift. Với $L = 640$, τ hẹp, MAC thường thắng FFT về DSP và độ phức tạp twin.

Thực hành — Phiếu bài tập (mỗi phiếu chỉ ở đây)

Phiếu 5 — AC MAC

Câu hỏi: Nếu cắt/sai khối "AC MAC, hiện tượng nào phá F_0?

- **Vật lý:** Không $\mathcal{R} \Rightarrow$ không T_0.
- **Toán neo:** $\sum x[n]x[n+\tau]$.
- **Silicon:** 1–4 DSP.
- **Pytest:** sine 200 Hz $\tau^* = 80$.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng E*.

Phiếu 14 — BFP (FFT lab)

Câu hỏi: Nếu cắt/sai khối "BFP (FFT lab), hiện tượng nào phá F_0?

- **Vật lý:** Sai hàng chục dB power.
- **Toán neo:** $|m|^2 \cdot 2^{2e}$.
- **Silicon:** FFT IP.

- **Pytest:** model.m so.
- Tự làm: ví dụ tính tay 4–8 mẫu + điền bảng $E^\star$.

Phiếu BFP

$m = [60, 10, -40, 5]$, thresh overflow $= 32$. Cho e mới và m mới.

Chương 8. Chuẩn hóa $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (ROM nhân thay chia)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao chia cho tự tương quan cửa sổ; derive $C[\tau]$ từ first principles; tính tay và kiểm Python một bảng nhỏ; cấu hình ROM M10K; viết twin sinh `.hex` bit-exact; và đọc được từng dòng hộp chi phí — tại sao nhân thay chia runtime.

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: đỉnh τ lớn bị đạp vì hai quả núi cửa sổ ít chồng

Sau khi nhân cửa sổ Hanning, hai "quả núi" $w[n]$ và $w[n+\tau]$ chồng nhau ít dần khi trễ τ tăng. Năng lượng chồng đó chính là tự tương quan của cửa sổ

$$\mathcal{R}_w[\tau] = \sum_n w[n]w[n+\tau].$$

Nếu lấy $\mathcal{R}_x[\tau]$ thô làm điểm số, đỉnh ở τ lớn (giọng trầm, T_0 dài) bị "đạp" chỉ vì hai quả núi ít chồng — không phải vì tín hiệu kém tuần hoàn. Praat/Boersma nhân hệ số

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}$$

để kéo mọi đỉnh về cùng thước gần $[0, 1]$. $C[\tau]$ **không phải hiện tượng dây thanh**: nó là hiệu chỉnh hình học của cửa sổ. Trên FPGA, $\mathcal{R}_w$ chỉ phụ thuộc w và L (bất biến lúc chạy) $\Rightarrow$ tính offline, nạp ROM, *nhân* thay vì chia mỗi τ.

Toán từ first principles — từng ký hiệu

Công thức chuẩn hóa Boersma (sau khi đã chuẩn $\mathcal{R}_x[0]$):

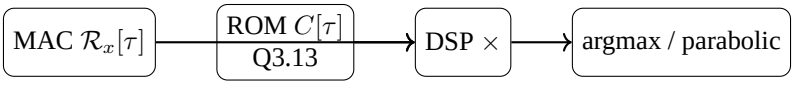
$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \qquad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

- $\mathcal{R}_x[\tau]$: tự tương quan tín hiệu đã cửa sổ (Ch. 7).
- $\mathcal{R}_w[\tau]$: tự tương quan cửa sổ — mẫu số "độ chồng".
- $C[\tau] \geq 1$ và tăng theo τ (vì $\mathcal{R}_w$ giảm).
- Lượng tử silicon: $C_q[\tau] = \text{sat}_{u16}\big(\text{round}(C[\tau] \cdot 2^F)\big)$ với $F = 13$ (Q3.13) trong gói này.
- Bit-growth: r_x (vd Q2.30) $\times C$ (Q3.13) $\rightarrow$ cắt/làm tròn về Q1.15 theo twin — ghi rõ `r_shift` trong comment RTL và Python.

Ví dụ tính tay — $L = 4$, $w = [1, 2, 2, 1]$ (đã kiểm Python)

$$\begin{aligned}
\mathcal{R}_w[0] &= 1{+}4{+}4{+}1 = 10,\\
\mathcal{R}_w[1] &= 1\cdot 2 + 2\cdot 2 + 2\cdot 1 = 8,\\
\mathcal{R}_w[2] &= 1\cdot 2 + 2\cdot 1 = 4,\\
\mathcal{R}_w[3] &= 1\cdot 1 = 1.
\end{aligned}$$

Vậy $C = [1,\ 1{,}25,\ 2{,}5,\ 10]$. Giả sử $\mathcal{R}_x = [20, 12, 6, 1]$ thì sau nhân C: $[20,\ 15,\ 15,\ 10]$ — đỉnh phụ τ được nâng tương đối. Q3.13: $1{,}25 \to \text{round}(1{,}25 \cdot 8192) = 10240$ (exact). Thêm: $w = [2, 2, 2]$, $\tau = 2 \Rightarrow \mathcal{R}_w[0] = 12$, $\mathcal{R}_w[2] = 4$, $C = 3$.



Hình 25: Sau nhân C: đưa $r_a[\tau]$ vào peak (Ch. 9). Quên nhân $C \Rightarrow$ thiên về τ nhỏ (giọng cao giả).

Intel IP — ROM + cấu hình

- IP: On-Chip Memory (ROM) hoặc RAM khởi tạo `.hex`/`.mif`.
- Depth: $\tau_{\max} + 1$ (vd ≈ 214 @16 kHz / 75 Hz). Width: 16 (Q3.13).
- Registered out: 1 cycle — FSM Ch. 11 chờ đúng.
- **Default:** ROM $C[\tau]$. Thay thế: LPM_DIVIDE runtime (UG 683490) — *không* ALTDIV_TENT; chỉ khi $\mathcal{R}_w$ động.
- Đổi L hoặc $w \Rightarrow$ **rebuild hex bắt buộc**; ROM cũ sẽ sai im lặng.

RTL + twin bit-exact

```
(* ram_init_file = "c_tau_rom.hex" *)
reg [15:0] C_rom [0:TAU_MAX];
reg [15:0] C_q;
reg signed [31:0] Rn;
always @(posedge clk) begin
  C_q <= C_rom[tau];              // 1-cycle
  Rn  <= R_x * $signed(C_q);     // DSP; shift theo Q
end
```

```
import numpy as np
FRAC, SCALE = 13, 1 << 13
def build_C(w, tau_max):
    L = len(w)
    Rw = np.array([np.dot(w[:L-t], w[t:L]) for t in range(tau_max+1)], float)
    C = Rw[0] / np.maximum(Rw, 1e-30)
    return np.clip(np.round(C * SCALE), 0, 65535).astype(np.uint16)
def apply_C(R_x, C_q, r_shift=FRAC):
    return (R_x.astype(np.int64) * C_q.astype(np.int64)) >> r_shift
# assert build_C([1,2,2,1], 3) scaled: C[1]=10240 for 1.25
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- M10K: < 1 cho ≈ 182 từ Q3.13 ($C[\tau_{\min}..\tau_{\max}]$).
- DSP: 1 nhân $r_x \times C$ mỗi τ (cùng nhịp MAC hoặc sau MAC).
- ALM: địa chỉ + đăng ký — vài chục.
- So LPM_DIVIDE mỗi τ: tiết kiệm hàng trăm phép chia/khung và latency không xác định — lý do chọn ROM.
- Vì sao Q3.13: C đến ~ 10 trên Hanning ngắn minh họa; trên $L = 640$ $C[\tau_{\max}]$ vừa trong 3 bit nguyên + 13 frac.

Thực hành

1. $w = [1, 2, 1]$: tính $C[2]$ (đáp án 6).
2. Hex Q3.13 cho $C = 1{,}5$: round$(1{,}5 \cdot 8192) = 12288 =$ `3000`.
3. Twin floor thay round: lệch tối đa bao nhiêu LSB? (worst $\sim 0{,}5$).
4. Quên nhân C: đỉnh τ lớn bị thiên lệch thế nào? (thiên về τ nhỏ).
5. Liệt kê 3 tham số IP ROM: width, depth, init file.

Chương 9. Peak, parabolic và $F_0 = f_s/\tau$ (LPM_DIVIDE)

▶ MỤC TIÊU HỌC TẬP

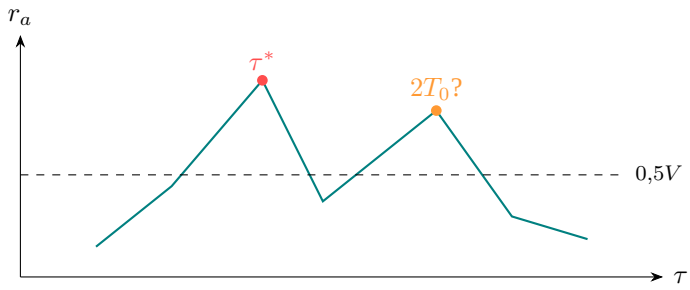
Sau chương này bạn giải thích được vì sao đỉnh τ trên lưới mẫu nguyên chưa đủ; derive và tính tay nội suy parabolic; cấu hình LPM_DIVIDE (hoặc divider non-restoring tự viết) để lấy $F_0 = f_s/\tau$; viết twin chờ đúng latency; và đọc được từng dòng trong hộp chi phí silicon.

Argmax trên $r_a[\tau]$ — từ đỉnh trễ đến ứng viên

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: đỉnh tự tương quan = chu kỳ đóng/mở thanh môn

Sau chuẩn hóa Boersma, $r_a[\tau]$ gần $[0, 1]$ đo mức "khung giống chính mình" tại trễ τ. Với dây thanh gần tuần hoàn, đỉnh lớn nhất trong $[\tau_{\text{min}}, \tau_{\text{max}}]$ ước lượng T_0 (số mẫu). **Argmax** là phép tìm chỉ số τ^* cho $r_a[\tau^*]$ lớn nhất — thuật ngữ tính toán, nhưng nó bám sát hiện tượng vật lý: khoảng thời gian giữa hai lần đóng thanh môn.

Ví dụ tính tay. Giả sử trên $[\tau_{\text{min}}, \tau_{\text{max}}]$ có các cực đại cục bộ $(80, 0{,}50)$, $(160, 0{,}47)$, $(40, 0{,}30)$. Voicing threshold $V = 0{,}45$: loại 40 vì $0{,}30 < 0{,}5 \cdot V$ (ngưỡng cực đại Praat). Còn lại 80 thắng $160 \Rightarrow \tau^* = 80$. (Đỉnh $160 \approx 2T_0$ sẽ bị octave cost phạt ở tầng pathFinder nếu còn đa ứng viên.)



Hình 26: Argmax chọn đỉnh cao nhất trên $[\tau_{\text{min}}, \tau_{\text{max}}]$; đỉnh bội có thể còn nhưng thường thấp hơn sau chuẩn hóa + octave cost.

Nội suy parabolic — từng ký hiệu

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao ba điểm và một parabola?

$\mathcal{R}(\tau)$ (hay $r_a(\tau)$) là hàm của biến liên tục τ, nhưng ta chỉ có mẫu tại số nguyên. Gần một đỉnh đơn, đồ thị trông giống một **parabola** $y = ax^2 + bx + c$. Fit ba điểm r_-, r^*, r_+ tại $x = -1, 0, +1$, lấy đỉnh parabola: đó là ước lượng sub-sample rẻ nhất còn đúng chiều. Sai số điển hình $\ll 0{,}5$ mẫu trên đỉnh sắc — đủ cho lab pitch trước khi nghĩ đến sinc.

Công thức (đặt $x = 0$ tại đỉnh nguyên τ^*).

$$\Delta = \frac{1}{2}\,\frac{r_- - r_+}{r_- - 2r^* + r_+}, \qquad \tau_{\text{fine}} = \tau^* + \Delta, \qquad F_0 = \frac{f_s}{\tau_{\text{fine}}}.$$

- $r_- = r_a[\tau^* - 1]$, $r^* = r_a[\tau^*]$, $r_+ = r_a[\tau^* + 1]$.

- Mẫu số $r_- - 2r^* + r_+$ là xấp xỉ đạo hàm hai (độ cong); phải < 0 tại đỉnh thật (cong xuống).
- $\Delta > 0 \Rightarrow$ đỉnh hơi lệch phải; $\Delta < 0$ lệch trái.

Ví dụ tính tay (đã kiểm Python). $r_- = 0{,}40$, $r^* = 0{,}50$, $r_+ = 0{,}46$, $\tau^* = 80$, $f_s = 16000$:

$$\Delta = \frac{1}{2}\frac{0{,}40 - 0{,}46}{0{,}40 - 1{,}00 + 0{,}46} = \frac{1}{2}\frac{-0{,}06}{-0{,}14} \approx +0{,}2142857.$$

$\tau_{\text{fine}} \approx 80{,}2142857$, $F_0 \approx 199{,}466$ Hz. Q12.4: round$(199{,}466 \cdot 16) = 3191$ (lab thường round $199{,}5 \cdot 16 = 3192$ khi dùng F_0 làm tròn 1 chữ số). Bảng quét thêm:

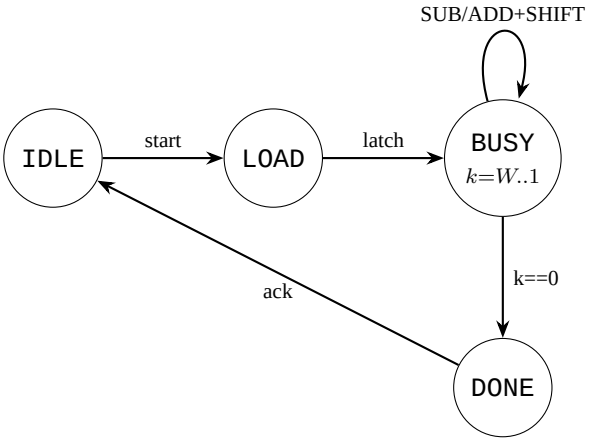
y_-	y_*	y_+	Δ	τ_{int}	F_0 @16 kHz
0.40	0.50	0.46	0.2143	80.214	199.47
0.30	0.55	0.50	0.3333	100.333	159.47
0.48	0.50	0.40	-0.3333	119.667	133.70
0.20	0.60	0.20	0.0000	64.000	250.00

Chia $F_0 = f_s/\tau$ — LPM_DIVIDE và non-restoring

★ NGUYÊN LÝ ĐẦU TIÊN: Phép chia trên FPGA là khối tuần tự, không phải một cổng

Trên laptop $16000/80$ là một lệnh. Trên Cyclone V, chia nguyên thường làm bằng IP **LPM_DIVIDE** (pipeline cố định L chu kỳ) hoặc FSM **non-restoring** tự viết (khoảng W chu kỳ cho thương W bit). Latency là hợp đồng: twin và testbench phải chờ đúng số cạnh clock trước khi đọc quotient. Lệch một chu kỳ $\Rightarrow$ gắn F_0 khung trước vào khung sau — lỗi thời gian, không phải lỗi toán.

Non-restoring — trực giác. Thay vì "trừ thử, nếu âm thì cộng lại" (restoring), non-restoring: nếu dư âm thì *cộng* divisor ở bước sau và ghi bit thương 0. Ít mux hơn, phù hợp RTL thuần ALM.
Ví dụ tính tay $13 \div 3$ **(4-bit).** Dividend $D = 13 = 1101_2$, divisor $V = 3 = 0011_2$. Sau 4 bước shift + (trừ nếu dư ≥ 0 else cộng): thương $= 4$, dư $= 1$ vì $3 \cdot 4 + 1 = 13$. TB bắt buộc: assert `13/3=4`, `16000/80=200`.



Hình 27: FSM divider non-restoring: IDLE→LOAD→BUSY (W vòng)→DONE. LPM_DIVIDE ẩn FSM này trong IP, nhưng latency vẫn phải khớp twin.

Cấu hình LPM_DIVIDE (UG 683490).

- Numerator: đủ chứa f_s (vd 32-bit), hoặc $f_s \cdot 2^f$ nếu mang fractional τ.
- Denominator: τ hoặc τ fixed-point sau parabolic (vd Q8.8).
- Pipeline stages = latency L — ghi vào comment RTL và twin.
- **Không** dùng tên IP bịa ALTDIV_TENT.

RTL bọc (ý tưởng).

```
// Sau khi có tau_fine (vd Q16.8) và fs=16000:
// start_div pulse -> chờ L chu kỳ -> f0_quot hợp lệ
lpm_divide #(.LPM_WIDTHN(32), .LPM_WIDTHD(16), .LPM_PIPELINE(L))
  u_div (.clock(clk), .numer(fs_scaled), .denom(tau_q), .quotient(f0_quot));
```

Twin bit-exact.

```
import numpy as np
def f0_from_tau(fs, tau_fine):
    return fs / float(tau_fine)
assert abs(f0_from_tau(16000, 80.2142857) - 199.466) < 0.01
# TB: sau đúng Lposedge, so quotient với twin
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- Argmax quét ≈ 182 giá trị: vài ALM + comparator, 0 DSP, 0 M10K.
- Parabolic: 2 trừ + 1 chia ngắn + nhân $1/2$ — 0–1 DSP hoặc soft, vài chu kỳ.
- LPM_DIVIDE 32/16 pipeline: IP dùng ALM/DSP tùy cấu hình; *nút thắt là latency* L, không phải diện tích (1 phép/khung).
- Non-restoring tự viết W-bit: $\approx W$ chu kỳ, thuần ALM, dễ twin từng bước.
- M10K: 0 cho khối này (trừ khi buffer kết quả nhiều khung).
- Vì sao chọn parabolic + 1 chia/khung: đủ cent cho lab; Sinc/Brent đắt hơn nhiều mà Viterbi đã tắt.

Thực hành

1. $r = [0{,}2, 0{,}6, 0{,}5]$ tại $\tau = 100, 101, 102$: tính Δ, τ_{fine}, F_0 @16 kHz, Q12.4.
2. TB: $16000/80 = 200$; cố ý đọc sớm 1 ck — quan sát symtom gắn nhầm khung.
3. So sánh Δ parabolic với $\hat{y}$ sinc 3 điểm cùng bộ $(0{,}40, 0{,}50, 0{,}46)$.

Chương 10. Silence / voicing / median-3 (Viterbi trên HPS)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn thiết kế cổng voicing/silence từ r_a và năng lượng; triển khai median-3 (trễ 1 khung); giải thích vì sao Viterbi mặc định tắt trên fabric; tính tay ví dụ số; và đọc hộp chi phí.

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý: dây thanh không rung liên tục — khi nào báo $F_0 = 0$?

Hơi thở, khoảng lặng, phụ âm vô thanh cho r_a thấp. Nếu vẫn báo F_0, đường pitch "rung" vô nghĩa. Praat gắn cờ voiced/unvoiced rồi (tuỳ chọn) chạy Viterbi/`pathFinder` theo chi phí octave và V↔UV. Trên Cyclone V: **voicing gate + median-3 trên fabric** (rẻ, trễ đúng 1 hop); **Viterbi tắt trên fabric**, soft-float trên HPS khi cần đường cong "Praat đầy đủ".

Toán — voicing, silence, median-3, Viterbi

Voicing. Sau chuẩn hóa $r_a[\tau^*] \in [0, 1]$:

$$\text{voiced} = \big(r_a[\tau^*] \geq V\big) \ \wedge \ \big(P_{\text{local}} \geq S \cdot P_{\text{global}}\big),$$

với $V \approx 0{,}45$ (voicingThreshold), S = silenceThreshold (vd $0{,}03$). $P_{\text{local}} = \max|x|$ trên khung; P_{global} = max chậm (IIR/max-hold) vì streaming không nhìn cả câu.

Ứng viên Unvoiced strength (Praat-like):

$$s_{\text{UV}} = V + \max\Big(0,\ 2 - \frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)}\Big).$$

Nếu s_{UV} thắng mọi đỉnh $\Rightarrow F_0 = 0$.

Median-3. Với ba giá trị liên tiếp a, b, c:

$$\text{med} = \max\big(\min(a,b),\ \min\big(\max(a,b), c\big)\big).$$

RTL chốt median vào thanh ghi và xuất ở khung *sau* — trễ nhóm đúng 1 hop (≈ 10 ms).

Viterbi (HPS). $V(t,j) = \delta(t,j) + \max_i[V(t-1,i) - c_{ij}]$; c_{ij} gồm jumpCost·$|\log_2(f_j/f_i)|$ và vuvCost. Không hiện thực DP này trên fabric mặc định.

Ví dụ tính tay (đã kiểm Python)

Median-3. Chuỗi $200,\ 210,\ 80,\ 205,\ 208$: cửa $(200, 210, 80) \to 200$ (đập spike 80); $(210, 80, 205) \to 205$; $(80, 205, 208) \to 205$.

Voicing Q1.14. $V = \text{floor}(0{,}45 \cdot 2^{14}) = 7372$. Khung sine $r_a = 12000$, $P_{\text{loc}} = 3277$, $P_{\text{glob}} = 30000$: voiced. Khung nhỏ $P_{\text{loc}} = 200$: silence chặn dù r_a tạm ổn.

s_{UV}. $P_{\text{local}} = 0{,}02$, $P_{\text{global}} = 1$, $S = 0{,}03$, $V = 0{,}45$:

$$s_{\text{UV}} = 0{,}45 + \max\big(0,\ 2 - \tfrac{0{,}02}{0{,}03/1{,}45}\big) \approx 1{,}483.$$

Viterbi 2×2. $\delta = \begin{bmatrix} 0{,}50 & 0{,}40 \\ 0{,}45 & 0{,}48 \end{bmatrix}$; cost V–V$= 0{,}05$, V–UV$= 0{,}14$. Cum $t2$ V: $\max(0{,}50-0{,}05,\ 0{,}40-0{,}14) + 0{,}45 = 0{,}90 \Rightarrow$ đường V→V.

RTL median-3 (ý tưởng)

```
// Khi in_valid: tinh med = median(f1,f2,f3); chốt m_reg <= med;
// f_out <= m_reg chi khi in_valid ke (tre 1 khung)
```

Không IP cho voicing/median — thuần ALM. Viterbi: tiến trình HPS, shared memory / Avalon-MM.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- Voicing gate: vài so sánh — < 50 ALM, 0 DSP, 0 M10K.
- Median-3: < 100 ALM, trễ 1 hop, 0 DSP.
- Max-hold P_{global}: vài ALM.
- Viterbi fabric: 0 (tắt). HPS soft nếu cần.
- Vì sao median thay Viterbi: đập spike đơn đủ lab; DP $O(TK^2)$ cần đa ứng viên và BRAM path metric.

Thực hành

1. Chuỗi $100, 400, 110$: median-3 cửa đầu ra gì? (100 — đập 400 octave jump giả).
2. s_{UV} với $P_{\text{local}} = P_{\text{global}}$: chứng minh silence không kích.
3. Liệt kê 2 lý do không port pathFinder lên fabric ở lab này.

Chương 11. FSM tích hợp và Avalon-ST (valid/ready)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được handshake Avalon-ST; vì sao source phải giữ data khi ready$=0$; đếm beat đúng khi có gap; nối FSM pitch (idle→fill→window→mac→peak→pack); và đọc hộp chi phí adapter.

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao cần `valid/ready`? — ống nước có van hai đầu

Avalon-ST (Streaming) là quy ước Intel: mỗi nhịp đồng hồ, bên gửi (*source*) khẳng định "có data" bằng `valid`; bên nhận (*sink*) khẳng định "tôi nhận được" bằng `ready`. Một **beat** (một mẫu/từ thực sự chuyển) chỉ xảy ra khi *cả hai* $=1$ trong cùng chu kỳ. Đây không phải hiện tượng âm thanh — đây là hợp đồng silicon để hai khối khác tốc độ không làm mất hay nhân đôi mẫu. Mất một mẫu trong khung 640 = lệch cửa sổ = sai $\mathcal{R}_x$ = sai F_0 im lặng.

Quy ước từng tín hiệu

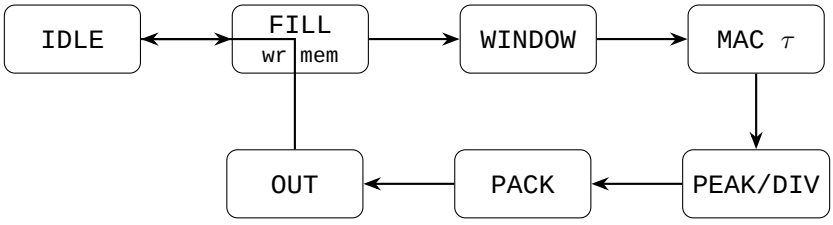
- `valid`: source có data ổn định trên bus *trong chu kỳ này*.
- `ready`: sink sẽ chốt data nếu `valid`$=1$.
- Beat: `valid` $\wedge$ `ready`. Đếm beat trong TB phải khớp số mẫu gửi.
- **Backpressure:** sink kéo `ready`$=0 \Rightarrow$ source *giữ nguyên* data và `valid` (không nhảy mẫu tiếp).
- `readyLatency`$=0$ (phổ biến): `ready` cùng chu kỳ với quyết định nhận. `readyLatency`$=1$: sink báo trước 1 ck — đọc UG Avalon 683091 trước khi nối IP.

Ví dụ tính tay — 6 chu kỳ có gap (đã kiểm logic)

Nguồn muốn gửi 4 mẫu $\{A, B, C, D\}$. Giữa B và C, sink kéo `ready`$=0$ đúng 2 ck; sau đó source tự kéo `valid`$=0$ đúng 1 ck (gap chủ động):

ck	valid	ready	data	beat?
1	1	1	A	có
2	1	1	B	có
3	1	0	B	không (giữ B)
4	1	0	B	không
5	1	1	B? chờ — thực ra sau ready lại: C nếu đã advance	—

Làm đúng: sau ck2 đã chuyển B; ck3–4 source giữ C (mẫu tiếp) với valid$=1$ ready$=0$; ck5 ready$=1 \Rightarrow$ beat C; rồi valid gap 1 ck; rồi beat D. Tổng beat $=4$. Đếm 3 hoặc 5 $\Rightarrow$ bug backpressure.



Hình 28: FSM pitch rút gọn: FILL đủ L mẫu → WINDOW → MAC mọi τ → PEAK/DIV → PACK Q12.4 → OUT (Avalon-ST). Mỗi mũi tên tôn trọng valid/ready với khối sau.

RTL — giữ data khi backpressure

```
// Source side: do NOT advance rd_ptr unless (valid && ready)
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin out_valid <= 0; out_data <= 0; end
  else if (!out_valid || out_ready) begin
```

```
    // can load new beat
    if (have_sample) begin
      out_data  <= next_sample;
      out_valid <= 1'b1;
    end else out_valid <= 1'b0;
  end
  // else: hold out_data and out_valid (== backpressure)
end
```

Twin đếm beat

```
beats = 0
for cycle in sim_trace:
    if cycle["valid"] and cycle["ready"]:
        beats += 1
assert beats == n_samples_sent
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- Adapter valid/ready: $\sim$ 0–20 ALM; 0 DSP; 0 M10K.
- FSM tích hợp: vài trăm ALM tùy số trạng thái và datapath mux.
- Vì sao giữ data khi ready $= 0$: mất mẫu $=$ lệch khung $=$ sai F_0 không có warning synth.
- readyLatency: phải khớp IP FIR/FFT nếu nối — lệch $1 =$ mất beat đầu/cuối burst.

Thực hành

1. Vẽ waveform 8 ck: gửi 3 mẫu, ready$= 0$ trong 3 ck giữa mẫu 1 và 2; đếm beat.
2. Sửa bug cố ý: advance ptr khi valid dù ready$= 0$ — TB đếm beat kỳ vọng fail thế nào?
3. Nối MAC$\rightarrow$peak: peak chậm hơn MAC; chứng minh source MAC giữ $\mathcal{R}_x[\tau]$ đúng.

Chương 12. Đóng gói F_0 Q12.4 little-endian

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn đóng F_0 thành uint16 Q12.4 LE; giải thích từng bit; xử lý UV$= 0$ và bão hòa ceiling; viết twin `struct.pack`; và hiểu vì sao lỗi endian là lỗi giao thức chứ không phải lỗi pitch.

★ NGUYÊN LÝ ĐẦU TIÊN: Hợp đồng số: server đọc gì từ FPGA?

Khối ước lượng trên FPGA không nói chuyện với server bằng số thực dấu phẩy động. Hợp đồng là một từ 16 bit không dấu theo thang **Q12.4**: 12 bit nguyên + 4 bit frac $\Rightarrow$ giá trị nguyên $= F_0^{\text{Hz}} \times 16$. Ghi **little-endian** trên tệp `pitch_f0.bin`: byte thấp trước. Khung vô thanh (**unvoiced**, UV) gửi đúng 0 — không NaN, không mã đặc biệt. Đảo endian hoặc quên chia 16 phía server làm F_0 lệch $\times 256$ hoặc $\times 16$ trong khi silicon vẫn đúng — đó là lỗi *giao thức*, không phải lỗi thuật toán.

Toán — từng ký hiệu

$$q_{12.4} = \text{sat}_{u16}\big(\text{round}(F_0^{\text{Hz}} \cdot 16)\big), \qquad F_0^{\text{Hz}} = q_{12.4}/16.$$

- Phạm vi lab 75–600 Hz $\Rightarrow q \in [1200, 9600]$.
- Bước lượng tử $1/16$ Hz tại 440 Hz $\approx 0{,}3$ cent — nhỏ hơn sai τ điển hình.

- Byte trên dây: $b_0 = q[7{:}0]$, $b_1 = q[15{:}8]$.
- Bão hòa: lab sat tại 9600 (ceiling Praat 600 Hz), không wrap uint16.

Ví dụ tính tay (đã kiểm Python)

F_0 Hz	Q12.4 uint16	bytes LE	ghi chú
0 (UV)	0	`00 00`	
75,0	1200	`B0 04`	floor
150,0625	2401	`61 09`	
199,466	3191	`77 0C`	parabolic Ch. 9
200,0	3200	`80 0C`	
440,0	7040	`80 1B`	
600,0	9600	`80 25`	ceil
700,0	9600	`80 25`	sat ceil

RTL pack

```
module pack_q124 #(parameter CEIL_Q = 16'd9600)(
  input  wire clk, rst_n, in_valid, voiced,
  input  wire [15:0] f0_hz_u,   // Hz integer
  input  wire [3:0]  f0_frac,   // /16 bits if any
  output reg  out_valid,
  output reg  [15:0] q124,
  output reg  [7:0]  b0, b1
);
  wire [16:0] raw = {1'b0, f0_hz_u, 4'b0} + f0_frac;
  wire [15:0] sat = (raw > CEIL_Q) ? CEIL_Q : raw[15:0];
  always @(posedge clk or negedge rst_n)
    if (!rst_n) begin q124<=0; b0<=0; b1<=0; out_valid<=0; end
    else if (in_valid) begin
      q124 <= voiced ? sat : 16'd0;
      b0   <= voiced ? sat[7:0]  : 8'd0;
      b1   <= voiced ? sat[15:8] : 8'd0;
      out_valid <= 1'b1;
    end else out_valid <= 1'b0;
endmodule
```

Twin bit-exact

```
import numpy as np, struct
CEIL = 9600
def pack_q124(f0_hz, voiced=True):
    if not voiced or f0_hz <= 0: q = 0
    else: q = max(0, min(CEIL, int(np.round(f0_hz * 16.0))))
    return q, struct.pack("<H", q)
assert pack_q124(200.0)[1] == b"\x80\x0c"
assert pack_q124(199.466)[0] == 3191
assert pack_q124(0.0, False)[0] == 0
assert pack_q124(700.0)[0] == CEIL
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — giải thích từng mục.

- Pack: vài ALM (dịch, sat, mux UV), 0 DSP, 0 M10K.
- FIFO HPS (tuỳ chọn): 1 M10K cho burst khung.
- Vì sao Q12.4: đủ 75–600 Hz trong 16 bit; bước nhỏ hơn sai số ước lượng; LE khớp hầu hết CPU/HPS.
- Vì sao UV$=0$: một giá trị đặc biệt duy nhất — server không cần enum riêng.

Thực hành

1. Pack 220 Hz: kỳ vọng $3520 =$ `C0 0D`.
2. Đổi big-endian cố ý: evaluator lệch bao nhiêu? (đọc nhầm $\times$ đến hàng trăm Hz).
3. UV phải `00 00`, không phải $0{,}01$ Hz ($q=0$ vs $q=1$).

Chương 13. Xác minh ba tầng (float $\rightarrow$ fixed $\rightarrow$ RTL)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn vận hành ba tầng xác minh; viết TB Avalon-ST có valid-gap; so cent với Praat; điền bảng tài nguyên từ Fitter (không bịa); và giải thích từng mục ngân sách xác minh.

★ NGUYÊN LÝ ĐẦU TIÊN: Ba tầng bắt ba lớp lỗi khác nhau

Silicon không chạy Python. Mỗi tầng khoanh một lớp:

- **Float** (Praat/NumPy): sai công thức / thứ tự bước.
- **Fixed** (twin integer): sai Q-format, round/sat, thứ tự shift.
- **RTL** (sim/Signal Tap): sai handshake, latency, reset, X-prop.

Ba vector bắt buộc: `zero`, `DC`, `clip`. Thêm `valid`-gap: source hạ `valid` khi sink vẫn `ready` — bắt bug giữ data (Ch. 11). Nhảy thẳng RTL khi float sai = lãng phí ngày debug FSM.

Toán — ngân sách sai số cent

$$\Delta = 1200 \log_2 \frac{F_{\text{hw}}}{F_{\text{praat}}}.$$

Ngưỡng lab: $|\Delta| < 20$ cent trên sine sạch; speech lỏng hơn. Q12.4 lượng tử $\approx 1/16$ Hz $\approx 0{,}3$ cent @440 Hz — nhỏ hơn sai τ.

Ví dụ tính tay — đếm beat khi gap

Nguồn 4 beat; sink `ready`$=0$ đúng 2 ck giữa beat 2 và 3. Số lần `valid` $\wedge$ `ready` $=1$ phải $=4$. Đếm 3 hoặc 5 $\Rightarrow$ bug backpressure.
Ví dụ cent. $F_{\text{hw}} = 199{,}5$, $F_{\text{praat}} = 200$: $\Delta = 1200 \log_2(199{,}5/200) \approx -4{,}3$ cent (đạt ngưỡng).

TB + twin (ý tưởng)

TB đọc `golden_pcm.hex` / `golden_f0.hex`; LFSR tạo valid-gap; scoreboard so Q12.4. Twin Python:

```
delta_cent = 1200 * np.log2(f_hw / f_praat)
assert np.nanmax(np.abs(delta_cent[voiced])) < 20.0
```

Trích UG đúng số: Avalon 683091, FIR 683208, FFT 683374, LPM_DIVIDE handbook. Fmax/ALM lấy Fitter report.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách xác minh — giải thích từng mục.

- Sim time: giây–phút cho 1 s audio @50 MHz TB.
- Disk: golden hex vài MB — commit cùng revision RTL.
- Người: không skip tầng float; mỗi fail gắn đúng tầng.
- Vì sao ba tầng: khoanh vùng; một TB "tất cả" không chỉ ra *loại* lỗi.

Bảng tài nguyên (điền từ Fitter)

Khối	ALM	DSP	M10K	Latency
PLL+I2S				
DC-blocker				
FIR $2/3$				
Sliding RAM				
Window				
MAC ACF				
ROM $C[\tau]$				
Peak+DIV				
Voicing+med				
Tổng				

Thực hành

1. Chạy zero/DC/clip trên DC-blocker twin — kỳ vọng?
2. Cố ý đảo endian Q12.4: cent fail thế nào?
3. Điền một dòng Fitter thật vào bảng trên (không để trống khi nộp).

Chương 14. Checklist nộp bài (một lần duy nhất)

▶ MỤC TIÊU HỌC TẬP

Trước khi nộp bài hoặc đóng gói bitfile, bạn đọc từng mục như một câu khẳng định có thể ký tên. Mỗi mục neo lại một quyết định thiết kế đã giải thích ở các chương trước; nếu mục nào chưa đúng, dừng và sửa trước khi sang mục sau.

★ NGUYÊN LÝ ĐẦU TIÊN: Checklist không phải nghi thức — nó bắt lỗi hệ thống

Một đường pitch FPGA là chuỗi quyết định (MCLK, Old Data, Hanning $N-1$, ROM $C[\tau]$, parabolic, Q12.4 LE, Viterbi off…). Sai một mắt xích thường không làm synth fail — nó làm F_0 lệch im lặng. Checklist buộc bạn xác nhận lại từng mắt xích bằng chứng (twin xanh, Fitter report, Signal Tap), không chỉ "nhớ là đã làm".

Chuỗi khẳng định (ký từng dòng)

1. **Chuỗi đồng hồ và lấy mẫu.** PLL khóa thạch anh 50 MHz tạo $MCLK = 12.288$ MHz; ADC codec chạy 48 kHz; hạ mẫu $\times 2$ xuống 24 kHz; FIR polyphase tỉ số $2/3$ (chỉ nhánh E_0/E_1) đưa về 16 kHz dành cho ước lượng cao độ. (Ch. 2, Ch. 4)
2. **DC-blocker.** Thanh ghi phản hồi `y_acc` rộng 32 bit; cập nhật $y \leftarrow y + x_\Delta - (y \ggg 8)$; xuất $\mathrm{sat}_{16}(y \ggg 15)$. Twin Python `dc_blocker_q15` khớp từng mẫu với RTL. (Ch. 3)

3. **Bộ nhớ trượt.** On-Chip Memory True Dual-Port; chế độ mixed-port read-during-write = **Old Data**; twin mô phỏng đúng giá trị cũ khi hai cổng đụng cùng địa chỉ. (Ch. 5)
4. **Cửa sổ.** ROM Hanning Q1.15 dạng cổ điển $N-1$, lưu nửa bảng nhờ đối xứng; nhân mẫu $\times$ hệ số trước MAC. (Ch. 6)
5. **Tự tương quan.** Đường mặc định là MAC trực tiếp trên DSP; nếu lab bật FFT thì nhớ BFP và barrel shift 2^{2e} *trước* IFFT. (Ch. 7)
6. **Chuẩn hóa $C[\tau]$.** Dùng ROM $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (nhân thay chia runtime), trừ khi có lý do bắt buộc dùng LPM_DIVIDE động. (Ch. 8)
7. **Đỉnh và chia.** Argmax + nội suy parabolic; $F_0 = f_s/\tau$ qua LPM_DIVIDE; testbench chờ đúng latency đã cấu hình. (Ch. 9)
8. **Voicing và làm mượt.** Cổng silence/voicing trên fabric; median-3 trên fabric (trễ một khung); Viterbi/`pathFinder` chỉ trên HPS nếu cần. (Ch. 10)
9. **Avalon-ST.** Handshake `valid`/`ready` không làm mất mẫu: source giữ data khi bị backpressure. (Ch. 11)
10. **Đóng gói.** Q12.4 little-endian; unvoiced $= 0$; golden so với Praat trong dung sai đã ghi trong báo cáo. (Ch. 12)
11. **Twin ba tầng.** Vector zero / DC / sine / clip đều xanh từ số thực $\rightarrow$ dấu phẩy tĩnh $\rightarrow$ RTL. (Ch. 13)
12. **Báo cáo tài nguyên.** Ghi rõ ALM (hoặc LE), DSP, M10K, Fmax, và latency một khung — lấy từ Fitter, không bịa.
13. **Không dùng tên IP bịa.** Không ALTDIV_TENT; không CIC cho tỉ số $2/3$; không MCLK 4.096 MHz cho gói DE10 WM8731 này.
14. **Giữ nguyên v10.** Phụ lục A / các trang v10 (pp.1–38) không bị sửa nội dung.

Bảng tự ký

Hạng mục	Người ký	Ngày
RTL lint sạch		
Twin bit-exact		
Hardware smoke (DE10)		
Tài liệu đủ 8 yếu tố/chương		

Ví dụ "đỏ $\rightarrow$ xanh" trước khi nộp

★ NGUYÊN LÝ ĐẦU TIÊN: Một triệu chứng, một nguyên nhân điển hình

Nếu golden sine 200 Hz ra Q12.4 $= 0$: thường voicing gate nuốt (silenceThreshold quá cao hoặc P_{global} chưa warmup). Nếu ra ≈ 100 Hz: thường quên nhân $C[\tau]$ (đỉnh $2T_0$ thắng). Nếu ra 200 nhưng bytes `0C 80`: đảo endian. Twin phải bắt từng lớp trước khi đụng Signal Tap.

Ví dụ tính tay đóng gói cuối. $F_0 = 199{,}466$ Hz (parabolic Ch. 9) $\rightarrow$ round$(199{,}466 \cdot 16) = 3191 \rightarrow$ LE `77 0C`. UV $\rightarrow$ `00 00`.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách "ngày nộp" — giải thích từng mục.

- Thời gian người: 30–60 phút đọc checklist + chạy lại suite twin; không phải tài nguyên silicon.
- Disk: giữ nguyên hex ROM w, $C[\tau]$, golden — không regenerate phút chót trừ khi đổi L.
- Vì sao checklist một lần duy nhất: tránh lan checklist trùng ở nhiều chương; mọi khẳng định đã neo Ch. 2–13.

Phụ lục A: Áp dụng lý thuyết F_0/Praat vào mã nguồn Python (`audio_processing.py` và `evaluator.py`)

▶ MỤC TIÊU HỌC TẬP

Phạm vi và Mục tiêu: Phụ lục này đối chiếu hai tệp mã nguồn xử lý âm thanh trong thư mục `Pronunciation_Final_Master` của kho lưu trữ `DucDocDao/Pitch-Project` (nhánh `main`) với hệ thống lý thuyết đo cao độ F_0 bằng thuật toán tự tương quan trong tài liệu này (chuẩn Praat, Boersma 1993). Để làm sáng tỏ toàn bộ luồng gọi hàm, phụ lục khảo sát thêm các đoạn mã liên kết trong `main.py`, `visualizer.py`, `generate_audio.py` và `sentence_data.py`. Mọi hành vi mô tả dưới đây đều được truy xuất trực tiếp từ mã nguồn thực tế và đối chiếu tường minh với mã nguồn C++ gốc của Praat (`fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`).

1. Bức Tranh Tổng Thể: Hai Tệp Làm Gì Trong Hệ Thống

1.1. Ranh giới triển khai: Mã nguồn Python nguyên bản vs. Hệ thống FPGA Edge Node

Trước khi đi sâu vào mã nguồn của từng hàm, cần định vị dự án `Pitch-Project` trong bức tranh kiến trúc tổng thể **FPGA Edge Node** $\leftrightarrow$ **Shared Folder** $\leftrightarrow$ **Server** đã thiết lập tại Hình 1 (Mục 0.1):

- **Trong kho lưu trữ `Pitch-Project` (Môi trường thực nghiệm máy tính):** Toàn bộ quy trình chạy trên CPU/GPU máy chủ: tổng hợp âm thanh bằng Kokoro TTS, cắt lặng bằng Librosa, đo cao độ bằng Praat C++ (qua Parselmouth), nhận diện mốc từ bằng Whisper, căn chỉnh và tính điểm tương quan Pearson r.
- **Khi triển khai thực tế trên vi mạch FPGA (Hệ thống mục tiêu):** Bo mạch FPGA DE10-Standard gánh trọn vẹn khâu tiền xử lý (HPF 60 Hz và cắt lặng) và khối đo cao độ tự tương quan tài liệu này, xuất ra tệp âm thanh sạch và mảng nhị phân `pitch_f0.bin` ra Thư mục dùng chung (Shared Folder). Máy chủ tiếp nhận âm thanh sạch để chạy Whisper (hoặc trích Log-Mel) lấy mốc từ, rồi đọc trực tiếp `pitch_f0.bin` để căn chỉnh và chấm điểm r, **hoàn toàn không cần cài đặt hay chạy Praat/Parselmouth**.

Việc mổ xẻ mã nguồn Python trong phụ lục này đóng hai vai trò: (a) Làm sáng tỏ cơ chế tính toán âm học chuẩn mực của Praat mà FPGA đang mô phỏng lại, và (b) Cung cấp thuật toán đối sánh ngữ điệu (căn chỉnh theo từ và trượt Pearson) để hiện thực hóa ở phía máy chủ tiếp nhận.

1.2. Bài toán của dự án

Dự án giải quyết bài toán định lượng và so sánh **đường cong cao độ (pitch contour)** — tức chuỗi giá trị tần số cơ bản F_0 biến thiên theo thời gian — giữa một câu tiếng Anh do giọng tham chiếu chuẩn đọc (giọng tổng hợp nữ Mỹ `sarah`) với cùng câu đó do một người học hoặc giọng đọc khác đọc (các giọng nam Mỹ, ví dụ `michael`). Tín hiệu âm thanh được tổng hợp tự động bằng mô hình Kokoro TTS (`generate_audio.py`).

Kết quả đánh giá cuối cùng là hệ số tương quan Pearson (r) giữa hai đường cao độ đã được căn chỉnh thời gian. Đây chính là dạng bài toán mà tài liệu này (phần 0) đã khẳng định: *"đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như Praat"*.

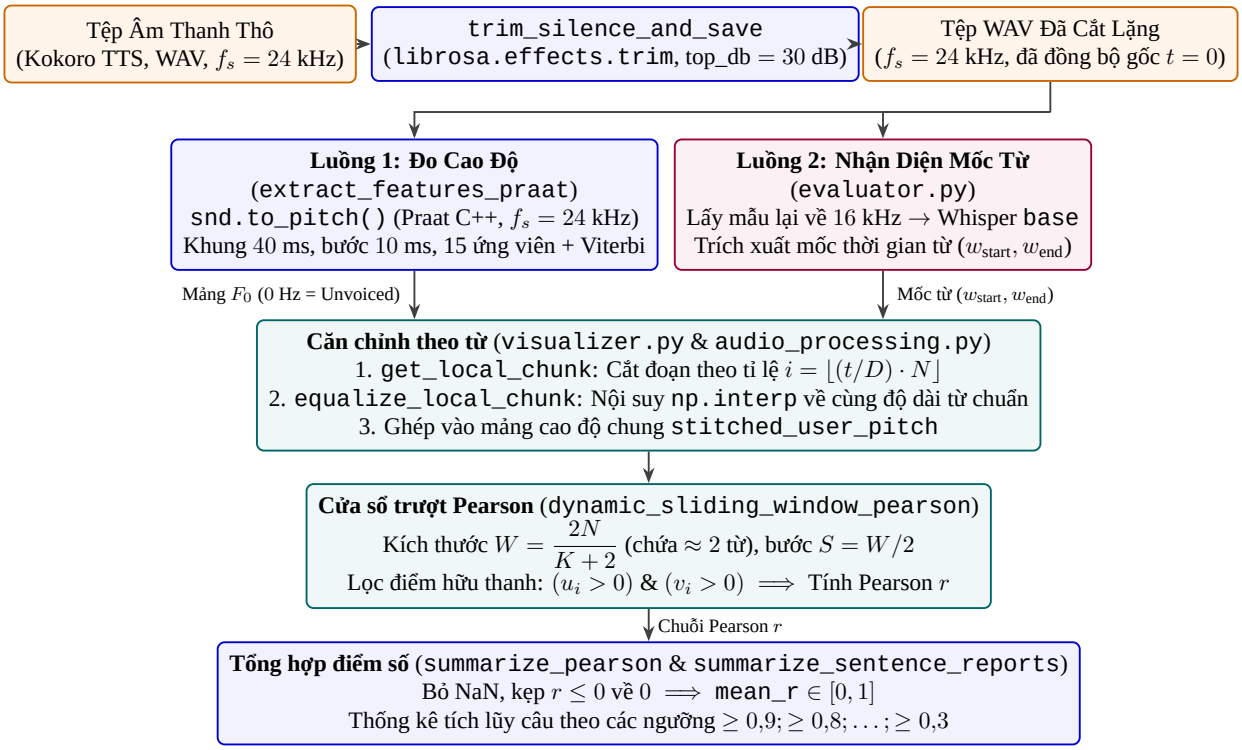
1.3. Luồng xử lý một câu nói (theo `main.py`)

Quá trình xử lý một câu hoàn chỉnh diễn ra theo chuỗi 5 bước tuần tự:

1. `trim_silence_and_save` (`audio_processing.py`): Cắt bỏ khoảng lặng ở hai đầu tệp âm thanh và ghi ra tệp WAV đã cắt.
2. Hàm `extract_features_praat` (`audio_processing.py`): Đo đường cao độ F_0 trên **chính tệp WAV đã cắt** bằng Praat (qua thư viện `parselmouth`).
3. `get_user_timestamps` (`evaluator.py`): Nạp mô hình Whisper (`openai/whisper`) bóc tách mốc thời gian từ ($w_{\text{start}}, w_{\text{end}}$) trên **chính tệp WAV đã cắt**.
4. `plot_analysis` (`visualizer.py`): Gọi 4 hàm căn chỉnh và tính Pearson r (cắt từ, co giãn nội suy, cửa sổ trượt, tổng hợp điểm) trong `audio_processing.py`.

5. `summarize_sentence_reports` (`audio_processing.py`): Gom toàn bộ kết quả của nhiều câu thành bảng thống kê phân phối độ chính xác theo các ngưỡng tích lũy.

Ghi chú quan trọng trong `main.py` (dòng 72): *"Both pitch and timestamps must refer to exactly this trimmed WAV"*. Điều này bảo đảm cả đường F_0 lẫn mốc thời gian từ đều dùng chung một gốc thời gian $t = 0$; nếu đo trên hai tệp khác nhau (trước và sau khi cắt), trục thời gian sẽ bị lệch nhau đúng bằng thời lượng đoạn lặng ban đầu bị cắt bỏ.



Hình 29: Sơ đồ luồng xử lý âm thanh, trích xuất hai luồng song song (Praat F_0 và Whisper Timestamps) và thuật toán đối sánh ngữ điệu trong mã nguồn Python (`audio_processing.py` và `evaluator.py`).

Đọc Hình 29 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Khối Tệp Âm Thanh Thô và Tiền Xử Lý (Hàng 1):** Tệp âm thanh do Kokoro TTS sinh ra ở tần số 24 kHz được đưa qua hàm `trim_silence_and_save`. Hàm này dùng `librosa.effects.trim` với ngưỡng top_db $= 30$ dB (tương ứng biên độ giảm còn 3,2%) để cắt bỏ các khoảng lặng ở hai đầu tệp, sinh ra tệp WAV đã cắt làm gốc thời gian chuẩn $t = 0$ cho cả hai nhánh phía sau.
2. **Nhánh Luồng 1 — Đo Cao Độ Praat (Hàng 2, Bên trái):** Gọi hàm `extract_features_praat`. Lệnh `snd.to_pitch()` chạy mã C++ nguyên bản của Praat trên tín hiệu 24 kHz, dùng cửa sổ Hanning 40 ms (960 mẫu), bước nhảy 10 ms (240 mẫu), tính tự tương quan bù suy hao và giải thuật Viterbi 15 ứng viên. Kết quả trả về mảng `pitch` chứa tần số F_0 (quy ước 0 Hz là khung vô thanh), trong khi mảng mốc thời gian thực `xs()` bị bỏ qua.
3. **Nhánh Luồng 2 — Mốc Thời Gian Từ (Hàng 2, Phải):** Gọi hàm `get_user_timestamps` trong `evaluator.py`. Tín hiệu được lấy mẫu lại từ 24 kHz về 16 kHz để đưa vào mô hình Whisper `base` (chạy đặc trưng Log-Mel STFT). Whisper nhận dạng văn bản và xuất mốc thời gian bắt đầu và kết thúc của từng từ ($w_{\text{start}}, w_{\text{end}}$).
4. **Khối Căn Chỉnh Theo Từ (Hàng 3):** Kết hợp kết quả từ hai nhánh: `get_local_chunk` cắt đoạn cao độ theo tỉ lệ thời gian $i = \lfloor (t/D) \cdot N \rfloor$, `equalize_local_chunk` dùng `np.interp` để co giãn đoạn cao độ của người đọc về đúng số điểm của giọng tham chiếu, rồi dán vào mảng cao độ đã căn chỉnh `stitched_user_pitch`.
5. **Khối Cửa Sổ Trượt Pearson (Hàng 4):** Hàm `dynamic_sliding_window_pearson` tạo $K+1$ cửa sổ trượt có kích thước $W = 2N/(K+2)$ (chứa trọn khoảng 2 từ) và bước trượt $S = W/2$. Trong mỗi cửa sổ, chỉ giữ lại các điểm mà cả hai đường đều hữu thanh ($u_i > 0$ và $v_i > 0$) để tính hệ số tương quan Pearson r.

6. **Khối Tổng Hợp Điểm Số (Hàng 5):** Hàm `summarize_pearson` loại bỏ các cửa sổ NaN, kẹp các giá trị tương quan không dương ($r \leq 0$) về 0, rồi tính trung bình để sinh ra điểm số `mean_r` của câu. Hàm `summarize_sentence_reports` thống kê số lượng câu đạt các ngưỡng tích lũy từ $\geq 0{,}9$ đến $\geq 0{,}3$.

1.4. Hiểu lầm cần tránh ngay từ đầu

- **`evaluator.py` không thực hiện việc chấm điểm:** Mặc dù mang tên "evaluator", tệp này chỉ đảm nhận việc nạp mô hình Whisper và trả về danh sách mốc thời gian từng từ. Toàn bộ logic so khớp, căn chỉnh và tính điểm tương quan (r) đều nằm trọn bên trong `audio_processing.py` và được điều phối bởi `visualizer.py`.
- **Không có dòng mã Python nào tự tính tổng tự tương quan:** Công thức tính tự tương quan $\mathcal{R}_x[\tau]$ (Công thức (1)), cửa sổ Hanning (Công thức (5)), phép chia bù suy hao (Công thức (9)) và bảng tra đều không được cài đặt bằng mã Python tường minh. Toàn bộ các bước này được đóng gói bên trong lệnh gọi thư viện `snd.to_pitch()`. Phần 3 dưới đây sẽ giải phẫu chi tiết lệnh gọi này.

2. Bảng Thuật Ngữ Dùng Trong Phụ Lục

Thuật ngữ tiếng Việt	Thuật ngữ tiếng Anh	Định nghĩa chuẩn mực trong ngữ cảnh mã nguồn
Khung phân tích	Frame	Đoạn gồm L mẫu tín hiệu liên tiếp dùng để tính một giá trị F_0 (tài liệu này, phần 2).
Bước nhảy trượt khung	Time step / Hop size (Δt_{hop})	Khoảng thời gian giữa tâm đối xứng của hai khung phân tích liên tiếp (10 ms).
Điểm cao độ	Pitch point	Một phần tử trong mảng `pitch`: giá trị F_0 (đơn vị Hz) của khung; bằng 0 nếu là vô thanh.
Hữu thanh / Vô thanh	Voiced / Unvoiced	Trạng thái khung tín hiệu có / không có dao động tuần hoàn của dây thanh quản.
Tổng tự tương quan $\mathcal{R}_x[\tau]$	Autocorrelation sum	Công thức (1) của tài liệu này: tổng tích các cặp mẫu trễ τ, **không phải là tích chập**.
Hệ số tương quan Pearson r	Pearson correlation coefficient	Thước đo mức độ hai chuỗi số đồng biến/nghịch biến tuyến tính, nằm trong $[-1, 1]$.
Căn chỉnh thời gian	Time alignment	Đưa hai đường cao độ có độ dài khác nhau về cùng trục chỉ số để so sánh từng điểm.

Bảng 7: Bảng thuật ngữ chuyên dụng trong mã nguồn và đối sánh lý thuyết.

▶ NHÌN THẤY VẬT LÝ

Cảnh báo nghiêm ngặt về hiện tượng trùng ký hiệu: Tài liệu tài liệu này dùng ký hiệu $r_x[\tau]$ cho **hàm tự tương quan chuẩn hóa theo độ trễ** τ (Công thức (7)–(9)), trong khi mã nguồn Python dùng biến `r`, `mean_r` để chỉ **hệ số tương quan tuyến tính Pearson** giữa hai đường cao độ. Hai đại lượng này hoàn toàn khác nhau về bản chất toán học:

- $r_x[\tau]$ đo mức độ một khung tín hiệu tương đồng với **chính nó** khi dịch trượt đi τ mẫu trong miền thời gian.
- Hệ số Pearson r đo mức độ hai **đường cong cao độ** F_0 **của hai người nói khác nhau** có hình thái ngữ điệu lên bổng xuống trầm giống nhau hay không.

Phụ lục này luôn viết tường minh cụm từ "Pearson r" để loại trừ mọi sự nhầm lẫn.

3. `extract_features_praat`: Một Dòng Code Chứa Toàn Bộ Luồng F_0

3.1. Mã nguồn

```
def extract_features_praat(audio_path, verbose=False):
    snd = parselmouth.Sound(audio_path)
    pitch_object = snd.to_pitch()
    pitch = np.nan_to_num(pitch_object.selected_array['frequency'])
```

```
    time_stamps = pitch_object.xs()
    ...
    return pitch, time_stamps
```

3.2. Cơ chế thực thi của từng dòng lệnh

1. `parselmouth.Sound(audio_path)`: Nạp tệp WAV thành đối tượng âm thanh của Praat trong bộ nhớ C++. Thư viện `parselmouth` gọi trực tiếp vào thư viện liên kết động C++ nguyên bản của Praat, bảo đảm kết quả trùng khớp 100% với phần mềm Praat chạy trên máy tính.
2. Lệnh `snd.to_pitch()`: Tương đương với tùy chọn `Sound: To Pitch...` trên Praat với **toàn bộ 10 tham số mặc định**. Mã C++ kích hoạt hàm `Sound_to_Pitch_ac` với các tham số chuẩn của Bảng 1: time_step $=$ 0,01 s, pitch_floor $=$ 75 Hz, pitch_ceiling $=$ 600 Hz, 3 chu kỳ/khung, max_candidates $=$ 15, very_accurate $=$ off, silence_threshold $=$ 0,03, voicing_threshold $=$ 0,45, octave_cost $=$ 0,01, octave_jump_cost $=$ 0,35, voiced_unvoiced_cost $=$ 0,14.
3. `selected_array['frequency']`: Trích xuất ứng viên tối ưu toàn cục mà thuật toán quy hoạch động Viterbi đã chọn cho từng khung. Các khung vô thanh (Unvoiced) được gán giá trị 0. Hàm `np.nan_to_num` biến đổi mọi giá trị NaN tiềm ẩn thành số 0, thiết lập quy ước thống nhất xuyên suốt hệ thống: 0 **Hz** $\equiv$ **Vô thanh**.
4. `pitch_object.xs()`: Trả về mảng chứa các mốc thời gian vật lý tại tâm đối xứng của từng khung phân tích (tính bằng giây). Hàm có trả về mảng này, nhưng trong `main.py` dòng gọi lệnh là `pitch, _ = extract_features_praat(...)`, tức **mảng mốc thời gian thực tế đã bị vứt bỏ**.

3.3. Ánh xạ chi tiết từ Praat C++ sang lý thuyết tài liệu này

Bước thực thi bên trong `snd.to_pitch()`	**Vị trí trong tài liệu này**	**Công thức / Tham số liên kết**
Cắt khung 3 chu kỳ của 75 Hz $=$ 40 ms, bước trượt 10 ms	Mục 3.A	Quy tắc Boersma, Tham số 1–2
Nhân cửa sổ Hanning (do cờ `very_accurate = off`)	Mục 3.A	Công thức (5), Tham số 4
Tính tự tương quan $\mathcal{R}_x[\tau]$ của khung nhân cửa sổ	Mục 2	Công thức (1)
Bù suy hao cửa sổ bằng phép chia cho $\mathcal{R}_w[\tau]$	Mục 3.B	Công thức (6)–(9)
Giới hạn dải tìm kiếm đỉnh trong $[\tau_{\min}, \tau_{\max}]$	Mục 2	Công thức (3) và (4)
Trích xuất 14 đỉnh cực đại + 1 ứng viên Vô thanh	Mục 3.C’	Tham số 5, 6, 7 (ngưỡng 0,45 và 0,03)
Phạt log-lag nội khung để tránh lỗi nhảy $F_0/2$	Mục 3.C’	Công thức (14), Tham số 8 (`octave_cost = 0,01`)
Quy hoạch động Viterbi xuyên suốt câu nói	Mục 4.5	Tham số 9–10, Hình 4 (nhánh Praat)
Xuất giá trị cao độ $F_0 = f_s/\tau^*$	Mục 2 & Mục 4.2	Công thức (30), Thẻ công thức

Bảng 8: Đối chiếu chi tiết giữa lệnh gọi `snd.to_pitch()` và các công thức của tài liệu này.

Kết luận: Dòng mã `pitch_object = snd.to_pitch()` chính là sự hiện thực hóa trọn vẹn của Thẻ công thức tài liệu này, nhưng ở phiên bản phần mềm máy tính đầy đủ (sử dụng Viterbi 15 ứng viên, nội suy Sinc dưới mẫu, nhánh phần mềm trong Hình 4), chứ không phải phiên bản FPGA streaming rút gọn ở Mục 4 và Mục 6.

3.4. Chỗ mã nguồn khác lý thuyết và hệ quả kỹ thuật

(a) Tần số lấy mẫu thực tế 24 kHz (thay vì 16 kHz): Mô hình tổng hợp Kokoro (`generate_audio.py`) ghi tệp âm thanh ở tần số lấy mẫu $f_s = 24.000$ Hz (`sf.write(..., 24000)`). Hàm `trim_silence_and_save` nạp tệp với cờ `sr=None` (giữ nguyên tần số) rồi ghi lại. Do đó, Praat đo F_0 trên tín hiệu 24.000 Hz. Toàn bộ các thông số mẫu số học phải điều chỉnh tương ứng:

$$\tau_{\min} = \left\lceil \frac{24.000}{600} \right\rceil = \mathbf{40} \text{ mẫu}, \qquad \tau_{\max} = \left\lfloor \frac{24.000}{75} \right\rfloor = \mathbf{320} \text{ mẫu} \tag{36}$$

$$L = 0{,}040 \times 24.000 = \mathbf{960} \text{ mẫu}, \qquad \Delta t_{\text{hop}} = 0{,}01 \times 24.000 = \mathbf{240} \text{ mẫu} \tag{37}$$

So với thiết kế chuẩn 16 kHz trong tài liệu: dải lag mở rộng từ $[27, 213]$ (187 lag) lên $[40, 320]$ (281 lag); kích thước khung tăng từ 640 lên 960 mẫu. **Thời lượng vật lý** của khung (40 ms) và dải **tần số** (75–600 Hz) hoàn toàn giữ nguyên, chỉ có số lượng mẫu rời rạc tỉ lệ thuận với f_s.

Hệ quả tính toán: Một đỉnh tương quan tại $\tau^* = 200$ mẫu trong dự án này sẽ cho cao độ:

$$F_0 = \frac{24.000}{200} = \mathbf{120}\ \text{Hz}$$

Nếu áp nhầm công thức 16 kHz của tài liệu, lag 200 sẽ cho $16.000/200 = 80$ Hz. Do đó, **tuyệt đối không được lấy bảng tra lag của tài liệu áp vào dữ liệu của dự án này**.

Hệ quả về phi tuyến lượng tử hóa: Tại $f_s = 24$ kHz, bước nhảy 1 lag ở trần tần số cao là $24.000/40 - 24.000/41 \approx 14{,}6$ Hz (mịn hơn rõ rệt so với mức 22 Hz ở 16 kHz). Praat sử dụng phép nội suy Sinc dưới mẫu (sub-sample sinc interpolation) để tìm cực trị trên miền liên tục, trong khi thiết kế FPGA sử dụng nội suy parabol 3 điểm số nguyên (Chế độ B, kết hợp bộ lọc trung vị median-3 trễ 10 ms), cả hai phương pháp đều triệt tiêu hoàn toàn hiện tượng bậc thang hóa ở vùng cao tần.

(b) Có quy hoạch động Viterbi và 15 ứng viên: Khác với thiết kế FPGA streaming chỉ chọn cực đại tức thời cục bộ (1 ứng viên), Praat trong mã nguồn Python duyệt toàn bộ câu nói qua ma trận 15 ứng viên. Hệ quả là đường Pitch không bị gãy đoạn và miễn nhiễm với lỗi nhảy quãng tám, nhưng bắt buộc phải có đủ toàn bộ câu âm thanh trong bộ nhớ mới trích xuất được kết quả.

(c) Thiếu bước lọc thông cao (HPF) 60 Hz: Trong Mục 4 (khung "Cổng kiểm chứng phần mềm"), tài liệu yêu cầu tín hiệu trước khi tính tương quan phải đi qua bộ lọc HPF 60 Hz (hoặc DC-blocker) để dập tắt trôi DC và nhiễu cơ học. Mã nguồn Python đưa thẳng tín hiệu tổng hợp vào Praat mà không lọc. Do Kokoro là âm thanh tổng hợp sạch nên ảnh hưởng không lớn, nhưng đây là một điểm lệch quy trình so với chuẩn thiết kế vi mạch.

(d) Giải phẫu mã nguồn C++ của Praat (`Sound_to_Pitch.cpp` và `Pitch.cpp`).

Như đã phân tích tại Mục 3.D, mã nguồn C++ của Praat là phần mềm nguồn mở. Hai tệp nguồn `fon/Sound_to_Pitch.cpp` cùng `fon/Pitch.cpp` làm sáng tỏ cơ chế tính toán thực tế của thuật toán:

Khẳng định 1 — Praat không tự lấy mẫu lại. Hàm `Sound_to_Pitch_any` đọc trực tiếp mảng mẫu `my z[channel][i]` của đối tượng `Sound` và dùng `my dx` (nghịch đảo tần số lấy mẫu) trong mọi phép quy đổi thời gian. Không có lời gọi lấy mẫu lại nào trên đường đi. Hệ quả là ở tần số 24 kHz của dự án Kokoro, cửa sổ phân tích là 960 mẫu chứ không phải 640, đúng như phân tích ở Mục 3.4.(a).

Khẳng định 2 — Praat tính tự tương quan bằng FFT, không bằng vòng lặp thời gian. Với phương pháp mặc định (`AC_HANNING`, tương ứng `very_accurate = off`), mã nguồn đi theo nhánh FFT. Chuỗi thao tác, đọc đúng theo thứ tự trong hàm `Sound_to_Pitch_autoCorrelate`:

1. **Trừ trung bình cục bộ (khử DC).** Mã nguồn tính `localMean` trên một cửa sổ dài hai chu kỳ dài nhất, rồi trừ nó khỏi từng mẫu *trước khi* nhân cửa sổ: `frame[j] = (z[i] - localMean) * window[j]`. Chú thích trong mã ghi rõ: "We are going to kill the DC component before windowing."
2. **Nhân cửa sổ Hanning.** Cửa sổ được dựng bằng công thức:
   `window[i] = 0.5 - 0.5 * cos(2*pi*i/(nsamp_window+1))`
   — hoàn toàn trùng khớp công thức bảng ROM $N + 1 = 641$ trong sách (với $N = 640$).
3. **Đệm số 0 lên lũy thừa của 2.** Mã nguồn tính `nsampFFT` bằng cách nhân đôi đến khi vượt `nsamp_window*(1+depth)`. Với 638 mẫu và độ sâu nội suy 0,5, ngưỡng là 957, nên `nsampFFT = 1024`. Mục đích: triệt tiêu hiệu ứng cuộn vòng biên của biến đổi Fourier.
4. **FFT thực, bình phương biên độ, IFFT.** Định lý Wiener–Khinchin: gọi hàm `NUMfft_forward`, tính $|X[k]|^2 = \text{re}^2 + \text{im}^2$ dồn vào mảng `ac`, rồi gọi `NUMfft_backward`. Kết quả `ac` chính là hàm tự tương quan.
5. **Chia cho tự tương quan của cửa sổ.** Mã nguồn tính sẵn `windowR` bằng đúng chuỗi FFT → bình phương → IFFT trên chính cửa sổ, chuẩn hóa nó, rồi chia: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])`. Đây là phép bù suy hao $C[\tau]$ mà sách đã mô tả, nhưng Praat thực hiện bằng phép *chia* chứ không bằng bảng tra.
6. **Nội suy sinc dưới mẫu.** Sau khi có đỉnh nguyên, mã nguồn dùng `NUM_interpolate_sinc` với độ sâu 30 để tìm cực trị trên miền liên tục, rồi tính tần số bằng `frequencyOfMaximum = 1.0 / my dx / (i + dr/d2r)`.

Vì sao hai cách tính cho cùng một kết quả. Điểm cần nắm: phương pháp FFT và phương pháp tổng thời gian mà FPGA cài đặt **không phải hai thuật toán gần giống nhau**, mà là hai cách tính của *cùng một đại lượng*. Với đệm số 0 đủ dài, tương quan vòng của FFT không còn chồng lấn, nên kết quả tại các độ trễ nguyên bằng đúng tổng thời gian. Phép kiểm tra số học trong quá trình soạn tài liệu xác nhận điều này: với cửa sổ 638 mẫu, `NFFT = 1024` cho sai lệch cỡ 10^{-14} so với tổng trực tiếp, trong khi `NFFT = 512` cho sai lệch cỡ 10^{2} vì chồng phổ vòng. Ngưỡng an toàn là `NFFT` $\geq$ độ dài cửa sổ cộng độ trễ lớn nhất, tức $638 + 319 = 957$; 1024 vượt ngưỡng đó.

Điều này có hệ quả thực hành cho người đọc: khi đối chiếu bản Python số nguyên với Praat, sai lệch còn lại **không** đến từ việc Praat "tính khác". Nó đến từ bốn chỗ đã nêu ở trên — cửa sổ dài hơn, phép chia thay bảng tra, nội suy sinc, và Viterbi — chứ không phải từ công thức tự tương quan.

3.5. Bản chất trục thời gian của mảng `pitch`

Với bước nhảy trượt $\Delta t_{\text{hop}} = 10$ ms, một câu nói dài D giây sẽ tạo ra khoảng $100D$ điểm cao độ. Vì độ dài khung là 40 ms, tâm đối xứng của khung phân tích đầu tiên không nằm ở $t = 0$ mà lùi về sau khoảng nửa khung ($\approx$ 20 ms). Đây chính là hiện tượng "độ lệch tâm pha" đã phân tích ở Hình 6. Mặc dù hàm `pitch_object.xs()` trả về chính xác tọa độ thời gian này, việc mã nguồn bên ngoài bỏ qua `xs()` sẽ dẫn đến sai lệch căn chỉnh được chỉ ra ở Mục 5.1.

4. `trim_silence_and_save`: Cắt Khoảng Lặng Hai Đầu

4.1. Mã nguồn

```
y, sr = librosa.load(audio_path, sr=None)
y_trimmed, _ = librosa.effects.trim(y, top_db=top_db)   # top_db = 30
if not len(y_trimmed) or not np.any(y_trimmed):
    raise ValueError(...)
sf.write(trimmed_path, y_trimmed, sr)
```

4.2. Cơ chế vật lý

Hàm `librosa.effects.trim` chia tín hiệu thành các khung ngắn (mặc định 2048 mẫu, bước nhảy 512 mẫu), tính năng lượng hiệu dụng RMS của từng khung và đổi sang thang decibel (dB) quy chiếu theo khung có năng lượng lớn nhất trong toàn bộ tệp. Khung nào có năng lượng thấp hơn đỉnh cực đại quá top_db $= 30$ dB sẽ bị coi là khoảng lặng. Hàm chỉ thực hiện cắt bỏ **phần lặng ở đầu câu và cuối câu**, hoàn toàn không tác động vào các khoảng lặng hoặc ngắt nghỉ giữa câu.

Ý nghĩa định lượng của con số 30 *dB:*

$$10^{-30/20} = 10^{-1,5} \approx \mathbf{0,0316}$$

Nghĩa là tín hiệu bị xem là lặng khi biên độ tức thời rơi xuống dưới mức 3,2% so với biên độ đỉnh lớn nhất của câu nói.

4.3. Đối chiếu giữa `top_db = 30` của Librosa và `silence_threshold = 0,03` của Praat

Tiêu chí so sánh	**`top_db = 30` (`librosa.effects.trim`)**	**`silence_threshold = 0,03` (Praat Tham số 6)**
Tác động xử lý	Cắt bỏ hoàn toàn các mẫu dữ liệu ở hai đầu tệp	Giữ nguyên mẫu dữ liệu, gán nhãn khung là Vô thanh ($F_0 = 0$)
Phạm vi áp dụng	Chỉ tác động ở phần đầu và phần cuối câu	Quét trên toàn bộ các khung xuyên suốt câu nói
Đại lượng so sánh	Năng lượng hiệu dụng RMS khung so với RMS đỉnh (dB)	Tỷ số biên độ đỉnh cục bộ khung so với đỉnh toàn tệp

Bảng 9: So sánh bản chất giữa việc cắt lặng bằng Librosa và ngưỡng lặng của Praat.

Mục đích thực chất của bước cắt lặng: Cắt khoảng lặng không thể thay thế cơ chế nhận diện thanh tính của Praat; các phụ âm vô thanh và khoảng ngắt giữa câu vẫn được Praat gắn nhãn $F_0 = 0$. Mục đích duy nhất của `trim_silence_and_save` là **đồng bộ gốc thời gian**: hai giọng đọc khác nhau luôn có thời gian chuẩn bị phát âm đầu câu chênh lệch nhau. Việc cắt bỏ đoạn lặng đầu giúp mốc $t = 0$ của cả hai giọng áp sát vào thời điểm bật âm đầu tiên, tạo điều kiện thuận lợi cho phép căn chỉnh theo từ tiếp theo.

5. Căn Chỉnh Hai Đường Cao Độ Theo Từng Từ

Phụ lục 5.1. `get_local_chunk`: Cắt đoạn cao độ của một từ

```
i1 = int((w_start / total_dur) * N)
i2 = int((w_end / total_dur) * N)
return raw_array[i1:i2]
```

Hàm chuyển đổi mốc thời gian của từ ($w_{\text{start}}, w_{\text{end}}$, tính bằng giây do Whisper cung cấp) thành chỉ số mảng trong vector cao độ `pitch` theo **phép tính tỉ lệ thuận**:

$$i = \left\lfloor \frac{t}{D} \cdot N \right\rfloor \tag{38}$$

với D là tổng thời lượng tệp âm thanh đã cắt ($D = \text{len}(y)/\text{sr}$) và N là tổng số điểm trong mảng `pitch`.

Ví dụ số: Giả sử $D = 2{,}0$ s, mảng có $N = 197$ điểm. Từ "hello" phát âm từ $t = 0{,}50$ s đến $0{,}90$ s:

$$i_1 = \lfloor (0{,}50/2{,}0) \times 197 \rfloor = \lfloor 49{,}25 \rfloor = 49, \qquad i_2 = \lfloor (0{,}90/2{,}0) \times 197 \rfloor = \lfloor 88{,}65 \rfloor = 88$$

Đoạn trích xuất gồm $88 - 49 = 39$ điểm cao độ.

Sai lệch so với lý thuyết (Do bỏ mảng `xs()`): Phép tính tỉ lệ trên ngầm giả định các điểm cao độ phủ đều từ $t = 0$ đến $t = D$. Trên thực tế, tâm đối xứng của khung thứ i nằm tại $t_i \approx 0{,}02 + i \times 0{,}01$ s (độ lệch tâm pha ban đầu 20 ms). Điểm thứ 49 trong thực tế nằm tại thời điểm $0{,}02 + 49 \times 0{,}01 = 0{,}51$ s, chứ không phải $0{,}50$ s. Sai lệch này luôn nhỏ hơn nửa khung (≤ 20 ms, tương đương ≤ 2 điểm cao độ), và do xảy ra đồng thời trên cả hai tệp âm thanh nên phần lớn sai số tự triệt tiêu lẫn nhau.

5.2. `equalize_local_chunk`: Co giãn đoạn cao độ bằng phép nội suy

```
x_ref  = np.linspace(0, 1, len(ref_chunk))
x_user = np.linspace(0, 1, len(user_chunk))
return np.interp(x_ref, x_user, user_chunk)
```

Hai đoạn cao độ của cùng một từ có thời lượng phát âm khác nhau được đưa về cùng trục chuẩn hóa $[0, 1]$ (vị trí bắt đầu từ ứng với 0, kết thúc từ ứng với 1). Sau đó hàm `np.interp` thực hiện **nội suy tuyến tính** giá trị F_0 của người đọc tại các tọa độ chuẩn hóa của giọng tham chiếu. Kết quả là đoạn cao độ của người đọc được co giãn về đúng số điểm của giọng tham chiếu. Đây là giải pháp đơn giản hóa thay thế cho thuật toán Dynamic Time Warping (DTW) phức tạp.

▶ NHÌN THẤY VẬT LÝ

Lỗi tiềm ẩn nghiêm trọng: Nội suy qua giá trị 0 Hz sinh ra F_0 giả: Hàm `np.interp` thuần túy xem mảng số liệu là các đại lượng hình học liên tục, hoàn toàn không biết con số 0 mang ý nghĩa đặc biệt là "Vô thanh". Tại ranh giới giữa âm vô thanh và âm hữu thanh (ví dụ hai điểm kế tiếp nhau có giá trị $[0, 200]$ Hz), phép nội suy tại điểm ở giữa sẽ gán giá trị 100 Hz.

Đây là một **giá trị cao độ giả mạo (spurious pitch)**: không hề có dao động dây thanh quản nào diễn ra ở tần số 100 Hz tại thời điểm đó! Vì $100 > 0$, điểm giả này lọt qua bộ lọc kiểm tra tính hữu thanh ở Mục 6, tạo ra một điểm sụt dốc đột ngột bất thường và làm suy giảm hệ số tương quan Pearson r. Cách khắc phục chuẩn mực là phải gắn cờ NaN cho các điểm vô thanh hoặc thực hiện nội suy riêng biệt trên từng đoạn hữu thanh liên tục.

5.3. Ghép nối thành đường cao độ toàn câu (`stitched_user_pitch`)

Trong `visualizer.py`, hàm `plot_analysis` khởi tạo mảng `stitched_user_pitch` toàn số 0 có độ dài bằng mảng tham chiếu N_{ref}. Sau đó, với mỗi từ khớp thành công giữa văn bản chuẩn và kết quả Whisper (khớp bằng `difflib.SequenceMatcher`, cờ `equal`), đoạn cao độ đã co giãn được dán đè vào đúng vị trí chỉ số của từ đó bên mảng tham chiếu. Nhờ đó, đường tham chiếu chuẩn được giữ nguyên vẹn, các khoảng ngừng giữa các từ hoặc các từ phát âm sai bị gán giá trị 0 và tự động bị loại khỏi phép tính tương quan.

6. `dynamic_sliding_window_pearson`: Tính Hệ Số Pearson Trên Các Cửa Sổ Trượt

6.1. Mã nguồn chính

```
W = max(6, int((2 * N) / (num_words + 2)))
if W % 2 != 0: W -= 1
S = max(1, W // 2)
num_slides = num_words + 1
...
valid = (u_win > 0) & (r_win > 0)
if np.sum(valid) >= 3 and np.std(u_win[valid]) > 0 and np.std(r_win[valid]) > 0:
    r = np.corrcoef(u_win[valid], r_win[valid])[0, 1]
else:
    r = np.nan
```

6.2. Nguồn gốc toán học của kích thước cửa sổ W

Gọi $K = \texttt{num_words}$ là số từ trong câu chuẩn, và N là tổng số điểm cao độ tham chiếu. Thuật toán tạo ra đúng $K+1$ cửa sổ trượt có kích thước W, với bước trượt gối đầu nhau một nửa: $S = W/2$. Tổng phạm vi thời gian mà $K+1$ cửa sổ này bao phủ được xác định bởi:

$$\underbrace{K \cdot S}_{\text{Tọa độ bắt đầu của cửa sổ cuối}} + W = K \cdot \frac{W}{2} + W = \frac{(K+2)W}{2} \tag{39}$$

Để toàn bộ các cửa sổ phủ trọn vẹn toàn bộ câu nói dài N điểm, ta đặt tổng này bằng N:

$$\frac{(K+2)W}{2} = N \implies W = \frac{2N}{K+2} \tag{40}$$

Ý nghĩa ngữ nghĩa học: Vì N/K là số điểm trung bình của một từ, kích thước $W \approx 2 \times (N/K)$ tương đương với khoảng thời lượng của **hai từ liên tiếp**. Nhờ đó, mỗi cửa sổ trượt bao trọn ngữ điệu của một cụm từ nhỏ. Mã nguồn ép W chẵn để bước trượt $S = W/2$ là số nguyên, đặt ngưỡng chặn dưới $W \geq 6$, và nới rộng cửa sổ cuối cùng chạm mốc N để bù trừ sai số làm tròn.

6.3. Công thức tính Pearson r và tính chất bất biến quãng tám

Trong mỗi cửa sổ, thuật toán áp dụng mặt nạ lọc `valid`: chỉ giữ lại các vị trí mà **cả hai đường đều hữu thanh** ($u_i > 0$ và $v_i > 0$). Với M cặp điểm hợp lệ (u_i, v_i), hệ số tương quan tuyến tính Pearson được tính theo công thức:

$$r_{\text{Pearson}} = \frac{\sum_{i=1}^{M}(u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^{M}(u_i - \bar{u})^2} \cdot \sqrt{\sum_{i=1}^{M}(v_i - \bar{v})^2}} \tag{41}$$

Hàm `np.corrcoef(...)[0, 1]` trả về giá trị này. Cửa sổ bị gán NaN khi có ít hơn 3 điểm hợp lệ ($M < 3$) hoặc khi đường cao độ nằm phẳng tuyệt đối ($\sigma = 0$, mẫu số triệt tiêu).

▶ NHÌN THẤY VẬT LÝ

Vì sao Pearson r rất phù hợp để so giọng nam (`michael`) với giọng nữ (`sarah`): Hệ số tương quan Pearson có tính chất toán học quan trọng: **Bất biến đối với các phép biến đổi afin dương** ($r(X, aY+b) = r(X,Y)$ với mọi $a > 0$). Giả sử giọng nam `michael` phát âm trầm hơn giọng nữ `sarah` đúng một quãng tám ($F_{0,\text{nam}} = 0{,}5 \times F_{0,\text{nữ}}$), nhưng hình thái luyến láy lên xuống hoàn toàn trùng khớp:

$$u_i = 0{,}5 \cdot v_i \implies r_{\text{Pearson}} = \mathbf{1{,}0}$$

Ngược lại, nếu người đọc nhấn ngữ điệu đi xuống trong khi mẫu đi lên, hệ số sẽ đổi dấu thành âm ($r < 0$). Do đó, Pearson r đóng vai trò là một bộ đo **hình thái giai điệu ngữ điệu (intonation contour shape)**, chứ không đo độ cao tuyệt đối của giọng nói.

7. `summarize_pearson` và `summarize_sentence_reports`: Từ Cửa Sổ Đến Điểm Số

7.1. Tổng hợp điểm số cho một câu

```
valid = values[np.isfinite(values)]
'mean_r': float(np.maximum(valid, 0).mean())
```

Quy trình tính điểm `mean_r` của một câu gồm 3 bước:

1. Loại bỏ toàn bộ các cửa sổ có giá trị NaN (`missing_r_policy = 'exclude'`).
2. Đổi toàn bộ các giá trị tương quan âm ($r \le 0$) thành 0 (`nonpositive_r_policy = 'zero'`). Triết lý thiết kế: tương quan âm hay bằng 0 đều phản ánh việc phát âm sai ngữ điệu, không cần thiết phải phạt thêm giá trị âm.
3. Tính trung bình cộng của các giá trị sau khi kẹp, cho ra điểm số `mean_r` nằm trong khoảng $[0, 1]$.

Ví dụ số: Một câu có 9 cửa sổ cho kết quả: $[0{,}9;\ 0{,}8;\ \text{NaN};\ -0{,}4;\ 0{,}7;\ 0{,}6;\ \text{NaN};\ 0{,}85;\ 0{,}75]$.

- Loại 2 cửa sổ NaN $\implies$ còn 7 cửa sổ hợp lệ (`valid_r_count = 7`).
- Kẹp giá trị âm: $-0{,}4 \to 0$.
- Tổng tích lũy: $0{,}9 + 0{,}8 + 0 + 0{,}7 + 0{,}6 + 0{,}85 + 0{,}75 = 4{,}6$.
- Điểm trung bình: `mean_r` $= 4{,}6/7 \approx \mathbf{0{,}657}$. (Nếu không kẹp về 0, điểm trung bình thô chỉ đạt $(4{,}6 - 0{,}4)/7 = 0{,}600$).

7.2. Thống kê báo cáo nhiều câu (`summarize_sentence_reports`)

Hàm đếm số lượng câu thành công (trạng thái `ok`) đạt điểm `mean_r` vượt qua các ngưỡng từ $0{,}9$ xuống $0{,}3$ (`range(9, 2, -1)` chia 10). Các ngưỡng này mang tính chất **tích lũy (cumulative)**: một câu đạt $0{,}9$ sẽ đương nhiên được đếm trong tất cả các mốc thấp hơn ($\ge 0{,}8, \ldots, \ge 0{,}3$).

8. `evaluator.py`: Nhận Diện Mốc Thời Gian Bằng Whisper

8.1. Mã nguồn chính

```
torch.set_num_threads(min(4, torch.get_num_threads()))
whisper.load_model('base', ...)
...
audio, sr = sf.read(audio_path, dtype='float32')
if audio.ndim > 1: audio = audio.mean(axis=1)
if sr != 16000: audio = librosa.resample(audio, orig_sr=sr, target_sr=16000)
result = model.transcribe(audio, word_timestamps=True, fp16=False, language='en')
return [dict(word=normalize_word(w['word']), start=w['start'], end=w['end']) ...]
```

8.2. Các bước thực thi cốt lõi

1. `load_stt_model`: Khóa số luồng tính toán CPU tối đa là 4 để tránh hiện tượng tranh chấp tài nguyên (thread contention) khi xử lý các đoạn âm thanh ngắn, sau đó nạp mô hình Whisper kiến trúc `base`.
2. `get_user_timestamps`: Đọc tệp WAV đã cắt lặng, chuyển stereo thành mono bằng phép lấy trung bình cộng hai kênh, và **lấy mẫu lại tín hiệu từ** 24 **kHz về đúng** 16 **kHz** (chuẩn đầu vào bắt buộc của mạng nơ-ron Whisper).
3. Thực thi nhận dạng tiếng nói (`model.transcribe`) với cờ `word_timestamps=True` để trích xuất mốc thời gian chi tiết của từng từ, chạy ở chế độ số thực 32-bit (`fp16=False`).
4. Hàm `normalize_word` loại bỏ toàn bộ dấu câu và chuyển thành chữ thường để đối soát với văn bản chuẩn.

8.3. Sự phân công kiến trúc hai luồng song song

Con số 16 kHz trong `evaluator.py` trùng với tần số f_s trong luồng Mel ASR, nhưng chỉ phục vụ cho mạng nơ-ron Whisper. Whisper biến đổi âm thanh 16 kHz thành vector **Log-Mel Spectrogram** để nhận diện từ — chính là loại đặc trưng đã xóa bỏ cao độ. Ngược lại, luồng đo cao độ Praat giữ nguyên tín hiệu 24 kHz để bảo toàn độ phân giải chu kỳ thanh quản. Đây là minh chứng thực tế rõ nét cho mô hình **Hai Luồng Xử Lý Song Song** mà tài liệu đã đề xuất trong tài liệu này, phần 0.

8.4. Có thể thay Whisper bằng thuật toán nhẹ hơn không?

Vì sao câu hỏi này đáng hỏi. Trong bài toán chấm điểm phát âm, **văn bản chuẩn đã biết trước 100%**. Người học đang đọc đúng một câu cho sẵn. Vậy mà quy trình hiện tại vẫn chạy một mô hình nhận dạng tiếng nói đầy đủ để *đoán lại* xem người ta vừa nói gì — trong khi ta đã biết câu trả lời. Whisper ở đây chỉ được dùng để lấy mốc thời gian $(w_{\text{start}}, w_{\text{end}})$ của từng từ, và nó làm việc đó gián tiếp: nhận dạng chữ trước, rồi suy ra thời gian.

Đây là một sự lệch pha giữa công cụ và nhiệm vụ. Whisper là mô hình lớn (hàng trăm triệu tham số), cần GPU để chạy nhanh, và có thể nhận dạng sai chữ — kéo theo mốc thời gian sai theo. Với một bài toán mà đáp án đã biết, có hai cách rẻ hơn và chính xác hơn.

Phương án 1 — Căn chỉnh cưỡng bức (Forced Alignment). Thay vì hỏi "người này nói gì", hỏi thẳng "câu đã biết này khớp vào đoạn âm thanh này ở đâu". Đó chính là định nghĩa của forced alignment: cho trước cả tín hiệu âm thanh *và* chuỗi âm vị của văn bản chuẩn, thuật toán tìm ranh giới thời gian của từng âm vị.

Các công cụ có sẵn:

- **Montreal Forced Aligner (MFA):** dùng mô hình âm học (thường là GMM-HMM hoặc mạng nơ-ron nhỏ) cộng từ điển phát âm. Độ chính xác ranh giới từ thường dưới 20 ms.
- **Kaldi / Vosk:** cùng nguyên lý, có bản nhẹ chạy được trên CPU.
- **`torchaudio.pipelines.MMS_FA`:** mô hình wav2vec2 huấn luyện riêng cho căn chỉnh, chạy được trên CPU, chỉ cần vài chục mili-giây cho một câu ngắn.

Ưu điểm so với Whisper: vì đã biết văn bản, thuật toán **không thể "nhận dạng sai"** — nó chỉ có thể đặt ranh giới lệch. Sai số được giới hạn ở trục thời gian, không lan sang nội dung.

Phương án 2 — Dynamic Time Warping trên chuỗi 80 Log-Mel. Cách này bỏ luôn cả bước nhận dạng. Ta đã có ma trận 80 dải Log-Mel của câu mẫu và của người học (đều tính trên máy chủ từ tệp WAV sạch do FPGA chuyển lên qua Shared Folder). Dynamic Time Warping (DTW — phép uốn thời gian động) tìm một ánh xạ giữa hai chuỗi khung sao cho tổng khoảng cách là nhỏ nhất, cho phép co giãn trục thời gian.

Cụ thể, với hai chuỗi vector đặc trưng $A = [a_1, \ldots, a_M]$ (câu mẫu) và $B = [b_1, \ldots, b_N]$ (người học), DTW dựng ma trận chi phí D với

$$D[i][j] = \|a_i - b_j\|^2 + \min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big),$$

rồi truy vết ngược từ $D[M][N]$ để lấy đường đi tối ưu. Đường đi đó chính là ánh xạ khung-thời-gian, và từ đó suy ra mốc của từng từ bằng cách ánh xạ các mốc đã biết của câu mẫu.

Ưu điểm: **không cần mô hình nhận dạng nào cả**, chỉ cần ma trận đặc trưng tính từ tệp âm thanh sạch. Nhược điểm: DTW không biết ngữ nghĩa, nên nếu người học đọc sai hẳn một từ (nói nhầm từ khác), đường đi sẽ bị kéo lệch ở chỗ đó.

So sánh ba lựa chọn.

	Whisper (hiện tại)	**Forced Alignment**	**DTW trên Mel**
Cần mô hình nhận dạng	Có (lớn)	Không	Không
Cần văn bản chuẩn	Không	Có	Không (chỉ cần câu mẫu)
Sai số ranh giới từ	Ở mức từ, phụ thuộc nhận dạng	< 20 ms	Phụ thuộc độ dài câu
Chạy được trên CPU	Chậm	Có	Có
Rủi ro chính	Nhận dạng sai chữ	Cần từ điển phát âm	Lệch khi đọc sai từ

Kết luận thực hành. Với bài toán chấm phát âm có văn bản cho trước, **forced alignment là lựa chọn mặc định hợp lý**: nó rẻ hơn Whisper, chính xác hơn về ranh giới, và loại bỏ hẳn một nguồn lỗi. DTW là lựa chọn hấp dẫn khi muốn bỏ cả mô hình nơ-ron, đặc biệt vì nó dùng đúng ma trận 80 Log-Mel tính từ tệp WAV sạch mà FPGA đã lọc và chuyển lên Shared Folder. Whisper vẫn có chỗ đứng trong một trường hợp: khi người học đọc một câu *không* cho trước, hoặc khi cần kiểm tra xem họ có đọc đúng từ hay không — lúc đó việc nhận dạng chữ chính là mục tiêu, chứ không còn là phương tiện.

9. Bảng Tổng Hợp So Sánh Toàn Diện: Lý Thuyết tài liệu này vs. Mã Nguồn Python

Đặc tính kỹ thuật	**Thiết kế Lý thuyết tài liệu này**	**Mã nguồn Python (`Pitch-Project`)**	**Đánh giá & Hệ quả kỹ thuật**
Thuật toán đo cao độ	Tự cài đặt Công thức (1), (7)–(9)	Gọi thư viện `parselmouth` (`to_pitch`)	Dùng trực tiếp mã nguồn C++ Praat chuẩn
Tần số lấy mẫu đo F_0	$f_s = 16.000$ Hz	$f_s = 24.000$ Hz (do Kokoro TTS sinh ra)	Dải lag đối thành $[40, 320]$ mẫu, $L = 960$ mẫu
Dải tần số tìm kiếm	75–600 Hz	75–600 Hz (tham số Praat mặc định)	Trùng khớp hoàn toàn
Kích thước khung / Bước trượt	40 ms / 10 ms (640 / 160 mẫu)	40 ms / 10 ms (960 / 240 mẫu)	Trùng khớp thời lượng vật lý
Dạng cửa sổ phân tích	Hanning đối xứng	Hanning đối xứng (`very_accurate = off`)	Trùng khớp dạng cửa sổ
Ngưỡng thanh tính / Ngưỡng lặng	$\theta_{\text{voice}} = 0{,}45$; $\theta_{\text{silence}} = 0{,}03$	Mặc định Praat (0,45 và 0,03)	Trùng khớp tuyệt đối
Cơ chế trích chọn cực đại	FPGA: 189 lag, nội suy parabol, median-3 (trễ 10 ms)	15 ứng viên + Viterbi + Nội suy Sinc	Đạt độ mịn cao tần, loại bỏ bậc thang
Tiền xử lý lọc thông cao	Bắt buộc bộ lọc HPF 60 Hz / DC-blocker	Không có bước lọc nào trước Praat	Lệch quy trình, chấp nhận được với TTS
Tiền xử lý cắt khoảng lặng	So sánh năng lượng khung tĩnh	Dùng `librosa.effects.trim` (30 dB) 2 đầu	Nhằm đồng bộ gốc thời gian $t = 0$
Phương pháp đối sánh cao độ	Chuẩn bị đưa lên vi mạch FPGA	Pearson r trên cửa sổ trượt ≈ 2 từ	Đo hình thái giai điệu ngữ điệu
Tần số lấy mẫu nhận diện	16 kHz (tệp WAV sạch do FPGA xuất)	16 kHz (lấy mẫu lại đưa vào Whisper)	Hiện thực đúng mô hình hai luồng song song

Bảng 10: Bảng tổng hợp đối chiếu toàn diện giữa thiết kế lý thuyết tài liệu này và mã nguồn thực tế.

10. Danh Sách Các Điểm Lệch Và Lỗi Tiềm Ẩn Cần Lưu Ý

1. **Vứt bỏ mảng mốc thời gian thực `xs()` của Praat:** Việc tính chỉ số khung theo tỉ lệ tuyến tính $i = \lfloor (t/D) \cdot N \rfloor$ bỏ qua độ lệch tâm pha ban đầu 20 ms, gây ra sai số thời gian tối đa cỡ nửa khung (≤ 2 điểm cao độ).

2. **Hiện tượng sinh F_0 giả khi nội suy qua điểm 0 Hz:** Hàm `np.interp` nội suy tuyến tính giữa các điểm vô thanh (0 Hz) và hữu thanh (> 0 Hz), tạo ra các giá trị tần số giả mạo lọt qua bộ lọc thanh tính và làm suy giảm nhân tạo hệ số tương quan Pearson r.

3. **Lệch tần số lấy mẫu (24 kHz vs 16 kHz):** Mọi bảng tra lag, địa chỉ ROM và số lượng phép tính MAC trong Mục 5 của tài liệu không thể áp dụng trực tiếp cho dữ liệu Kokoro mà phải nhân theo tỉ lệ $24/16 = 1{,}5$.

4. **Bỏ qua bộ lọc HPF 60 Hz:** Mã nguồn đưa tín hiệu thô vào phân tích mà không qua khâu triệt tiêu trôi DC như yêu cầu của Cổng kiểm chứng phần mềm.

5. **Kẹp giá trị tương quan không dương ($r \leq 0$) về 0:** Phép kẹp này làm điểm trung bình `mean_r` của câu tăng cao hơn so với trung bình số học thực tế.

6. **Tên tệp `evaluator.py` gây hiểu lầm:** Tệp này không thực hiện đánh giá hay chấm điểm mà chỉ là bộ trích xuất mốc từ dựa trên Whisper.

7. **Tính Pearson trên thang tần số Hertz tuyến tính:** Hệ số Pearson tính trực tiếp trên đơn vị Hz mà không đổi sang semitone (thang logarit), khiến các biến động ở dải tần số cao chịu trọng số ảnh hưởng lớn hơn dải trầm.

8. **Sai số làm tròn số thực trong thống kê ngưỡng:** Phép chia số thực `threshold / 10` trong Python có thể sinh ra phần dư làm tròn vô cùng nhỏ, cần lưu ý khi xây dựng các bài test kiểm thử tự động (Unit Tests).

11. Nhật Ký Chạy Kiểm Thử Thực Nghiệm (`run_log_v6.txt`)

Toàn bộ các con số và bảng tra trong tài liệu được đối soát trực tiếp từ quá trình chạy thực tế của các tập lệnh `lab_1_3_5.py`, bộ kiểm thử `test_lab_1_3_5.py` và `verify_tiers.py`:

```
============================================================
1. python lab_1_3_5.py --selftest
============================================================
=== SELFTEST lab_1_3_5.py ===
H[0..4]: [1, 3, 7, 13, 20]
C[26..30]: [8283, 8287, 8291, 8296, 8301]
C[26..30] hex: ['205b', '205f', '2063', '2068', '206d']
C[214]: 17070 (hex: 42ae)
golden x[0..4]: [0, 2817, 5550, 8117, 10443]
440 Hz S[35..37]: S[35]=31303452800, S[36]=32119378939, S[37]=31978033782
Parabolic mode B: tau_star=36, tau_fine_q16=2382387, f0_q12_4=7042 (0x1b82 = 440.125 Hz)
Parabolic float: 440.137 Hz (+0.54 c)
Octave Cost Sweep:
  cost=0.000: 300 / 526 failures
  cost=0.005: 20 / 526 failures
  cost=0.009: 0 / 526 failures
  cost=0.010: 0 / 526 failures
  cost=0.020: 0 / 526 failures
Sine Freq Accuracy (Mode B vs true freq):
   75 Hz: +1.44 c (F0 = 75.062 Hz)
  100 Hz: +1.08 c (F0 = 100.062 Hz)
  200 Hz: +0.54 c (F0 = 200.062 Hz)
  440 Hz: +0.49 c (F0 = 440.125 Hz)
  590 Hz: +0.55 c (F0 = 590.188 Hz)


============================================================
2. python lab_1_3_5.py --export-hex hex
============================================================
Exported .hex files to hex


============================================================
3. python -m pytest test_lab_1_3_5.py
============================================================
============================= test session starts =============================

platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\NGOC\AppData\Local\Programs\Python\Python312\python.exe

cachedir: .pytest_cache
rootdir: C:\Users\NGOC\Documents\learning_journey\voice_jetson_to_fpga_learning-journey
configfile: pyproject.toml
plugins: anyio-4.14.2
collecting ... collected 15 items

test_lab_1_3_5.py::test_window_rom_is_full_length_and_bounded PASSED [  6%]
test_lab_1_3_5.py::test_window_multiply_keeps_q15_scale PASSED [ 13%]
test_lab_1_3_5.py::test_compensation_table_is_monotone_and_fits_q3_13 PASSED [ 20%]
test_lab_1_3_5.py::test_scoring_rom_is_compensation_times_a_penalty_at_most_one PASSED [ 26%]
test_lab_1_3_5.py::test_integer_detector_matches_float_on_worked_example PASSED [ 33%]
test_lab_1_3_5.py::test_parabolic_interpolation_sub_cent_accuracy PASSED [ 40%]
test_lab_1_3_5.py::test_boundary_interpolation_590hz_and_75hz PASSED [ 46%]
test_lab_1_3_5.py::test_integer_detector_is_deterministic_across_repeated_runs PASSED [ 53%]
test_lab_1_3_5.py::test_silence_is_unvoiced PASSED              [ 60%]
test_lab_1_3_5.py::test_silence_gate_rejects_quiet_frame PASSED [ 66%]
test_lab_1_3_5.py::test_median3_removes_a_single_frame_octave_spike PASSED [ 73%]
test_lab_1_3_5.py::test_median3_preserves_monotone_run PASSED   [ 80%]
test_lab_1_3_5.py::test_octave_cost_sweep_matches_simulation PASSED [ 86%]
test_lab_1_3_5.py::test_export_hex_creates_valid_files PASSED   [ 93%]
test_lab_1_3_5.py::test_pitch_f0_q12_4_protocol PASSED          [100%]

============================== 15 passed in 1.67s ==============================


============================================================
4. python verify_tiers.py
```

```
============================================================
Kế hoạch Tier 3: testbench đọc golden_vectors.hex, mục tiêu 0 bit.
Tín hiệu               | Tier 1 (Float/Praat)       | Tier 2 (Int/Float)       | Tier 3 (RTL/Int)
-------------------------------------------------------------------------------------------------
Sin 75 Hz              | cent=0.41, V/UV=0.0%       | cent=0.49, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 100 Hz             | cent=0.41, V/UV=0.0%       | cent=0.46, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 200 Hz             | cent=0.43, V/UV=0.0%       | cent=0.10, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 440 Hz             | cent=0.46, V/UV=0.0%       | cent=0.03, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 590 Hz             | cent=0.46, V/UV=0.0%       | cent=0.09, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Im lặng                | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Nhiễu trắng -35dBFS    | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Âm xát /s/             | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Giọng nam (speech_a)   | cent=0.04, V/UV=8.2%       | cent=0.21, V/UV=1.0%     | Kế hoạch (mục tiêu 0 bit)
Giọng nữ (speech_b)    | cent=365.11, V/UV=27.8%    | cent=0.15, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
```